

CSE 220

Signals and Linear Systems

Convolution: 15 Practice Onlines

Written against the `DiscreteSignal` / `LTISystem` API
of the July 2025 offline on convolution

Problems	15 (12 new, 3 adapted from the January 2026 onlines)
Marks each	10
Time each	25–30 minutes
Prerequisite	a working <code>signal_lti.py</code> from the offline

How to use this set

Every problem is written to be attempted in a single sitting of 25–30 minutes, in the shape of the real online: a short setup, three to six tasks, ten marks, and one Python file to submit. Nothing here needs anything beyond the two classes you already wrote.

Ground rules

- Import `DiscreteSignal` and `LTISystem` from your offline `signal_lti.py`. Do not edit that file — if a problem needs a new method, subclass.
- No `numpy.convolve`, `scipy.signal.convolve`, `fftconvolve`, or any built-in convolution or correlation. NumPy for ordinary arithmetic is fine.
- **The templates contain no plotting code** — only a comment block describing what to draw and what to watch out for. Writing the Matplotlib calls is part of the exercise, so do that first. The solutions then give a complete worked implementation of every figure, together with the figure it produces, to check yours against.
- Signals do not start at $n = 0$. Ranges are computed, never hardcoded.

The API these problems assume

Method	Behaviour
<code>DiscreteSignal(start, end)</code>	zero signal on the inclusive range
<code>get_value_at_time(t)</code>	returns 0 outside the stored range
<code>set_value_at_time(t, v)</code>	raises <code>IndexError</code> outside the range
<code>shift(k)</code>	$x[n - k]$; <i>the range moves with it</i>
<code>add(other)</code>	sum over the <i>union</i> of the two ranges
<code>multiply(scalar)</code>	scaled copy, same range
<code>LTISystem.output(x)</code>	convolution sum, range computed automatically
<code>LTISystem.output_by_superposition(x)</code>	same result via component responses

The three adapted problems were originally written for the older `Signal(INF)` class, which stored every signal on one fixed window. They have been rewritten here, because that difference is not cosmetic: with `DiscreteSignal`, `shift` moves the range and `add` widens it, so range arithmetic becomes part of the answer. Problem 4 contains a worked example of a truncation artifact that the old class hid completely.

Files

<code>common/signal_lti.py</code>	your offline classes, unchanged
<code>templates/pnn_template.py</code>	starting point for each problem
<code>solutions/pnn_solution.py</code>	reference implementation
<code>outputs/pnn_output.txt</code>	expected console output
<code>data/</code>	input files for Problems 4, 7, 9 and 13

To run a problem, copy `signal_lti.py` and any needed data files next to the script.

Problem index

#	Title	Origin
1	Convolution properties toolbox	new
2	Interconnection of LTI blocks: two branches into a cascade	Adapted from Online B (B1, B2)
3	SuperSignal: filtering a weighted sum without forming the sum	Adapted from Online C (C1, C2)
4	Working from the step response instead of the impulse response	Adapted from Online A (A1, A2)
5	Deconvolution: recovering the input from the output	new
6	Cross-correlation and time-delay estimation	new
7	Matched-filter detection of a pattern buried in noise	new
8	A first-order recursive system and its truncated impulse response	new
9	Overlap-add block convolution	new
10	Circular convolution versus linear convolution	new
11	Upsampling by zero stuffing and interpolation	new
12	Filter delay and zero-phase forward-backward filtering	new
13	Choosing a smoothing filter by measuring the error	new
14	Causality, memory, BIBO stability, and attaining the bound	new
15	Sinusoids are eigenfunctions: measuring the frequency response	new

The set is ordered so that later problems can lean on earlier ones: Problems 1–4 establish the properties and the two ways of describing a system, 5–7 invert and detect, 8–11 handle practical processing concerns, and 12–15 move toward delay, error and frequency.

Problem 1. Convolution properties toolbox

Total Marks: 10
Suggested Time: 25 minutes
New

Convolution obeys four algebraic rules that every later problem quietly depends on:

$$x * h = h * x, \quad (x * h_1) * h_2 = x * (h_1 * h_2),$$

$$x * (h_1 + h_2) = x * h_1 + x * h_2, \quad x * \delta[n - k] = x[n - k].$$

In this online you will check all four numerically with your own classes. The signals deliberately start at negative indices and have different ranges, so the comparison helper cannot assume that two results are stored on the same window.

Tasks

1. Write `convolve(a, b)` that returns $(a * b)[n]$ using only your `LTISystem`.
2. Write `max_abs_diff(u, v)` returning $\max_n |u[n] - v[n]|$ over the *union* of both ranges. It must not assume equal lengths.
3. Verify commutativity with x and h_1 .
4. Verify associativity: $(x * h_1) * h_2$ against $x * (h_1 * h_2)$.
5. Verify distributivity: $x * (h_1 + h_2)$ against $x * h_1 + x * h_2$.
6. Verify the shifting property: $x * \delta[n - k]$ against `x.shift(k)`, for $k = 3$.
7. Print the range and values of every intermediate signal, then the four differences and one pass/fail verdict at tolerance 10^{-12} .

Marks Distribution

Component	Marks
<code>convolve</code> and <code>max_abs_diff</code> helpers (union of ranges handled)	2
Commutativity	1
Associativity	2
Distributivity	2
Shifting property	2
Printed report and verdict	1
Total	10

Template

```

1  """
2  Problem 01 - Convolution properties toolbox (TEMPLATE)
3
4  Rules:
5      - Use DiscreteSignal and LTISystem from your offline (signal_lti.py) unchanged.
6      - No numpy.convolve / scipy.signal / any built-in convolution or correlation.
7      - Every plotting block is described in comments; write that code yourself.
8  """
9
10 import numpy as np
11 from signal_lti import DiscreteSignal, LTISystem

```

```

12
13
14 def make_signal(start_time, values):
15     """Build a DiscreteSignal holding 'values' starting at index start_time."""
16     # TODO
17     raise NotImplementedError
18
19
20 def unit_impulse(k):
21     """delta[n - k] as a DiscreteSignal stored on the single index k."""
22     # TODO
23     raise NotImplementedError
24
25
26 def convolve(first, second):
27     """(first * second)[n], computed with the offline LTISystem only."""
28     # TODO
29     raise NotImplementedError
30
31
32 def max_abs_diff(first, second):
33     """
34     max |first[n] - second[n]| over the UNION of the two ranges.
35     The two signals may not share a range, so do not assume equal lengths.
36     """
37     # TODO
38     raise NotImplementedError
39
40
41 def main():
42     x = make_signal(-2, [1, 0, 2, -1, 0, 3])
43     h1 = make_signal(0, [0.5, 0.3, 0.2])
44     h2 = make_signal(-1, [1, -1, 0.5])
45     k = 3
46
47     # TODO (a) commutativity : x * h1 vs h1 * x
48     # TODO (b) associativity : (x * h1) * h2 vs x * (h1 * h2)
49     # TODO (c) distributivity: x * (h1 + h2) vs (x * h1) + (x * h2)
50     # TODO (d) shift property: x * delta[n-k] vs x.shift(k)
51     # TODO print the range and values of every intermediate signal, then print
52     #       the four max/diff/ values and a single pass/fail verdict at 1e-12.
53
54     # PLOTTING (write this part yourself):
55     # - Make one figure with 4 rows of subplots, one row per property.
56     # - In each row stem-plot the two signals being compared on the same axes,
57     #   using two different markers (e.g. filled circles vs. hollow squares)
58     #   so an exact overlap is visible.
59     # - Remember the two signals may have different ranges: plot each one over
60     #   its own times() array rather than assuming a shared n-axis.
61     # - Label every subplot with the property name and the measured max/diff/,
62     #   set xlabel "n", turn the grid on, and save the figure as p01_properties.png.
63
64
65 if __name__ == "__main__":
66     main()

```

Problem 1 — Solution

Solution — approach and discussion

The only real trap is `max_abs_diff`. Two mathematically equal results can be *stored* on different windows: $x * h_1$ lives on $[-2, 5]$, while $(x * h_1) * h_2$ and $x * (h_1 * h_2)$ both live on $[-3, 6]$. Subtracting arrays element by element would therefore compare the wrong samples, or crash. The clean fix reuses the offline's own range-merging logic:

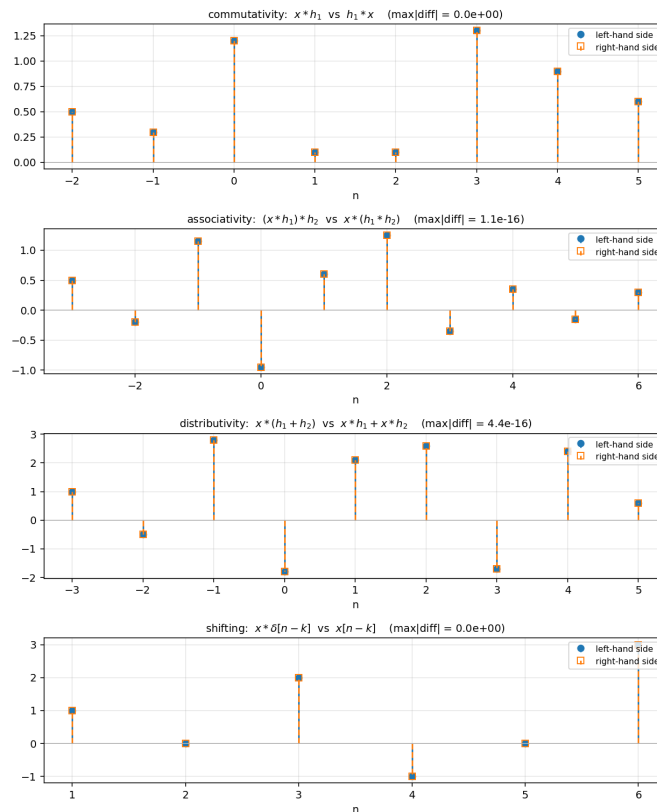
```
difference = u.add(v.multiply(-1)),
```

because `add` already widens to the union and treats missing samples as zero.

For the shifting property, note that $x * \delta[n - k]$ automatically lands on $[n_{x,\min} + k, n_{x,\max} + k]$, which is exactly the range `x.shift(k)` produces. The two signals agree sample for sample *and* index for index, so the difference is identically zero rather than merely small.

The associativity and distributivity residuals are of order 10^{-16} rather than exactly zero. That is floating-point round-off from adding the same products in a different order, not a bug — which is why the verdict uses a tolerance instead of an equality test.

Figures produced by the plotting code



Reference implementation

```
1 """
2 Problem 01 - Convolution properties toolbox (SOLUTION)
3
4 Uses DiscreteSignal and LTISystem from the offline (signal_lti.py) unchanged.
5 No built-in convolution / correlation is used anywhere.
```

```

6  """
7
8  import numpy as np
9  import matplotlib.pyplot as plt
10 from signal_lti import DiscreteSignal, LTISystem
11
12
13 # ----- small helpers -----
14
15 def make_signal(start_time, values):
16     """Build a DiscreteSignal holding 'values' starting at index start_time."""
17     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
18     for offset, value in enumerate(values):
19         signal.set_value_at_time(start_time + offset, value)
20     return signal
21
22
23 def unit_impulse(k):
24     """delta[n - k] as a DiscreteSignal stored on the single index k."""
25     impulse = DiscreteSignal(k, k)
26     impulse.set_value_at_time(k, 1.0)
27     return impulse
28
29
30 def convolve(first, second):
31     """(first * second)[n], computed with the offline LTISystem only."""
32     return LTISystem(second).output(first)
33
34
35 def max_abs_diff(first, second):
36     """
37     max |first[n] - second[n]| over the union of both ranges.
38     add() already widens to the union, so different ranges are handled.
39     """
40     difference = first.add(second.multiply(-1))
41     return float(np.max(np.abs(difference.values)))
42
43
44 def describe(name, signal):
45     values = " ".join(f"{v:.4f}" for v in signal.values)
46     return f"{name}: range [{signal.start_time}, {signal.end_time}] -> {values}"
47
48
49 # ----- the four properties -----
50
51 def main():
52     x = make_signal(-2, [1, 0, 2, -1, 0, 3])
53     h1 = make_signal(0, [0.5, 0.3, 0.2])
54     h2 = make_signal(-1, [1, -1, 0.5])
55     k = 3
56
57     print(describe("x[n]", x))
58     print(describe("h1[n]", h1))
59     print(describe("h2[n]", h2))
60     print()
61
62     # (a) commutativity: x * h1 == h1 * x
63     y_xh = convolve(x, h1)
64     y_hx = convolve(h1, x)
65     err_comm = max_abs_diff(y_xh, y_hx)
66
67     # (b) associativity: (x * h1) * h2 == x * (h1 * h2)
68     left = convolve(convolve(x, h1), h2)
69     h_series = convolve(h1, h2)
70     right = convolve(x, h_series)
71     err_assoc = max_abs_diff(left, right)
72
73     # (c) distributivity: x * (h1 + h2) == x * h1 + x * h2
74     h_parallel = h1.add(h2)
75     lhs = convolve(x, h_parallel)
76     rhs = convolve(x, h1).add(convolve(x, h2))
77     err_dist = max_abs_diff(lhs, rhs)
78
79     # (d) shifted impulse: x * delta[n - k] == x[n - k]

```

```

80 shifted_by_conv = convolve(x, unit_impulse(k))
81 shifted_directly = x.shift(k)
82 err_shift = max_abs_diff(shifted_by_conv, shifted_directly)
83
84 print(describe("x*h1", y_xh))
85 print(describe("h1*h2", h_series))
86 print(describe("h1+h2", h_parallel))
87 print(describe("x*delta[n-k]", shifted_by_conv))
88 print()
89
90 print(f"commutativity max|diff| = {err_comm:.3e}")
91 print(f"associativity max|diff| = {err_assoc:.3e}")
92 print(f"distributivity max|diff| = {err_dist:.3e}")
93 print(f"shift property max|diff| = {err_shift:.3e}")
94
95 tolerance = 1e-12
96 all_ok = max(err_comm, err_assoc, err_dist, err_shift) < tolerance
97 print("\nAll four properties verified:", all_ok)
98
99 # Figures described below; the implementation follows.
100 # - Make one figure with 4 rows of subplots, one row per property.
101 # - In each row stem-plot the two signals being compared on the same axes,
102 #   using two different markers (e.g. filled circles vs. hollow squares)
103 #   so an exact overlap is visible.
104 # - Remember the two signals may have different ranges: plot each one over
105 #   its own times() array rather than assuming a shared n-axis.
106 # - Label every subplot with the property name and the measured max|diff|,
107 #   set xlabel "n", turn the grid on, and save the figure as p01_properties.png.
108
109 # ---- Figures ----
110 # One row per property. The two signals are stemmed on the same axes with
111 # different markers; each is plotted over its OWN times() array, because a
112 # matching pair does not have to be stored on the same range.
113 comparisons = [
114     ("commutativity: $x*h_1$ vs $h_1*x$", y_xh, y_hx, err_comm),
115     ("associativity: $(x*h_1)*h_2$ vs $x*(h_1*h_2)$", left, right, err_assoc),
116     ("distributivity: $x*(h_1+h_2)$ vs $x*h_1 + x*h_2$", lhs, rhs, err_dist),
117     (r"shifting: $x*\delta[n-k]$ vs $x[n-k]$", shifted_by_conv, shifted_directly, err_shift),
118 ]
119
120 figure, axes = plt.subplots(4, 1, figsize=(9, 11))
121 for axis, (label, first, second, error) in zip(axes, comparisons):
122     axis.stem(first.times(), first.values, linefmt="C0-",
123              markerfmt="C0o", basefmt=" ", label="left-hand side")
124     markers, _, _ = axis.stem(second.times(), second.values, linefmt="C1--",
125                               markerfmt="C1s", basefmt=" ", label="right-hand side")
126     markers.set_markerfacecolor("none") # hollow, so an exact overlap is visible
127     axis.axhline(0, color="0.7", linewidth=0.8)
128     axis.set_title(f"{label} (max|diff| = {error:.1e})", fontsize=10)
129     axis.set_xlabel("n")
130     axis.grid(True, alpha=0.3)
131     axis.legend(fontsize=8, loc="upper right")
132
133 figure.tight_layout()
134 figure.savefig("p01_properties.png", dpi=140)
135 plt.close(figure)
136 print("\nsaved p01_properties.png")
137
138 if __name__ == "__main__":
139     main()

```

Expected console output

```

x[n] : range [-2, 3] -> 1.0000 0.0000 2.0000 -1.0000 0.0000 3.0000
h1[n]: range [0, 2] -> 0.5000 0.3000 0.2000
h2[n]: range [-1, 1] -> 1.0000 -1.0000 0.5000

x*h1      : range [-2, 5] -> 0.5000 0.3000 1.2000 0.1000 0.1000 1.3000 0.9000 0.6000
h1*h2     : range [-1, 3] -> 0.5000 -0.2000 0.1500 -0.0500 0.1000
h1+h2     : range [-1, 2] -> 1.0000 -0.5000 0.8000 0.2000

```

```
x*delta[n-k]: range [1, 6] -> 1.0000 0.0000 2.0000 -1.0000 0.0000 3.0000
```

```
commutativity  max|diff| = 0.000e+00  
associativity  max|diff| = 1.110e-16  
distributivity max|diff| = 4.441e-16  
shift property max|diff| = 0.000e+00
```

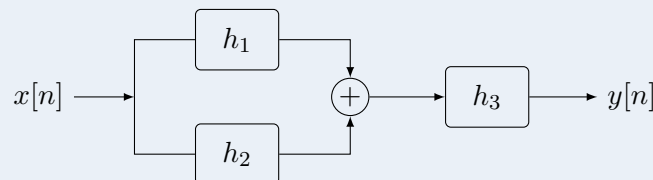
```
All four properties verified: True
```

```
saved p01_properties.png
```

Problem 2. Interconnection of LTI blocks: two branches into a cascade

Total Marks: 10 Suggested Time: 30 minutes Adapted from Online B (B1, B2)

Two systems in parallel are equivalent to one system whose impulse response is the *sum* of theirs; two systems in cascade are equivalent to one whose impulse response is the *convolution* of theirs. The structure below combines both:



with

$$x[n] = \{ \underset{n=0}{1}, 0, -1 \}, \quad h_1[n] = \delta[n], \quad h_2[n] = \frac{1}{2} \delta[n-1], \quad h_3[n] = \delta[n] + \delta[n-1].$$

Note that h_2 is stored on the single index $n = 1$, not on $[0, 1]$: your `add` has to reconcile that with h_1 , which is stored on $n = 0$.

Tasks

1. Determine the output by pushing the signal through the diagram one block at a time: top branch, bottom branch, summing junction, then h_3 .
2. Determine h_{eq} , the impulse response of a single LTI system equivalent to the whole diagram, and compute the output from it.
3. Verify the two outputs are the same signal, both in values and in stored range.
4. Confirm by hand that the output range equals $[n_{x,\text{min}} + n_{h_{\text{eq}},\text{min}}, n_{x,\text{max}} + n_{h_{\text{eq}},\text{max}}]$ and check your program agrees.

Marks Distribution

Component	Marks
Block-by-block propagation through the two branches and the adder	3
Correct final stage through h_3	1
Deriving h_{eq} (knowing which parts add and which convolve)	3
Output from the equivalent system	1
Verification and output-range check	2
Total	10

Template

```

1  """
2  Problem 02 - Interconnection of LTI blocks (TEMPLATE)
3
4  +--> [ h1 ] --+
```

```

5          x[n] ---+          (+)--> [ h3 ] --> y[n]
6              +--> [ h2 ] ---+
7
8 Rules:
9 - Use DiscreteSignal and LTISystem from your offline (signal_lti.py) unchanged.
10 - No built-in convolution.
11 - Plotting is described in comments; write it yourself.
12 """
13
14 import numpy as np
15 from signal_lti import DiscreteSignal, LTISystem
16
17
18 def make_signal(start_time, values):
19     # TODO
20     raise NotImplementedError
21
22
23 def convolve(first, second):
24     # TODO
25     raise NotImplementedError
26
27
28 def max_abs_diff(first, second):
29     # TODO (must work when the two ranges differ)
30     raise NotImplementedError
31
32
33 def main():
34     x = make_signal(0, [1, 0, -1])
35     h1 = make_signal(0, [1])          # h1[n] = delta[n]
36     h2 = make_signal(1, [0.5])        # h2[n] = 0.5 * delta[n-1]
37     h3 = make_signal(0, [1, 1])        # h3[n] = delta[n] + delta[n-1]
38
39     system1 = LTISystem(h1)
40     system2 = LTISystem(h2)
41     system3 = LTISystem(h3)
42
43     # TODO Path 1: push x through the diagram block by block.
44     #     branch1, branch2, adder output, then the h3 stage.
45     # TODO Path 2: collapse the whole diagram into a single h_equivalent.
46     #     Decide for yourself which parts add and which parts convolve.
47     # TODO compute the output from h_equivalent and verify the two paths agree.
48     # TODO print the range and values of every intermediate signal, and confirm
49     #     that the output range equals
50     #     [x.start + h_eq.start, x.end + h_eq.end] by hand as well.
51
52     # PLOTTING (write this part yourself):
53     # - Figure 1: 4 stacked stem subplots showing x[n], branch1, branch2 and
54     #   the adder output, so the parallel section can be inspected stage by stage.
55     # - Figure 2: 2 stacked stem subplots comparing the two outputs, plus a third
56     #   subplot overlaying them with different markers to show they coincide.
57     # - Also stem-plot h_equivalent on its own so the 3-tap equivalent response is visible.
58     # - Use xlabel "n", a title per subplot, grid on, and save as
59     #   p02_stages.png and p02_outputs.png.
60
61
62 if __name__ == "__main__":
63     main()

```

Problem 2 — Solution

Solution — approach and discussion

The equivalent response follows from distributivity and associativity, which is why this problem sits right after Problem 1:

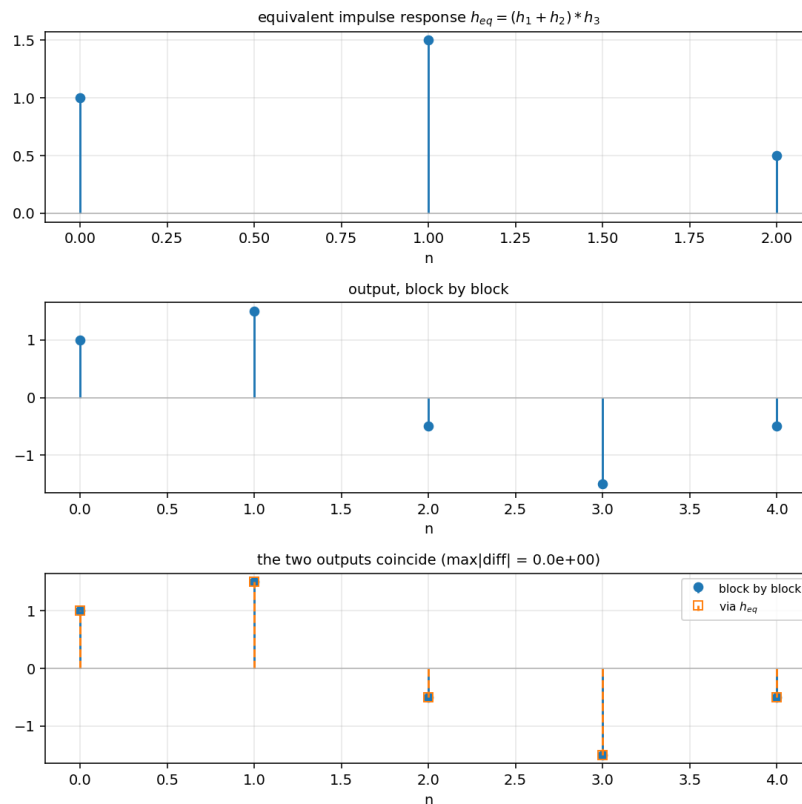
$$y = ((x * h_1) + (x * h_2)) * h_3 = (x * (h_1 + h_2)) * h_3 = x * \underbrace{(h_1 + h_2) * h_3}_{h_{eq}}.$$

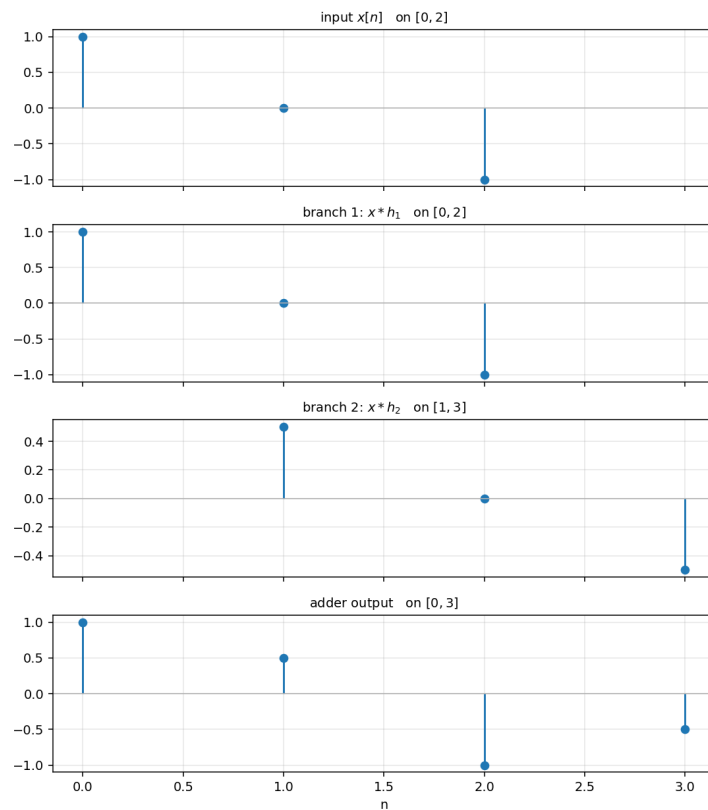
Numerically $h_1 + h_2 = \{1, 0.5\}$ on $[0, 1]$ and $h_{eq} = \{1, 1.5, 0.5\}$ on $[0, 2]$, giving $y = \{1, 1.5, -0.5, -1.5, -0.5\}$ on $[0, 4]$.

Two things are worth noticing when this is run against *your* offline API rather than the old `signal(INF)` one. First, the intermediate signals genuinely change range as they move through the diagram: the bottom branch comes out on $[1, 3]$ while the top branch is on $[0, 2]$, and the adder has to merge them into $[0, 3]$. The old fixed-window class hid this entirely. Second, the two paths agree exactly (difference 0, not 10^{-16}) because both end up summing the same products in the same order.

A common wrong answer is $h_{eq} = h_1 + h_2 + h_3$ or $h_1 * h_2 * h_3$. Parallel means add, cascade means convolve — and the order matters only in the sense that the adder comes first here.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 02 - Interconnection of LTI blocks: parallel branches feeding a cascade
3  (SOLUTION)
4
5  Structure:
6      x[n] ----+----> [ h1 ] ----+----> (+)--> [ h3 ] --> y[n]
7              +---> [ h2 ] ----+
8
9  Equivalent single system: h_eq = (h1 + h2) * h3
10 """
11
12 import numpy as np
13 import matplotlib.pyplot as plt
14 from signal_lti import DiscreteSignal, LTISystem
15
16
17 def make_signal(start_time, values):
18     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
19     for offset, value in enumerate(values):
20         signal.set_value_at_time(start_time + offset, value)
21     return signal
22
23
24 def convolve(first, second):
25     return LTISystem(second).output(first)
26
27
28 def max_abs_diff(first, second):
29     difference = first.add(second.multiply(-1))
30     return float(np.max(np.abs(difference.values)))
31
32
33 def describe(name, signal):
34     values = " ".join(f"{v:.4f}" for v in signal.values)
35     return f"{name}: range [{signal.start_time}, {signal.end_time}] -> {values}"
36

```

```

37
38 def main():
39     x = make_signal(0, [1, 0, -1])
40
41     h1 = make_signal(0, [1])           # h1[n] = delta[n]
42     h2 = make_signal(1, [0.5])       # h2[n] = 0.5 * delta[n-1]
43     h3 = make_signal(0, [1, 1])      # h3[n] = delta[n] + delta[n-1]
44
45     system1 = LTISystem(h1)
46     system2 = LTISystem(h2)
47     system3 = LTISystem(h3)
48
49     # ---- Path 1: propagate the signal block by block ----
50     branch1 = system1.output(x)       # top branch
51     branch2 = system2.output(x)       # bottom branch
52     adder_output = branch1.add(branch2) # summing junction
53     y_blockwise = system3.output(adder_output)
54
55     # ---- Path 2: collapse the whole structure into one impulse response ----
56     # Parallel branches add; the cascade convolves.
57     h_parallel = h1.add(h2)
58     h_equivalent = convolve(h_parallel, h3)
59     y_combined = LTISystem(h_equivalent).output(x)
60
61     # ---- Verification ----
62     difference = max_abs_diff(y_blockwise, y_combined)
63
64     print(describe("x[n]", x))
65     print(describe("h1[n]", h1))
66     print(describe("h2[n]", h2))
67     print(describe("h3[n]", h3))
68     print()
69     print(describe("branch1", branch1))
70     print(describe("branch2", branch2))
71     print(describe("adder output", adder_output))
72     print(describe("h1 + h2", h_parallel))
73     print(describe("h_equivalent", h_equivalent))
74     print()
75     print(describe("y block-by-block", y_blockwise))
76     print(describe("y combined", y_combined))
77     print(f"\nmax |y_blockwise[n] - y_combined[n]| = {difference:.3e}")
78     print("Outputs identical:", difference < 1e-12)
79
80     # Sanity check on the range arithmetic done by hand:
81     expected_start = x.start_time + h_equivalent.start_time
82     expected_end = x.end_time + h_equivalent.end_time
83     print(f"predicted output range = [{expected_start}, {expected_end}], "
84           f"actual = [{y_combined.start_time}, {y_combined.end_time}]")
85
86     # Figures described below; the implementation follows.
87     # - Figure 1: 4 stacked stem subplots showing x[n], branch1, branch2 and
88     #   the adder output, so the parallel section can be inspected stage by stage.
89     # - Figure 2: 2 stacked stem subplots comparing y_blockwise and y_combined.
90     #   Overlay them on a third subplot with different markers to show they coincide.
91     # - Also stem-plot h_equivalent on its own so the 3-tap equivalent response is visible.
92     # - Use xlabel "n", a title per subplot, grid on, and save as
93     #   p02_stages.png and p02_outputs.png.
94
95     # ---- Figures ----
96     # Figure 1: the parallel section, stage by stage.
97     stages = [("input $x[n]$", x), ("branch 1: $x*h_1$", branch1),
98              ("branch 2: $x*h_2$", branch2), ("adder output", adder_output)]
99
100     figure, axes = plt.subplots(4, 1, figsize=(8, 9), sharex=True)
101     for axis, (label, signal) in zip(axes, stages):
102         axis.stem(signal.times(), signal.values, basefmt=" ")
103         axis.axhline(0, color="0.7", linewidth=0.8)
104         axis.set_title(f"{label} on $[{{signal.start_time}}, {{signal.end_time}}]$", fontsize=10)
105         axis.grid(True, alpha=0.3)
106     axes[-1].set_xlabel("n")
107     figure.tight_layout()
108     figure.savefig("p02_stages.png", dpi=140)
109     plt.close(figure)
110

```

```

111 # Figure 2: the two routes to the output, and the equivalent impulse response.
112 figure, axes = plt.subplots(3, 1, figsize=(8, 8))
113 axes[0].stem(h_equivalent.times(), h_equivalent.values, basefmt=" ")
114 axes[0].set_title("equivalent impulse response  $h_{eq} = (h_1+h_2)*h_3$ ", fontsize=10)
115
116 axes[1].stem(y_blockwise.times(), y_blockwise.values, basefmt=" ")
117 axes[1].set_title("output, block by block", fontsize=10)
118
119 axes[2].stem(y_blockwise.times(), y_blockwise.values, linefmt="C0-",
120             markerfmt="C0o", basefmt=" ", label="block by block")
121 markers, _, _ = axes[2].stem(y_combined.times(), y_combined.values, linefmt="C1--",
122                             markerfmt="C1s", basefmt=" ", label="via  $h_{eq}$ ")
123 markers.set_markerfacecolor("none")
124 axes[2].set_title(f"the two outputs coincide (max|diff| = {difference:.1e})", fontsize=10)
125 axes[2].legend(fontsize=8)
126
127 for axis in axes:
128     axis.axhline(0, color="0.7", linewidth=0.8)
129     axis.set_xlabel("n")
130     axis.grid(True, alpha=0.3)
131 figure.tight_layout()
132 figure.savefig("p02_outputs.png", dpi=140)
133 plt.close(figure)
134 print("\nsaved p02_stages.png, p02_outputs.png")
135
136 if __name__ == "__main__":
137     main()

```

Expected console output

```

x[n]      : range [0, 2] -> 1.0000 0.0000 -1.0000
h1[n]     : range [0, 0] -> 1.0000
h2[n]     : range [1, 1] -> 0.5000
h3[n]     : range [0, 1] -> 1.0000 1.0000

branch1   : range [0, 2] -> 1.0000 0.0000 -1.0000
branch2   : range [1, 3] -> 0.5000 0.0000 -0.5000
adder output: range [0, 3] -> 1.0000 0.5000 -1.0000 -0.5000
h1 + h2   : range [0, 1] -> 1.0000 0.5000
h_equivalent: range [0, 2] -> 1.0000 1.5000 0.5000

y block-by-block: range [0, 4] -> 1.0000 1.5000 -0.5000 -1.5000 -0.5000
y combined      : range [0, 4] -> 1.0000 1.5000 -0.5000 -1.5000 -0.5000

max |y_blockwise[n] - y_combined[n]| = 0.000e+00
Outputs identical: True
predicted output range = [0, 4], actual = [0, 4]

saved p02_stages.png, p02_outputs.png

```

Problem 3. SuperSignal: filtering a weighted sum without forming the sum

Total Marks: 10 Suggested Time: 25 minutes Adapted from Online C (C1, C2)

An input is often described as a weighted combination of simpler pieces,

$$x[n] = \sum_i c_i x_i[n],$$

and linearity says the output can be assembled the same way:

$$y[n] = \sum_i c_i (x_i * h)[n].$$

A `SuperSignal` stores the pieces and their coefficients without ever adding them together. Your job is to teach the LTI system to consume one. Use $x_1[n] = \delta[n]$, $x_2[n] = \delta[n-2]$, $x_3[n] = \{1, 1, 1\}$ on $[-3, -1]$, with

$$x[n] = 2x_1[n] - x_2[n] + 0.5x_3[n], \quad h[n] = \left\{ \begin{matrix} 1 \\ 0.5 \end{matrix} \right\}_{n=0}.$$

Tasks

1. Complete `SuperSignal.collapse()`, which adds all weighted components into one ordinary `DiscreteSignal`.
2. Implement `output_super(super_signal)` in a subclass of your offline `LTISystem`, computing one component response at a time and superposing them. Do not call `collapse()` inside it.
3. Build the `SuperSignal` above and compute `y_super`.
4. Compute `y_direct` by collapsing first and then convolving, and verify the two agree.
5. Print the range and values of every component, the collapsed input, and both outputs.

Marks Distribution

Component	Marks
<code>collapse()</code> with a correct union range	2
<code>output_super</code> computing per-component responses	3
Correct result range covering every component's output	2
Building the <code>SuperSignal</code> with the right coefficients	1
Verification against the collapsed route	2
Total	10

Template

```

1  """
2  Problem 03 - SuperSignal and the linearity of an LTI system (TEMPLATE)
3
4  x[n] = sum_i c_i * x_i[n] is stored as a list of (coefficient, signal) pairs
5  and must never be summed before filtering. Linearity gives
6  y[n] = sum_i c_i * (x_i * h)[n].

```

```

7  """
8
9  import numpy as np
10 from signal_lti import DiscreteSignal, LTISystem
11
12
13 def make_signal(start_time, values):
14     # TODO
15     raise NotImplementedError
16
17
18 def max_abs_diff(first, second):
19     # TODO
20     raise NotImplementedError
21
22
23 class SuperSignal:
24     """An input kept as a list of (coefficient, DiscreteSignal) pairs."""
25
26     def __init__(self):
27         self.components = []
28
29     def add(self, signal, coefficient=1.0):
30         self.components.append((coefficient, signal))
31         return self
32
33     def collapse(self):
34         """Add all weighted components into one ordinary DiscreteSignal."""
35         # TODO: work out the union range first, then accumulate.
36         raise NotImplementedError
37
38
39 class LTISystemSuper(LTISystem):
40     """The offline LTISystem extended with a SuperSignal entry point."""
41
42     def output_super(self, super_signal):
43         """Output computed one component at a time, then superposed."""
44         # TODO: the result range must cover every component's own output range.
45         raise NotImplementedError
46
47
48 def main():
49     x1 = make_signal(0, [1])           # delta[n]
50     x2 = make_signal(2, [1])           # delta[n-2]
51     x3 = make_signal(-3, [1, 1, 1])     # a short rectangular burst
52
53     # x[n] = 2*x1[n] - x2[n] + 0.5*x3[n]
54     # TODO build the SuperSignal with those three coefficients.
55
56     h = make_signal(0, [1, 0.5])
57     # TODO create an LTISystemSuper, compute y_super via output_super,
58     #       compute y_direct by collapsing first and then convolving,
59     #       and verify the two agree.
60
61     # PLOTTING (write this part yourself):
62     # - Figure 1: one stem subplot per component x_i[n] (3 rows here). Put the
63     #   coefficient c_i in each subplot title, and add a 4th row with the
64     #   collapsed x[n] so the weighted sum is visually obvious.
65     # - Figure 2: stem-plot the individual scaled responses c_i*(x_i*h)[n] in
66     #   the top rows and y_super[n] in the bottom row.
67     # - Figure 3: overlay y_super[n] and y_direct[n] with different markers.
68     # - Loop over the component list instead of hardcoding three subplots so the
69     #   code still works if a fourth component is added.
70     # - xlabel "n", grid on, save as p03_components.png / p03_outputs.png.
71
72
73 if __name__ == "__main__":
74     main()

```

Problem 3 — Solution

Solution — approach and discussion

The interesting part is the range bookkeeping. Component i produces an output on

$$[n_{x_i,\min} + n_{h,\min}, n_{x_i,\max} + n_{h,\max}],$$

and these windows differ from each other — here $[-3, 0]$, $[0, 1]$ and $[2, 3]$. The accumulator must therefore start out spanning

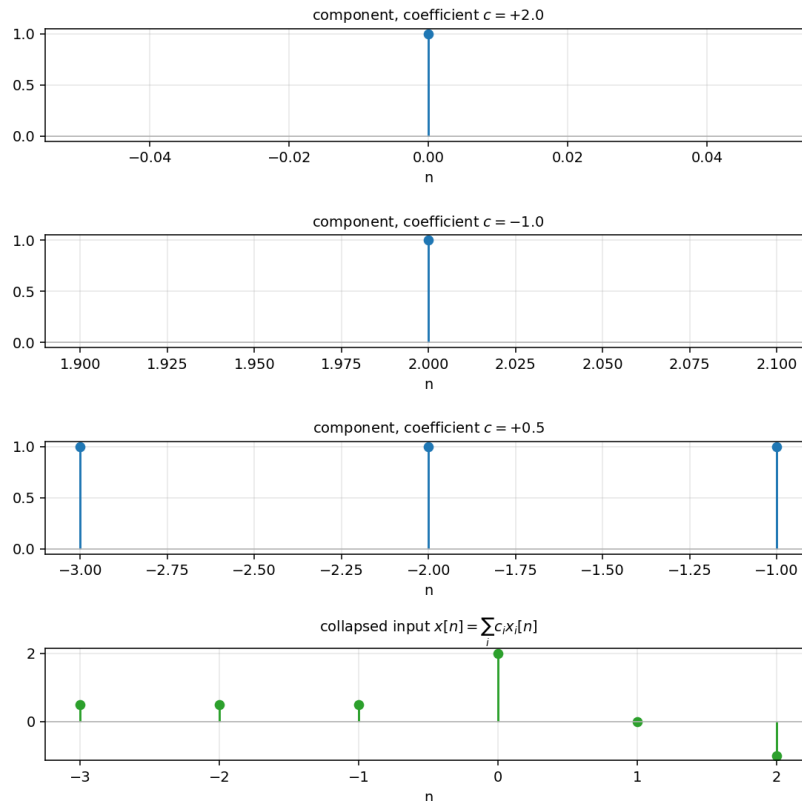
$$\left[\min_i n_{x_i,\min} + n_{h,\min}, \max_i n_{x_i,\max} + n_{h,\max} \right] = [-3, 3].$$

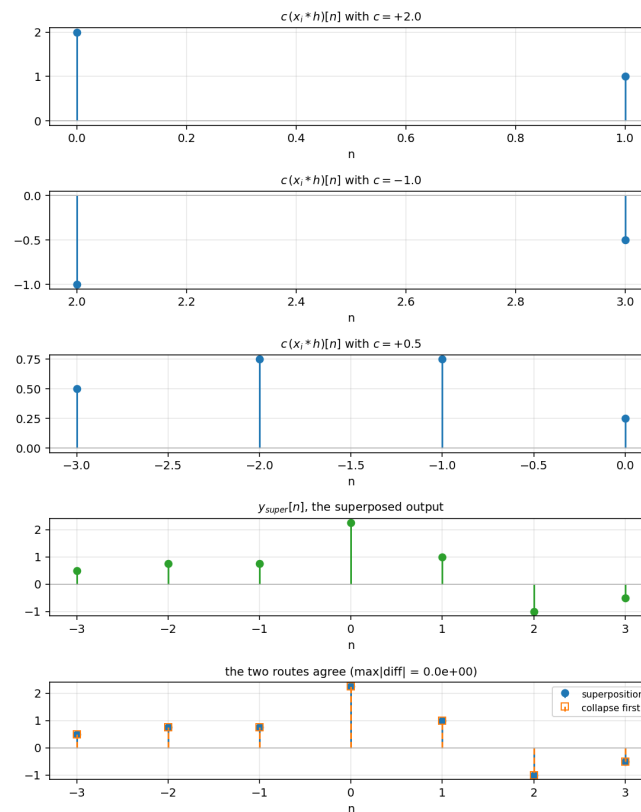
Starting it at, say, the first component's range and relying on `add` to grow it also works with this particular offline implementation, but stating the full range up front is clearer and is what the mark scheme asks for.

Subclassing `LTISystem` rather than editing it keeps your offline file untouched, which matters because several of these problems import the same `signal_lti.py`.

Note that `output_super` does strictly more arithmetic than the collapsed route here: three convolutions instead of one. The payoff is conceptual, not computational — it is the direct numerical statement of linearity, and it is exactly the decomposition your offline's `get_response_components` already performs one impulse at a time.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 03 - SuperSignal and the linearity of an LTI system (SOLUTION)
3
4  A SuperSignal stores an input written as a weighted sum of component signals,
5   $x[n] = \sum_i c_i * x_i[n]$ ,
6  without ever forming the sum. Linearity then gives
7   $y[n] = \sum_i c_i * (x_i * h)[n]$ .
8  """
9
10 import numpy as np
11 import matplotlib.pyplot as plt
12 from signal_lti import DiscreteSignal, LTISystem
13
14
15 def make_signal(start_time, values):
16     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
17     for offset, value in enumerate(values):
18         signal.set_value_at_time(start_time + offset, value)
19     return signal
20
21
22 def max_abs_diff(first, second):
23     difference = first.add(second.multiply(-1))
24     return float(np.max(np.abs(difference.values)))
25
26
27 def describe(name, signal):
28     values = " ".join(f"{v:.4f}" for v in signal.values)
29     return f"{name}: range [{signal.start_time}, {signal.end_time}] -> {values}"
30
31
32 class SuperSignal:
33     """An input kept as a list of (coefficient, DiscreteSignal) pairs."""
34
35     def __init__(self):
36         self.components = []

```

```

37
38     def add(self, signal, coefficient=1.0):
39         self.components.append((coefficient, signal))
40         return self
41
42     def collapse(self):
43         """Add all weighted components into one ordinary DiscreteSignal."""
44         if not self.components:
45             raise ValueError("SuperSignal is empty")
46
47         start_time = min(signal.start_time for _, signal in self.components)
48         end_time = max(signal.end_time for _, signal in self.components)
49
50         total = DiscreteSignal(start_time, end_time)
51         for coefficient, signal in self.components:
52             total = total.add(signal.multiply(coefficient))
53         return total
54
55
56 class LTISystemSuper(LTISystem):
57     """The offline LTISystem extended with a SuperSignal entry point."""
58
59     def output_super(self, super_signal):
60         """Output computed one component at a time, then superposed."""
61         if not super_signal.components:
62             raise ValueError("SuperSignal is empty")
63
64         start_time = (min(signal.start_time for _, signal in super_signal.components)
65                       + self.impulse_response.start_time)
66         end_time = (max(signal.end_time for _, signal in super_signal.components)
67                    + self.impulse_response.end_time)
68
69         total = DiscreteSignal(start_time, end_time)
70         for coefficient, signal in super_signal.components:
71             partial = self.output(signal).multiply(coefficient)
72             total = total.add(partial)
73         return total
74
75
76 def main():
77     x1 = make_signal(0, [1])                # delta[n]
78     x2 = make_signal(2, [1])                # delta[n-2]
79     x3 = make_signal(-3, [1, 1, 1])         # a short rectangular burst
80
81     #  $x[n] = 2x_1[n] - x_2[n] + 0.5x_3[n]$ 
82     x_super = SuperSignal()
83     x_super.add(x1, 2.0)
84     x_super.add(x2, -1.0)
85     x_super.add(x3, 0.5)
86
87     h = make_signal(0, [1, 0.5])
88     system = LTISystemSuper(h)
89
90     y_super = system.output_super(x_super)   # component-wise, then superpose
91     x_collapsed = x_super.collapse()
92     y_direct = system.output(x_collapsed)    # collapse first, then convolve
93
94     difference = max_abs_diff(y_super, y_direct)
95
96     for index, (coefficient, signal) in enumerate(x_super.components, start=1):
97         print(describe(f"c{index} = {coefficient:+.1f}, x{index}[n]", signal))
98     print(describe("h[n]", h))
99     print()
100    print(describe("collapsed x[n]", x_collapsed))
101    print(describe("y_super[n]", y_super))
102    print(describe("y_direct[n]", y_direct))
103    print(f"\nmax |y_super[n] - y_direct[n]| = {difference:.3e}")
104    print("Superposition verified:", difference < 1e-12)
105
106    # Figures described below; the implementation follows.
107    # - Figure 1: one stem subplot per component  $x_i[n]$  (3 rows here). Put the
108    #   coefficient  $c_i$  in each subplot title, and add a 4th row with the
109    #   collapsed  $x[n]$  so the weighted sum is visually obvious.
110    # - Figure 2: stem-plot the individual scaled responses  $c_i*(x_i*h)[n]$  in

```

```

111 # the top rows and y_super[n] in the bottom row.
112 # - Figure 3: overlay y_super[n] and y_direct[n] with different markers.
113 # - Loop over the component list instead of hardcoding three subplots so the
114 # code still works if a fourth component is added.
115 # - xlabel "n", grid on, save as p03_components.png / p03_outputs.png.
116
117 # ---- Figures ----
118 # Figure 1: the components and the signal they add up to. Looping over the
119 # component list means this still works if a fourth component is added.
120 count = len(x_super.components)
121 figure, axes = plt.subplots(count + 1, 1, figsize=(8, 2.0 * (count + 1)))
122 for axis, (coefficient, signal) in zip(axes, x_super.components):
123     axis.stem(signal.times(), signal.values, basefmt=" ")
124     axis.set_title(f"component {coefficient} $c = \{coefficient:+.1f\}$", fontsize=10)
125 axes[-1].stem(x_collapsed.times(), x_collapsed.values, linefmt="C2-",
126               markerfmt="C2o", basefmt=" ")
127 axes[-1].set_title(f"collapsed input $x[n] = \sum_i c_i x_i[n]$", fontsize=10)
128 for axis in axes:
129     axis.axhline(0, color="0.7", linewidth=0.8)
130     axis.set_xlabel("n")
131     axis.grid(True, alpha=0.3)
132 figure.tight_layout()
133 figure.savefig("p03_components.png", dpi=140)
134 plt.close(figure)
135
136 # Figure 2: each scaled component response, then their sum, then the check.
137 figure, axes = plt.subplots(count + 2, 1, figsize=(8, 2.0 * (count + 2)))
138 for axis, (coefficient, signal) in zip(axes, x_super.components):
139     partial = system.output(signal).multiply(coefficient)
140     axis.stem(partial.times(), partial.values, basefmt=" ")
141     axis.set_title(f"$c \backslash (x_i * h)[n]$ with $c = \{coefficient:+.1f\}$", fontsize=10)
142 axes[-2].stem(y_super.times(), y_super.values, linefmt="C2-",
143               markerfmt="C2o", basefmt=" ")
144 axes[-2].set_title(f"$y_{super}[n]$, the superposed output", fontsize=10)
145
146 axes[-1].stem(y_super.times(), y_super.values, linefmt="C0-",
147               markerfmt="C0o", basefmt=" ", label="superposition")
148 markers, _, _ = axes[-1].stem(y_direct.times(), y_direct.values, linefmt="C1--",
149                               markerfmt="C1s", basefmt=" ", label="collapse first")
150 markers.set_markerfacecolor("none")
151 axes[-1].set_title(f"the two routes agree (max|diff| = \{difference:.1e\})", fontsize=10)
152 axes[-1].legend(fontsize=8)
153
154 for axis in axes:
155     axis.axhline(0, color="0.7", linewidth=0.8)
156     axis.set_xlabel("n")
157     axis.grid(True, alpha=0.3)
158 figure.tight_layout()
159 figure.savefig("p03_outputs.png", dpi=140)
160 plt.close(figure)
161 print("\nsaved p03_components.png, p03_outputs.png")
162
163 if __name__ == "__main__":
164     main()

```

Expected console output

```

c1 = +2.0,  x1[n]: range [0, 0] -> 1.0000
c2 = -1.0,  x2[n]: range [2, 2] -> 1.0000
c3 = +0.5,  x3[n]: range [-3, -1] -> 1.0000 1.0000 1.0000
h[n]       : range [0, 1] -> 1.0000 0.5000

collapsed x[n]: range [-3, 2] -> 0.5000 0.5000 0.5000 2.0000 0.0000 -1.0000
y_super[n]    : range [-3, 3] -> 0.5000 0.7500 0.7500 2.2500 1.0000 -1.0000 -0.5000
y_direct[n]   : range [-3, 3] -> 0.5000 0.7500 0.7500 2.2500 1.0000 -1.0000 -0.5000

max |y_super[n] - y_direct[n]| = 0.000e+00
Superposition verified: True

saved p03_components.png, p03_outputs.png

```

Problem 4. Working from the step response instead of the impulse response

Total Marks: 10 Suggested Time: 30 minutes Adapted from Online A (A1, A2)

Sometimes a system is characterised by its *step* response $s[n]$ rather than its impulse response. The two are related by a first difference, and the output can be computed without ever forming h :

$$h[n] = s[n] - s[n-1] \quad (s[-1] = 0), \quad \Delta x[n] = x[n] - x[n-1] \quad (x[-1] = 0), \quad y[n] = (\Delta x * s)[n].$$

Two files are provided. Each has the range on line 1 and the samples on line 2.

Files provided: `step_response.txt`, `input_signal.txt` (copy them next to your script before running).

Tasks

1. Read $s[n]$ from `step_response.txt` and construct a `DiscreteSignal`.
2. Recover and report the impulse response $h[n] = s[n] - s[n-1]$, using only `shift`, `add` and `multiply`.
3. Read $x[n]$ from `input_signal.txt` and compute $\Delta x[n]$ the same way.
4. Compute $y_s[n] = (\Delta x * s)[n]$ using only the step response. Reuse your offline `LTISystem`; no `numpy.convolve`.
5. Verify with $y_h[n] = (x * h)[n]$ and show the two match.
6. Check that $\sum_n h[n]$ equals the final value of $s[n]$, and deal with the range artifact described below.

Marks Distribution

Component	Marks
Reading both files with a range/sample-count check	1
Recovering $h[n]$ with signal operations only	3
Computing $\Delta x[n]$	2
Output via the step response, y_s	3
Verification and DC-gain check	1
Total	10

Template

```

1  """
2  Problem 04 - Working from the step response instead of the impulse response
3  (TEMPLATE)
4
5      h[n] = s[n] - s[n-1]      (s[-1] = 0)
6      dx[n] = x[n] - x[n-1]    (x[-1] = 0)
7      y[n] = (dx * s)[n] = (x * h)[n]
8
9  Files provided: step_response.txt, input_signal.txt
10 Format: line 1 = "n_start n_end", line 2 = the samples.
11 """

```

```

12
13 import numpy as np
14 from signal_lti import DiscreteSignal, LTISystem
15
16
17 def read_signal_from_file(path):
18     # TODO: parse the two lines and check the sample count against the range.
19     raise NotImplementedError
20
21
22 def first_difference(signal):
23     """signal[n] - signal[n-1], using ONLY shift / add / multiply."""
24     # TODO
25     raise NotImplementedError
26
27
28 def impulse_from_step_response(step_response):
29     # TODO
30     raise NotImplementedError
31
32
33 def output_using_step_response(x, step_response):
34     """y[n] = (dx * s)[n], reusing your offline LTISystem."""
35     # TODO
36     raise NotImplementedError
37
38
39 def restrict(signal, start_time, end_time):
40     """Copy of 'signal' kept only on [start_time, end_time]."""
41     # TODO
42     raise NotImplementedError
43
44
45 def max_abs_diff(first, second):
46     # TODO
47     raise NotImplementedError
48
49
50 def main():
51     step_response_path = "step_response.txt"
52     input_signal_path = "input_signal.txt"
53
54     # TODO 1: read s[n] and x[n].
55     # TODO 2: recover h[n] from s[n].
56     #     Watch the range: shift(1) widens it by one sample, so the very last
57     #     sample of the difference is an artifact of s[n] being stored on a
58     #     finite window. Report it, then restrict h to the range of s.
59     # TODO 3: compute dx[n] and y_step = (dx * s).
60     # TODO 4: compute y_impulse = (x * h) and show the two agree.
61     # TODO 5: check that sum h[n] over the trimmed range equals the final value
62     #     of s[n] (the DC gain of the system).
63
64     # PLOTTING (write this part yourself):
65     # - Figure 1 (system identification): 2 stacked stem subplots, s[n] on top and
66     #   the recovered h[n] below. Both share the same n range, so you can build one
67     #   shared x-axis with plt.subplots(2, 1, sharex=True).
68     # - Figure 2 (input side): stem-plot x[n] and dx[n] in 2 rows.
69     # - Figure 3 (outputs): stem y_step[n] and y_impulse[n] in 2 rows, then a third
70     #   row with the sample-by-sample difference so the near-zero error is visible.
71     #   A log-scale y-axis on the error row makes the floating-point residue readable.
72     # - These signals are ~80 samples long, so thin the x-tick labels (the
73     #   readable_time_ticks helper from your offline does exactly this).
74     # - Save as p04_identification.png, p04_input.png, p04_outputs.png.
75
76
77 if __name__ == "__main__":
78     main()

```

Problem 4 — Solution

Solution — approach and discussion

The range artifact. This is the one place where your offline API behaves differently from the old `signal(INF)` class, and it is worth understanding before the lab. `s.shift(1)` returns a signal on $[-9, 31]$, so `s.add(s.shift(1).multiply(-1))` is stored on the union $[-10, 31]$ and produces

$$h[31] = s[31] - s[30] = 0 - 0.6 = -0.6.$$

That sample is pure fiction. The physical step response has already settled at 0.6 by $n = 30$ and stays there; the -0.6 only appears because the file stops. With `signal(INF)` the fixed window silently swallowed this. The fix is to restrict h back to the range of s , after which

$$\sum_{n=-10}^{30} h[n] = 0.6 = s[30],$$

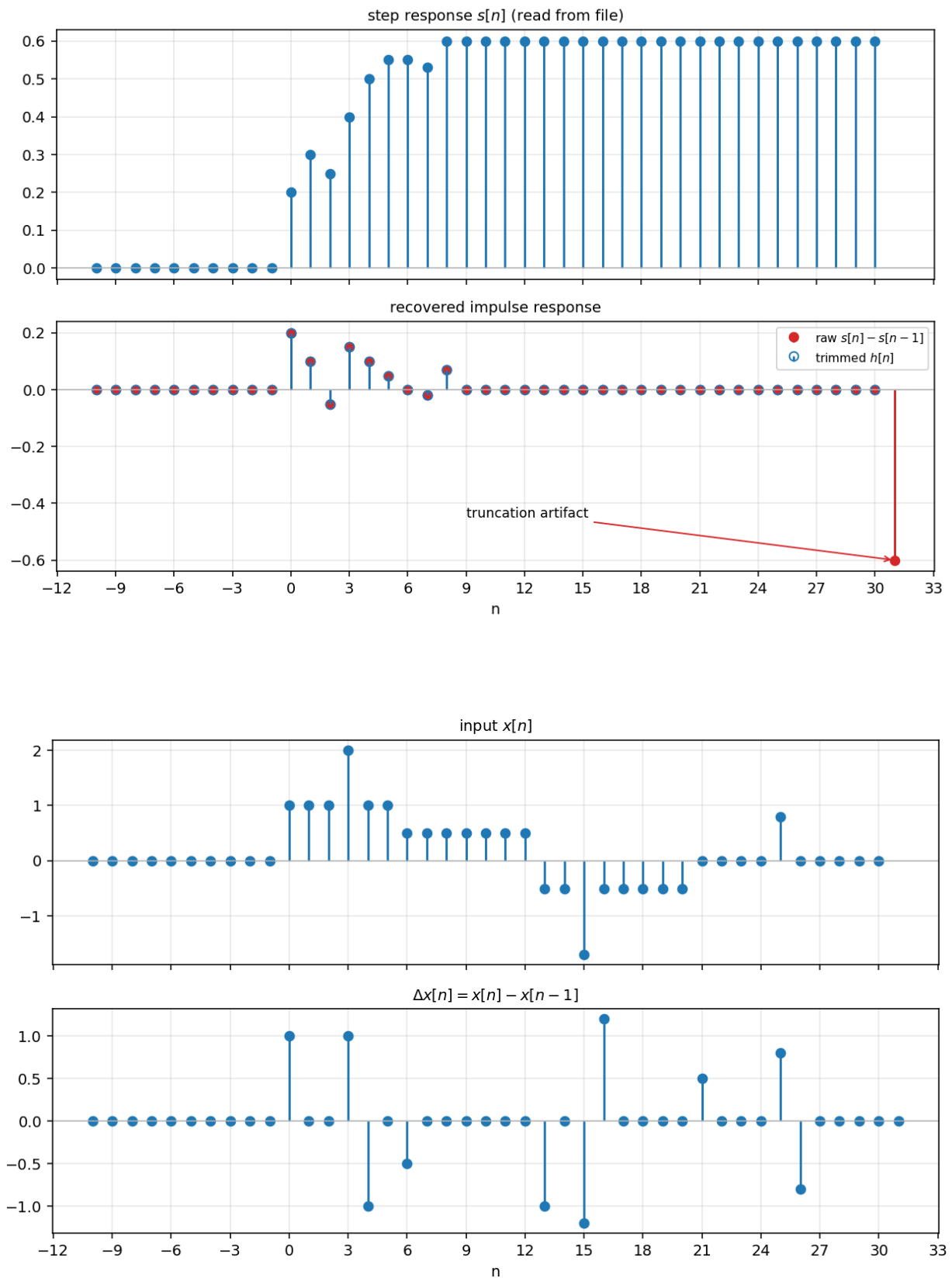
which is the DC gain of the system, as it must be for a telescoping sum.

Why the two routes agree exactly. Writing σ for the shift-by-one operator,

$$y_h = x * (s - \sigma s) = (x * s) - \sigma(x * s) = (x - \sigma x) * s = y_s,$$

so the identity is not an approximation — it holds because convolution commutes with shifting. The residual of 2×10^{-16} is round-off only. Note this means the truncation artifact affects *both* routes identically, so the verification passing does not prove h is physically right; the DC-gain check is what catches that.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 04 - Working from the step response instead of the impulse response
3  (SOLUTION)
4
5      h[n] = s[n] - s[n-1]          (s[-1] = 0)
6      dx[n] = x[n] - x[n-1]        (x[-1] = 0)
7      y[n] = (dx * s)[n] = (x * h)[n]
8  """
9
10 import numpy as np
11 import matplotlib.pyplot as plt
12 from signal_lti import DiscreteSignal, LTISystem
13
14
15 def read_signal_from_file(path):
16     """File format: line 1 = 'n_start n_end', line 2 = the samples."""
17     with open(path, "r", encoding="utf-8") as handle:
18         start_time, end_time = map(int, handle.readline().split())
19         values = list(map(float, handle.readline().split()))
20
21     if len(values) != end_time - start_time + 1:
22         raise ValueError(f"{path}: sample count does not match the declared range")
23
24     signal = DiscreteSignal(start_time, end_time)
25     for offset, value in enumerate(values):
26         signal.set_value_at_time(start_time + offset, value)
27     return signal
28
29
30 def first_difference(signal):
31     """signal[n] - signal[n-1], using only DiscreteSignal operations."""
32     return signal.add(signal.shift(1).multiply(-1))
33
34
35 def impulse_from_step_response(step_response):
36     """h[n] = s[n] - s[n-1]; samples before the stored range are already 0."""

```

```

37     return first_difference(step_response)
38
39
40 def output_using_step_response(x, step_response):
41     """y[n] = (dx * s)[n], reusing the offline LTISystem for the convolution."""
42     dx = first_difference(x)
43     return LTISystem(step_response).output(dx)
44
45
46 def restrict(signal, start_time, end_time):
47     """Copy of 'signal' kept only on [start_time, end_time]."""
48     trimmed = DiscreteSignal(start_time, end_time)
49     for n in range(start_time, end_time + 1):
50         trimmed.set_value_at_time(n, signal.get_value_at_time(n))
51     return trimmed
52
53
54 def max_abs_diff(first, second):
55     difference = first.add(second.multiply(-1))
56     return float(np.max(np.abs(difference.values)))
57
58
59 def summarize(name, signal, count=8):
60     head = " ".join(f"{v:.4f}" for v in signal.values[:count])
61     return f"{name}: range [{signal.start_time}, {signal.end_time}], first {count}: {head} ..."
62
63
64 def main():
65     s = read_signal_from_file("step_response.txt")
66     x = read_signal_from_file("input_signal.txt")
67
68     # Part 1: recover the impulse response from the step response
69     h = impulse_from_step_response(s)
70
71     # Part 2: output computed from the step response alone
72     dx = first_difference(x)
73     y_step = output_using_step_response(x, s)
74
75     # Part 3: cross-check with the ordinary impulse-response route
76     y_impulse = LTISystem(h).output(x)
77
78     difference = max_abs_diff(y_step, y_impulse)
79
80     print(summarize("s[n] ", s))
81     print(summarize("h[n] ", h))
82     print(summarize("x[n] ", x))
83     print(summarize("dx[n]", dx))
84     print(summarize("y_step[n] ", y_step))
85     print(summarize("y_impulse[n]", y_impulse))
86     print()
87     # The stored s[n] stops at n = 30 although the physical step response has
88     # already settled at 0.6 there. shift(1) widens the range to 31, so the
89     # difference invents a sample h[31] = 0 - s[30] = -0.6. That sample is a
90     # truncation artifact, not part of the system. Keep h on the range of s.
91     h_physical = restrict(h, s.start_time, s.end_time)
92     print(f"h before trimming: range [{h.start_time}, {h.end_time}], "
93           f"h[{h.end_time}] = {h.get_value_at_time(h.end_time):.4f} (truncation artifact)")
94     print(f"h after trimming : range [{h_physical.start_time}, {h_physical.end_time}], "
95           f"sum of h = {float(np.sum(h_physical.values)):.4f} "
96           f"f = final value of s[n] = {s.get_value_at_time(s.end_time):.4f} (DC gain)")
97     print(f"max |y_step[n] - y_impulse[n]| = {difference:.3e}")
98     print("The two routes agree:", difference < 1e-9)
99
100     # Figures described below; the implementation follows.
101     # - Figure 1 (system identification): 2 stacked stem subplots, s[n] on top and
102     #   the recovered h[n] below. Both share the same n range, so you can build one
103     #   shared x-axis with plt.subplots(2, 1, sharex=True).
104     # - Figure 2 (input side): stem-plot x[n] and dx[n] in 2 rows.
105     # - Figure 3 (outputs): stem y_step[n] and y_impulse[n] in 2 rows, then a third
106     #   row with the sample-by-sample difference so the near-zero error is visible.
107     #   A log-scale y-axis on the error row makes the floating-point residue readable.
108     # - These signals are ~80 samples long, so thin the x-tick labels (the
109     #   readable_time_ticks helper from your offline does exactly this).
110     # - Save as p04_identification.png, p04_input.png, p04_outputs.png.

```

```

111
112 # ---- Figures ----
113 # Note: the offline's readable_time_ticks() helper calls .append() on what
114 # times() returns, which is a numpy array, so it raises for long signals.
115 # MaxNLocator does the same job here without touching the offline file.
116 from matplotlib.ticker import MaxNLocator
117
118 # Figure 1: system identification. The trimmed h is drawn on top of the raw
119 # difference so the truncation artifact at the right-hand end stands out.
120 figure, axes = plt.subplots(2, 1, figsize=(9, 6), sharex=True)
121 axes[0].stem(s.times(), s.values, basefmt=" ")
122 axes[0].set_title("step response $s[n]$ (read from file)", fontsize=10)
123
124 axes[1].stem(h.times(), h.values, linefmt="C3-", markerfmt="C3o",
125             basefmt=" ", label="raw $s[n]-s[n-1]$")
126 markers, _, _ = axes[1].stem(h_physical.times(), h_physical.values, linefmt="C0-",
127                               markerfmt="C0o", basefmt=" ", label="trimmed $h[n]$")
128 markers.set_markerfacecolor("none")
129 axes[1].annotate("truncation artifact", xy=(h.end_time, h.get_value_at_time(h.end_time)),
130                xytext=(h.end_time - 22, -0.45), fontsize=9,
131                arrowprops=dict(arrowstyle="->", color="C3"))
132 axes[1].set_title("recovered impulse response", fontsize=10)
133 axes[1].legend(fontsize=8)
134 for axis in axes:
135     axis.axhline(0, color="0.7", linewidth=0.8)
136     axis.grid(True, alpha=0.3)
137     axis.xaxis.set_major_locator(MaxNLocator(nbins=16, integer=True))
138 axes[-1].set_xlabel("n")
139 figure.tight_layout()
140 figure.savefig("p04_identification.png", dpi=140)
141 plt.close.figure)
142
143 # Figure 2: the input side.
144 figure, axes = plt.subplots(2, 1, figsize=(9, 5.5), sharex=True)
145 for axis, (label, signal) in zip(axes, [("input $x[n]$", x),
146                                         (r"$\Delta x[n] = x[n]-x[n-1]$", dx)]):
147     axis.stem(signal.times(), signal.values, basefmt=" ")
148     axis.axhline(0, color="0.7", linewidth=0.8)
149     axis.set_title(label, fontsize=10)
150     axis.grid(True, alpha=0.3)
151     axis.xaxis.set_major_locator(MaxNLocator(nbins=16, integer=True))
152 axes[-1].set_xlabel("n")
153 figure.tight_layout()
154 figure.savefig("p04_input.png", dpi=140)
155 plt.close.figure)
156
157 # Figure 3: the two routes and their difference on a log axis.
158 error_signal = y_step.add(y_impulse.multiply(-1))
159 figure, axes = plt.subplots(3, 1, figsize=(9, 8), sharex=True)
160 axes[0].plot(y_step.times(), y_step.values, marker="o", markersize=3, linewidth=1)
161 axes[0].set_title(r"$y_s[n] = (\Delta x * s)[n]$", fontsize=10)
162 axes[1].plot(y_impulse.times(), y_impulse.values, marker="o", markersize=3,
163             linewidth=1, color="C1")
164 axes[1].set_title("$y_h[n] = (x * h)[n]$", fontsize=10)
165 axes[2].semilogy(error_signal.times(), np.abs(error_signal.values) + 1e-20,
166                 marker=".", linewidth=0.8, color="C3")
167 axes[2].set_title("$|y_s[n]-y_h[n]|$ (floating-point residue only)", fontsize=10)
168 for axis in axes:
169     axis.grid(True, alpha=0.3)
170     axis.xaxis.set_major_locator(MaxNLocator(nbins=16, integer=True))
171 axes[-1].set_xlabel("n")
172 figure.tight_layout()
173 figure.savefig("p04_outputs.png", dpi=140)
174 plt.close.figure)
175 print("\nsaved p04_identification.png, p04_input.png, p04_outputs.png")
176
177 if __name__ == "__main__":
178     main()

```

Expected console output

```
s[n] : range [-10, 30], first 8: 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 ...
h[n] : range [-10, 31], first 8: 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 ...
x[n] : range [-10, 30], first 8: 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 ...
dx[n]: range [-10, 31], first 8: 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 ...
y_step[n] : range [-20, 61], first 8: 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 ...
y_impulse[n]: range [-20, 61], first 8: 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 ...

h before trimming: range [-10, 31], h[31] = -0.6000 (truncation artifact)
h after trimming : range [-10, 30], sum of h = 0.6000 = final value of s[n] = 0.6000 (DC gain)
max |y_step[n] - y_impulse[n]| = 2.220e-16
The two routes agree: True

saved p04_identification.png, p04_input.png, p04_outputs.png
```

Problem 5. Deconvolution: recovering the input from the output

Total Marks: 10

Suggested Time: 30 minutes

New

Convolution can be run backwards. If $y = x * h$ and the first stored tap of h is nonzero, the earliest output sample involves only the earliest input sample, the next one involves only the first two, and so on. Unwinding gives

$$x[n - n_h] = \frac{1}{h[n_h]} \left(y[n] - \sum_{k < n - n_h} x[k] h[n - k] \right), \quad n_h = n_{h,\min}.$$

Every recovered sample is built from the ones before it, which is what makes this both elegant and fragile. Use $x[n] = \{2, -1, 0, 3, 1, -2, 0, 1, 2, -1\}$ on $[-1, 8]$, with $h[n] = \{1, -0.5, 0.25\}$ and later $h_{\text{bad}}[n] = \{0.25, -1, 0.5\}$, both on $[0, 2]$.

Tasks

1. Write `strip_leading_zeros(h)` so a leading zero tap does not cause a division by zero.
2. Write `deconvolve(y, h)`. Derive the input range from the output range first: y spans $[n_{x,\min} + n_{h,\min}, n_{x,\max} + n_{h,\max}]$.
3. Build $y = x * h$, recover x , and confirm the recovery is exact.
4. Re-convolve the recovered input as an independent check.
5. Perturb y by $10^{-3} \times$ standard normal noise, deconvolve again, and report the error and its amplification factor.
6. Repeat the noisy experiment with h_{bad} , print the error at every n , and explain the difference in behaviour.

Marks Distribution

Component	Marks
Correct input range derived from the output range	2
<code>deconvolve</code> forward substitution	4
Exact recovery demonstrated and re-convolution check	2
Noisy experiment and error amplification reported	1
Correct explanation of the h_{bad} blow-up	1
Total	10

Template

```

1  """
2  Problem 05 - Deconvolution: recovering the input from the output (TEMPLATE)
3
4  Given  $y = x * h$  with a nonzero leading tap, the convolution sum can be unwound
5  one sample at a time:
6       $x[n - n_h] = (y[n] - \sum_{k < n - n_h} x[k] h[n - k]) / h[n_h]$ .
7  """
8
9  import numpy as np

```

```

10 from signal_lti import DiscreteSignal, LTISystem
11
12
13 def make_signal(start_time, values):
14     # TODO
15     raise NotImplementedError
16
17
18 def max_abs_diff(first, second):
19     # TODO
20     raise NotImplementedError
21
22
23 def strip_leading_zeros(h, tolerance=1e-12):
24     """Move the stored start of h forward past any leading zero taps."""
25     # TODO
26     raise NotImplementedError
27
28
29 def deconvolve(y, h):
30     """
31     Recover x from  $y = x * h$  by forward substitution.
32     Work out the input range from the output range first:
33     y spans  $[x_{\text{start}} + h_{\text{start}}, x_{\text{end}} + h_{\text{end}}]$ .
34     """
35     # TODO
36     raise NotImplementedError
37
38
39 def main():
40     x_true = make_signal(-1, [2, -1, 0, 3, 1, -2, 0, 1, 2, -1])
41     h = make_signal(0, [1, -0.5, 0.25])
42
43     # TODO 1: build  $y = x_{\text{true}} * h$ , then recover x with deconvolve and compare.
44     # TODO 2: re-convolve the recovered input as an independent check.
45     # TODO 3: add a tiny perturbation ( $1e-3 * \text{standard normal}$ ) to y, deconvolve
46     #          again, and report the resulting error and its amplification factor.
47     # TODO 4: repeat everything with  $h_{\text{bad}} = [0.25, -1.0, 0.5]$ , print the error at
48     #          every n, and explain in one or two printed lines why one filter is
49     #          well behaved and the other is not. (Hint: look at where the roots of
50     #           $0.25 - z^{-1} + 0.5 z^{-2}$  sit relative to the unit circle.)
51
52     # PLOTTING (write this part yourself):
53     # - Figure 1: 3 stacked stem subplots -  $x_{\text{true}}[n]$ ,  $h[n]$ ,  $y[n]$ .
54     # - Figure 2: overlay  $x_{\text{true}}$  and  $x_{\text{recovered}}$  on one axes with two marker
55     #               styles (e.g. 'o' hollow and 'x'), so exact recovery shows as coincident marks.
56     # - Figure 3: stem the recovery error for the corrupted case. Use a semilogy
57     #               axis of |error| for  $h_{\text{bad}}$  so the geometric growth shows as a straight line.
58     # - xlabel "n", grid on, legends on the overlays, save as
59     #       p05_forward.png, p05_recovered.png, p05_noise_error.png.
60
61
62 if __name__ == "__main__":
63     main()

```

Problem 5 — Solution

Solution — approach and discussion

On clean data both filters give exact recovery — the residual is 0, not merely small, because forward substitution performs the same multiplications the forward convolution did.

The instructive part is what noise does. For $h = \{1, -0.5, 0.25\}$ a perturbation of 10^{-3} in y produces an error of about 1.8×10^{-3} in x : bounded, barely amplified. For $h_{\text{bad}} = \{0.25, -1, 0.5\}$ the same perturbation produces an error of roughly 766, and printing it sample by sample shows it multiplying by about 3.41 at every step.

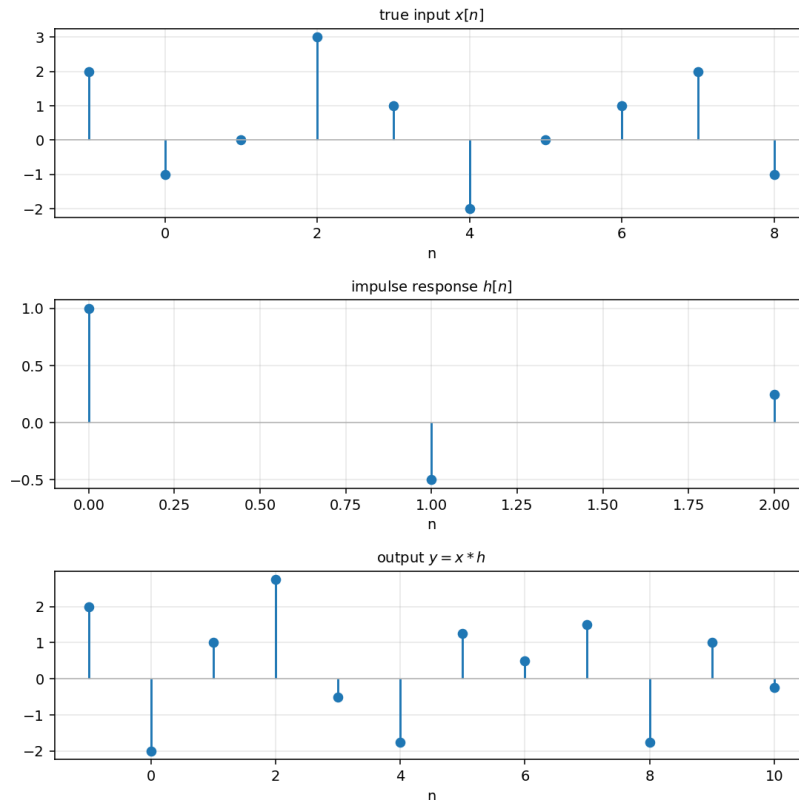
The reason is the location of the roots. Deconvolution is filtering by $1/H(z)$, whose poles are the zeros of

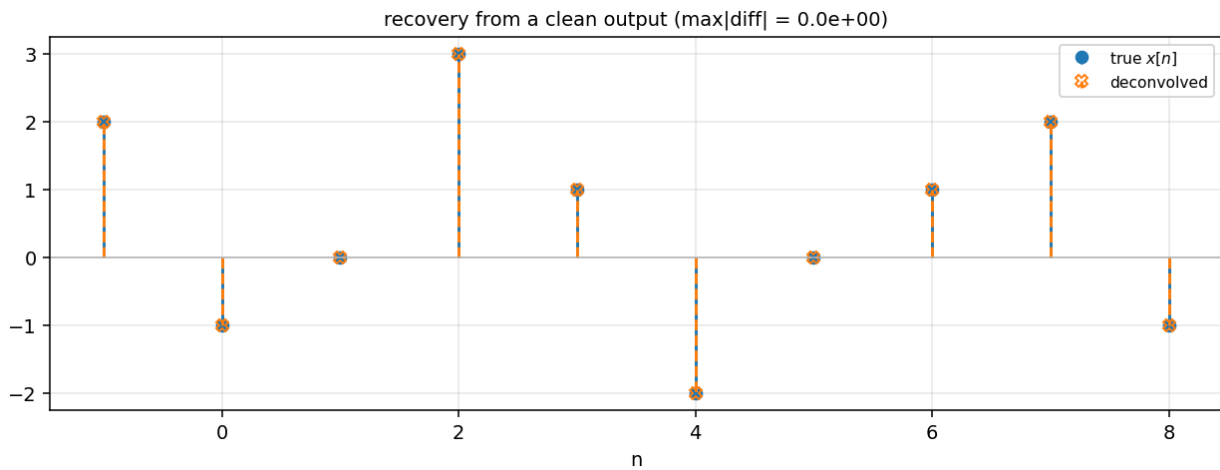
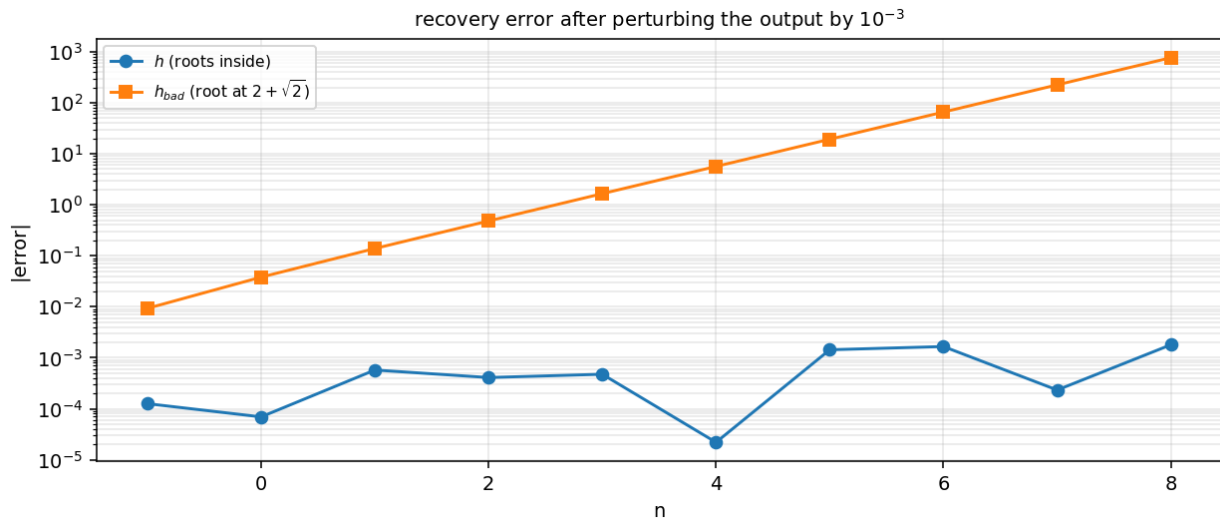
$$H(z) = 0.25 - z^{-1} + 0.5z^{-2} \implies z^2 - 4z + 2 = 0 \implies z = 2 \pm \sqrt{2} \approx 3.41, 0.586.$$

The root at 3.41 sits outside the unit circle, so the inverse filter is unstable and the recursion multiplies every past error by 3.41 — which is exactly the growth rate seen in the printed table. For $h = \{1, -0.5, 0.25\}$ the roots are $0.25 \pm 0.43j$, both of modulus 0.5, so early errors decay instead.

The practical moral: deconvolution is an inverse problem, and the impulse response decides whether it is a well-posed one. A small leading tap is a warning sign both because you divide by it and because it usually signals roots outside the unit circle.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 05 - Deconvolution: recovering the input from the output (SOLUTION)
3
4  Given  $y = x * h$  with  $h[n_h] \neq 0$  at its first stored index, the convolution sum
5   $y[n] = \sum_k x[k] h[n-k]$ 
6  can be unwound one sample at a time. The earliest output sample only involves the
7  earliest input sample, so
8   $x[n - n_h] = (y[n] - \sum_{k < n - n_h} x[k] h[n-k]) / h[n_h]$ .
9  """
10
11 import numpy as np
12 import matplotlib.pyplot as plt
13 from signal_lti import DiscreteSignal, LTISystem
14
15
16 def make_signal(start_time, values):
17     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
18     for offset, value in enumerate(values):
19         signal.set_value_at_time(start_time + offset, value)
20     return signal
21
22
23 def max_abs_diff(first, second):

```

```

24     difference = first.add(second.multiply(-1))
25     return float(np.max(np.abs(difference.values)))
26
27
28 def describe(name, signal):
29     values = " ".join(f"{v:+.4f}" for v in signal.values)
30     return f"{name}: range [{signal.start_time}, {signal.end_time}] -> {values}"
31
32
33 def strip_leading_zeros(h, tolerance=1e-12):
34     """Move the stored start of h forward past any leading zero taps."""
35     start_time = h.start_time
36     while start_time <= h.end_time and abs(h.get_value_at_time(start_time)) <= tolerance:
37         start_time += 1
38     if start_time > h.end_time:
39         raise ValueError("impulse response is identically zero")
40
41     trimmed = DiscreteSignal(start_time, h.end_time)
42     for n in range(start_time, h.end_time + 1):
43         trimmed.set_value_at_time(n, h.get_value_at_time(n))
44     return trimmed
45
46
47 def deconvolve(y, h):
48     """Recover x from y = x * h by forward substitution."""
49     h = strip_leading_zeros(h)
50     leading_tap = h.get_value_at_time(h.start_time)
51
52     # y spans [x_start + h_start, x_end + h_end], so the input range follows.
53     x_start = y.start_time - h.start_time
54     x_end = y.end_time - h.end_time
55     if x_end < x_start:
56         raise ValueError("output is shorter than the impulse response")
57
58     x = DiscreteSignal(x_start, x_end)
59     for k in range(x_start, x_end + 1):
60         n = k + h.start_time # output sample that exposes x[k]
61         already_explained = 0.0
62         for j in range(x_start, k): # contributions of earlier inputs
63             already_explained += x.get_value_at_time(j) * h.get_value_at_time(n - j)
64         x.set_value_at_time(k, (y.get_value_at_time(n) - already_explained) / leading_tap)
65     return x
66
67
68 def main():
69     x_true = make_signal(-1, [2, -1, 0, 3, 1, -2, 0, 1, 2, -1])
70     h = make_signal(0, [1, -0.5, 0.25])
71
72     y = LTISystem(h).output(x_true)
73
74     # ---- Part 1: exact recovery ----
75     x_recovered = deconvolve(y, h)
76     error_clean = max_abs_diff(x_true, x_recovered)
77
78     # ---- Part 2: re-convolve as an independent check ----
79     y_check = LTISystem(h).output(x_recovered)
80     residual = max_abs_diff(y, y_check)
81
82     # ---- Part 3: the same problem with a slightly corrupted output ----
83     rng = np.random.default_rng(0)
84     y_noisy = DiscreteSignal(y.start_time, y.end_time)
85     y_noisy.values = y.values + 1e-3 * rng.normal(size=len(y))
86     x_from_noisy = deconvolve(y_noisy, h)
87     error_noisy = max_abs_diff(x_true, x_from_noisy)
88
89     print(describe("x_true", x_true))
90     print(describe("h", h))
91     print(describe("y = x*h", y))
92     print(describe("x_recovered", x_recovered))
93     print(describe("x_from_noisy", x_from_noisy))
94     print()
95     print(f"max |x_true - x_recovered| (clean output) = {error_clean:.3e}")
96     print(f"max |y - x_recovered*h| = {residual:.3e}")
97     print(f"output perturbation added = 1.0e-03")

```

```

98     print(f"max |x_true - x_from_noisy| (noisy output) = {error_noisy:.3e}")
99     print(f"error amplification factor                = {error_noisy / 1e-3:.2f}x")
100
101     # ---- Part 4: an ill-conditioned impulse response ----
102     # h_bad has a small leading tap, so every recovered sample is divided by 0.25
103     # and the recursion multiplies past errors instead of damping them.
104     h_bad = make_signal(0, [0.25, -1.0, 0.5])
105     y_bad = LTISystem(h_bad).output(x_true)
106     y_bad_noisy = DiscreteSignal(y_bad.start_time, y_bad.end_time)
107     y_bad_noisy.values = y_bad.values + 1e-3 * rng.normal(size=len(y_bad))
108
109     x_bad_clean = deconvolve(y_bad, h_bad)
110     x_bad_noisy = deconvolve(y_bad_noisy, h_bad)
111     error_bad = max_abs_diff(x_true, x_bad_noisy)
112
113     print()
114     print(describe("h_bad", h_bad))
115     print(f"max |x_true - x_recovered| with h_bad, clean output = "
116           f"{max_abs_diff(x_true, x_bad_clean):.3e}")
117     print(f"max |x_true - x_recovered| with h_bad, noisy output = {error_bad:.3e}")
118     print(f"error amplification factor                = {error_bad / 1e-3:.1f}x")
119     print("per-sample error for h_bad:")
120     for n in range(x_true.start_time, x_true.end_time + 1):
121         gap = x_bad_noisy.get_value_at_time(n) - x_true.get_value_at_time(n)
122         print(f"  n = {n:3d}    error = {gap:+.5f}")
123
124     print()
125     print("Deconvolution is exact on clean data for both filters. With a corrupted")
126     print("output the behaviour splits: h has all its roots inside the unit circle so")
127     print("early errors die out, while h_bad has a root outside it, so each step")
128     print("multiplies the previous error and the recovery blows up along n.")
129
130     # Figures described below; the implementation follows.
131     # - Figure 1: 3 stacked stem subplots - x_true[n], h[n], y[n].
132     # - Figure 2: overlay x_true and x_recovered on one axes with two marker
133     #   styles (e.g. 'o' hollow and 'x'), so exact recovery shows as coincident marks.
134     # - Figure 3: stem the recovery error x_true - x_from_noisy for the corrupted case,
135     #   and note in the title how it grows with n.
136     # - Set xlabel "n", grid on, add a legend on the overlaid figure,
137     #   and save as p05_forward.png, p05_recovered.png, p05_noise_error.png.
138
139     # ---- Figures ----
140     # Figure 1: the forward problem.
141     figure, axes = plt.subplots(3, 1, figsize=(8, 8))
142     for axis, (label, signal) in zip(axes, [("true input $x[n]$", x_true),
143                                           ("impulse response $h[n]$", h),
144                                           ("output $y = x*h$", y)]):
145         axis.stem(signal.times(), signal.values, basefmt=" ")
146         axis.axhline(0, color="0.7", linewidth=0.8)
147         axis.set_title(label, fontsize=10)
148         axis.set_xlabel("n")
149         axis.grid(True, alpha=0.3)
150     figure.tight_layout()
151     figure.savefig("p05_forward.png", dpi=140)
152     plt.close(figure)
153
154     # Figure 2: exact recovery. Coincident markers mean the recovery is exact.
155     figure, axis = plt.subplots(figsize=(9, 3.6))
156     axis.stem(x_true.times(), x_true.values, linefmt="C0-",
157              markerfmt="C0o", basefmt=" ", label="true $x[n]$")
158     markers, _, _ = axis.stem(x_recovered.times(), x_recovered.values, linefmt="C1--",
159                             markerfmt="C1x", basefmt=" ", label="deconvolved")
160     markers.set_markerfacecolor("none")
161     axis.axhline(0, color="0.7", linewidth=0.8)
162     axis.set_title(f"recovery from a clean output (max|diff| = {error_clean:.1e})", fontsize=10)
163     axis.set_xlabel("n")
164     axis.grid(True, alpha=0.3)
165     axis.legend(fontsize=8)
166     figure.tight_layout()
167     figure.savefig("p05_recovered.png", dpi=140)
168     plt.close(figure)
169
170     # Figure 3: error growth on a log axis. A straight line means geometric growth,
171     # and its slope is the modulus of the root outside the unit circle.

```

```

172     times = x_true.times()
173     error_good = np.array([abs(x_from_noisy.get_value_at_time(n)
174                             - x_true.get_value_at_time(n)) for n in times])
175     error_bad = np.array([abs(x_bad_noisy.get_value_at_time(n)
176                             - x_true.get_value_at_time(n)) for n in times])
177
178     figure, axis = plt.subplots(figsize=(9, 4))
179     axis.semilogy(times, error_good + 1e-18, marker="o", label="$h$ (roots inside)")
180     axis.semilogy(times, error_bad + 1e-18, marker="s", label="$h_{bad}$ (root at $2+\\sqrt{2}$)")
181     axis.set_title("recovery error after perturbing the output by $10^{-3}$", fontsize=10)
182     axis.set_xlabel("n")
183     axis.set_ylabel("|error|")
184     axis.grid(True, alpha=0.3, which="both")
185     axis.legend(fontsize=8)
186     figure.tight_layout()
187     figure.savefig("p05_noise_error.png", dpi=140)
188     plt.close(figure)
189     print("\nsaved p05_forward.png, p05_recovered.png, p05_noise_error.png")
190
191 if __name__ == "__main__":
192     main()

```

Expected console output

```

x_true      : range [-1, 8] -> +2.0000 -1.0000 +0.0000 +3.0000 +1.0000 -2.0000 +0.0000 +1.0000 +2.0000 -1.0000
h           : range [0, 2] -> +1.0000 -0.5000 +0.2500
y = x*h     : range [-1, 10] -> +2.0000 -2.0000 +1.0000 +2.7500 -0.5000 -1.7500 +1.2500 +0.5000 +1.5000 -
              1.7500 +1.0000 -0.2500
x_recovered: range [-1, 8] -> +2.0000 -1.0000 +0.0000 +3.0000 +1.0000 -2.0000 +0.0000 +1.0000 +2.0000 -1.0000
x_from_noisy: range [-1, 8] -> +2.0001 -1.0001 +0.0006 +3.0004 +0.9995 -2.0000 +0.0014 +1.0017 +1.9998 -
              1.0018

max |x_true - x_recovered| (clean output) = 0.000e+00
max |y - x_recovered*h|              = 0.000e+00
output perturbation added              = 1.0e-03
max |x_true - x_from_noisy| (noisy output) = 1.796e-03
error amplification factor              = 1.80x

h_bad       : range [0, 2] -> +0.2500 -1.0000 +0.5000
max |x_true - x_recovered| with h_bad, clean output = 0.000e+00
max |x_true - x_recovered| with h_bad, noisy output = 7.666e+02
error amplification factor              = 766627.3x
per-sample error for h_bad:
  n = -1   error = -0.00930
  n =  0   error = -0.03808
  n =  1   error = -0.13869
  n =  2   error = -0.48152
  n =  3   error = -1.65089
  n =  4   error = -5.64179
  n =  5   error = -19.26373
  n =  6   error = -65.76718
  n =  7   error = -224.54178
  n =  8   error = -766.62730

```

Deconvolution is exact on clean data for both filters. With a corrupted output the behaviour splits: h has all its roots inside the unit circle so early errors die out, while h_{bad} has a root outside it, so each step multiplies the previous error and the recovery blows up along n .

saved p05_forward.png, p05_recovered.png, p05_noise_error.png

Problem 6. Cross-correlation and time-delay estimation

Total Marks: 10

Suggested Time: 25 minutes

New

Correlation is convolution with a time-reversed signal. Writing $\tilde{p}[m] = p[-m]$,

$$r_{yp}[l] = \sum_n y[n] p[n-l] = (y * \tilde{p})[l].$$

If $y[n] = A p[n-d]$ plus noise, then $r_{yp}[l]$ peaks at $l = d$, and the peak height divided by the energy of p estimates A . Everything is done with your existing convolution code plus one new signal operation. The received signal is built inside the template: a pattern $p[n] = \{1, 2, 3, 2, 1\}$ delayed by $d = 7$, scaled by 1.5, and buried in noise on the window $[-5, 25]$.

Tasks

1. Implement `flip(signal)` returning $g[n] = \text{signal}[-n]$. Get the new range right: it is $[-n_{\max}, -n_{\min}]$.
2. Implement `correlate(a, b)` as $a * \tilde{b}$ using your `LTISystem`.
3. Compute the autocorrelation r_{pp} . Confirm it peaks at $l = 0$, that the peak equals the energy of p , and that it is symmetric.
4. Compute r_{yp} , estimate $\hat{d}$ as its argmax, and compare with the true delay. Remember to return the *time index*, not the array position.
5. Estimate the amplitude from the peak height divided by the pattern energy.
6. Compute `correlate(pattern, y)` as well and explain why its peak sits at $-d$.

Marks Distribution

Component	Marks
<code>flip</code> with the correct reversed range	2
<code>correlate</code> built on the LTI system	2
Autocorrelation checks (peak at 0, equals energy, symmetric)	2
Delay estimate with argmax returning a time index	2
Amplitude estimate	1
Explanation of the non-commutativity of correlation	1
Total	10

Template

```

1  """
2  Problem 06 - Cross-correlation and time-delay estimation (TEMPLATE)
3
4       $r_{yp}[l] = \sum_n y[n] p[n-l] = (y * \text{flip}(p))[l]$ ,  $\text{flip}(p)[m] = p[-m]$ 
5  If  $y[n] = A * p[n-d]$  (+ noise) then  $r_{yp}[l]$  peaks at  $l = d$ .
6  """
7
8  import numpy as np
9  from signal_lti import DiscreteSignal, LTISystem
10
11
12  def make_signal(start_time, values):

```

```

13     # TODO
14     raise NotImplementedError
15
16
17 def flip(signal):
18     """Time reversal:  $g[n] = \text{signal}[-n]$ . Mind the new range."""
19     # TODO
20     raise NotImplementedError
21
22
23 def correlate(first, second):
24     """ $r[l] = \sum_n \text{first}[n] * \text{second}[n - l]$ , via convolution with flip(second)."""
25     # TODO
26     raise NotImplementedError
27
28
29 def argmax_time(signal):
30     """TIME INDEX of the largest sample, not the array position."""
31     # TODO
32     raise NotImplementedError
33
34
35 def main():
36     pattern = make_signal(0, [1, 2, 3, 2, 1])
37     true_delay = 7
38     amplitude = 1.5
39
40     rng = np.random.default_rng(220)
41     y = DiscreteSignal(-5, 25)
42     for n in range(y.start_time, y.end_time + 1):
43         clean = amplitude * pattern.get_value_at_time(n - true_delay)
44         y.set_value_at_time(n, clean + 0.30 * rng.normal())
45
46     # TODO 1: compute the autocorrelation of the pattern. Check it peaks at  $l = 0$ ,
47     #           that the peak equals the energy of  $p$ , and that it is symmetric.
48     # TODO 2: compute  $r_{yp}$  and estimate the delay as its argmax. Compare with true_delay.
49     # TODO 3: estimate the amplitude from the peak height divided by the pattern energy.
50     # TODO 4: compute correlate(pattern, y) as well and explain in a printed line why
51     #           its peak sits at  $-d$  instead of  $+d$ .
52     # TODO 5: print  $r_{yp}$  at a few lags either side of the peak.
53
54     # PLOTTING (write this part yourself):
55     # - Figure 1: 2 stacked stem subplots -  $p[n]$  on top, the received  $y[n]$  below.
56     #   Mark the true delay on the  $y[n]$  subplot with a vertical dashed line (axvline).
57     # - Figure 2: stem-plot  $r_{yp}[l]$  against its lag axis (use r_yp.times(), NOT 0..len-1,
58     #   because the lag axis starts at a negative number here). Draw a vertical line at
59     #   the estimated peak and annotate it with the value of  $\hat{d}$ .
60     # - Figure 3: stem  $r_{pp}[l]$  and check by eye that it is even and peaks at  $l = 0$ .
61     # - xlabel "lag  $l$ " on the correlation figures and " $n$ " on the signal figure,
62     #   grid on, save as p06_signals.png, p06_crosscorr.png, p06_autocorr.png.
63
64
65 if __name__ == "__main__":
66     main()

```

Problem 6 — Solution

Solution — approach and discussion

The two errors this problem is designed to catch are both about indices.

The range of `flip`. If p is stored on $[0, 4]$ then $\tilde{p}$ lives on $[-4, 0]$. Reversing the values array while leaving the range alone gives a silently wrong answer that still runs — and shifts every estimated delay by $n_{\max} + n_{\min}$.

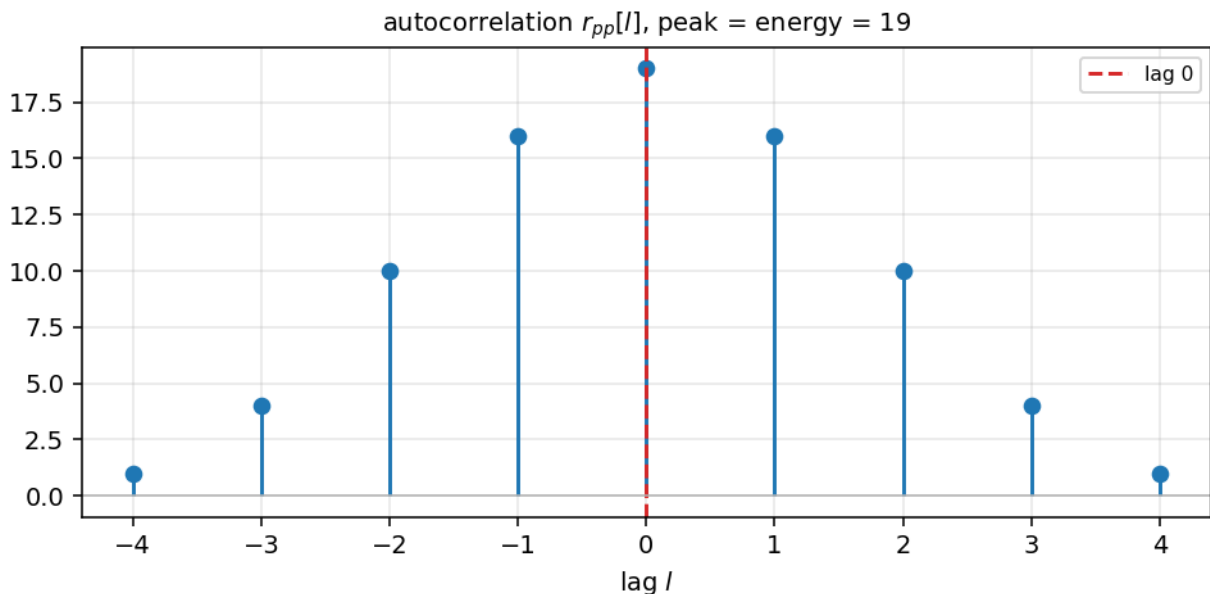
`argmax` is a position, not a time. `np.argmax(signal.values)` returns an offset into the array; the time index is `signal.start_time + position`. Here r_{yp} starts at $l = -9$, so forgetting the offset reports $\hat{d} = 16$ instead of 7.

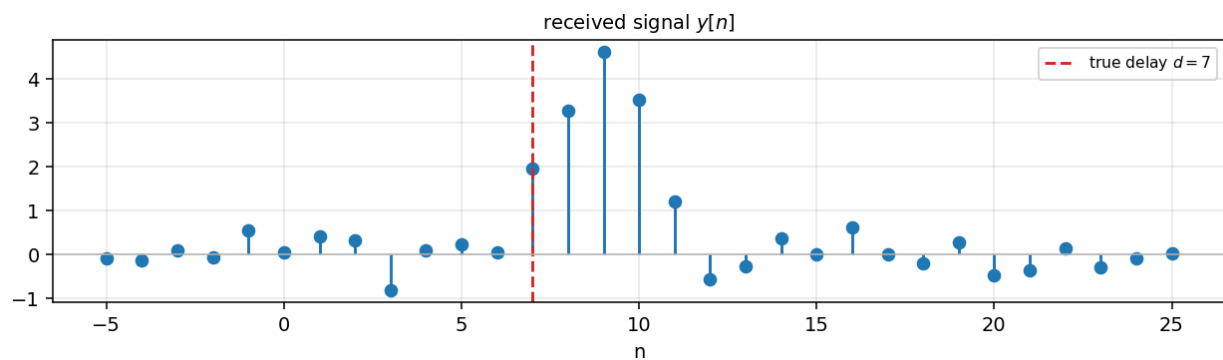
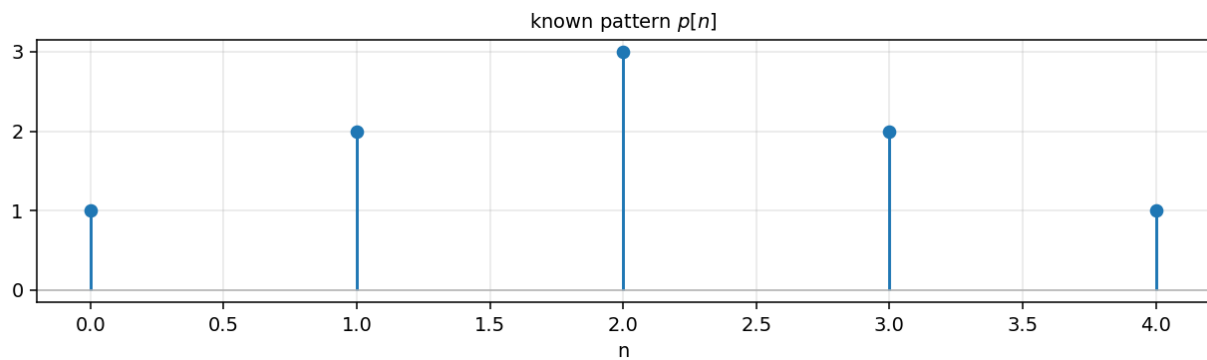
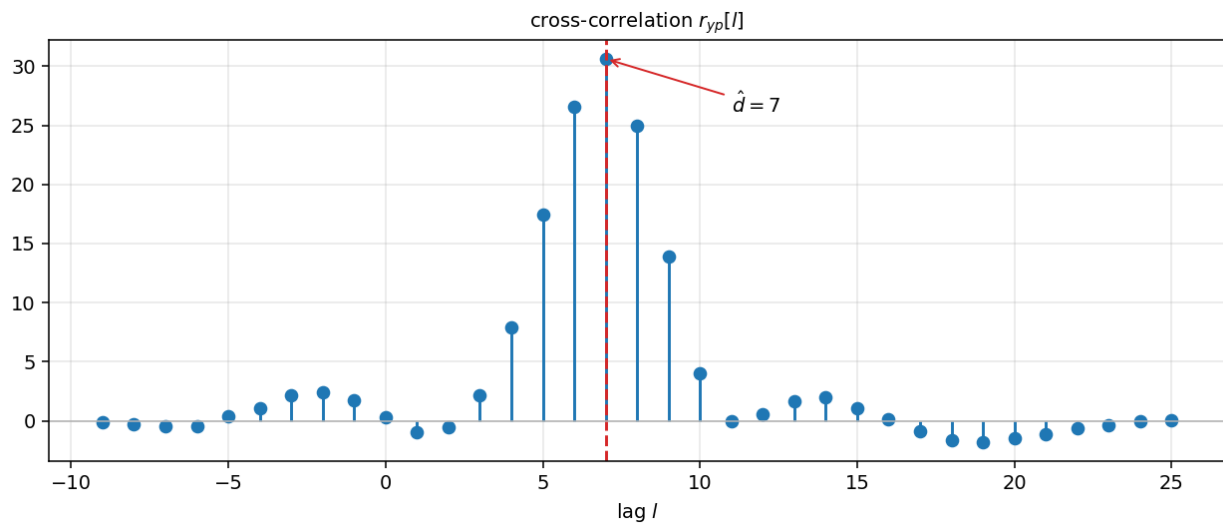
On the results: r_{pp} peaks at $19 = \sum_n p[n]^2$, as it must, since at zero lag every term is a square. The delay estimate is exact despite the noise, because the correlation peak is broad compared with the noise fluctuations. The amplitude estimate comes out at 1.61 against a true 1.5 — biased upward, since the noise that happens to sit under the pattern is counted as signal. Finally, correlation is *not* commutative:

$$r_{py}[l] = \sum_n p[n] y[n - l] = r_{yp}[-l],$$

so swapping the arguments mirrors the lag axis and the peak appears at -7 . Convolution is commutative; correlation only looks like it should be.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 06 - Cross-correlation and time-delay estimation (SOLUTION)
3
4  Correlation is convolution with a time-reversed signal:
5       $r_{yp}[l] = \sum_n y[n] p[n - l] = (y * p_{\text{flipped}})[l]$ ,    $p_{\text{flipped}}[m] = p[-m]$ .
6  If  $y[n] = A * p[n - d]$  (+ noise) then  $r_{yp}[l]$  peaks at  $l = d$ .
7  """
8
9  import numpy as np
10 import matplotlib.pyplot as plt
11 from signal_lti import DiscreteSignal, LTISystem
12
13 
```

```

14 def make_signal(start_time, values):
15     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
16     for offset, value in enumerate(values):
17         signal.set_value_at_time(start_time + offset, value)
18     return signal
19
20
21 def flip(signal):
22     """Time reversal: returns  $g[n] = \text{signal}[-n]$ ."""
23     reversed_signal = DiscreteSignal(-signal.end_time, -signal.start_time)
24     for n in range(signal.start_time, signal.end_time + 1):
25         reversed_signal.set_value_at_time(-n, signal.get_value_at_time(n))
26     return reversed_signal
27
28
29 def correlate(first, second):
30     """ $r[l] = \sum_n \text{first}[n] * \text{second}[n - l]$ , computed as  $\text{first} * \text{flip}(\text{second})$ ."""
31     return LTISystem(flip(second)).output(first)
32
33
34 def argmax_time(signal):
35     """Time index of the largest sample (not the array position)."""
36     position = int(np.argmax(signal.values))
37     return signal.start_time + position
38
39
40 def describe(name, signal):
41     values = " ".join(f"{v:+.3f}" for v in signal.values)
42     return f"{name}: range [{signal.start_time}, {signal.end_time}]-> {values}"
43
44
45 def main():
46     # A known pattern and a received copy of it, delayed and scaled.
47     pattern = make_signal(0, [1, 2, 3, 2, 1])
48     true_delay = 7
49     amplitude = 1.5
50
51     rng = np.random.default_rng(220)
52     y = DiscreteSignal(-5, 25)
53     for n in range(y.start_time, y.end_time + 1):
54         clean = amplitude * pattern.get_value_at_time(n - true_delay)
55         y.set_value_at_time(n, clean + 0.30 * rng.normal())
56
57     # ---- Task 1: autocorrelation must peak at lag 0 ----
58     r_pp = correlate(pattern, pattern)
59     print(describe("pattern p[n]", pattern))
60     print(describe("r_pp[l]", r_pp))
61     print(f"autocorrelation peaks at l = {argmax_time(r_pp)} "
62           f"(expected 0), peak value = {np.max(r_pp.values):.3f} "
63           f"= energy of p = {float(np.sum(pattern.values ** 2)):.3f}")
64     print(f"symmetric about 0: {np.allclose(r_pp.values, r_pp.values[::-1])}")
65     print()
66
67     # ---- Task 2: estimate the unknown delay ----
68     r_yp = correlate(y, pattern)
69     estimated_delay = argmax_time(r_yp)
70     print(f"correlation range      = [{r_yp.start_time}, {r_yp.end_time}]")
71     print(f"estimated delay d_hat      = {estimated_delay}")
72     print(f"true delay d                = {true_delay}")
73     print(f"estimate correct           = {estimated_delay == true_delay}")
74     print()
75
76     # ---- Task 3: recover the amplitude from the peak height ----
77     pattern_energy = float(np.sum(pattern.values ** 2))
78     estimated_amplitude = r_yp.get_value_at_time(estimated_delay) / pattern_energy
79     print(f"peak value r_yp[d_hat]      = {r_yp.get_value_at_time(estimated_delay):.3f}")
80     print(f"estimated amplitude         = {estimated_amplitude:.3f} (true {amplitude})")
81     print()
82
83     # ---- Task 4: correlation is NOT commutative ----
84     r_py = correlate(pattern, y)
85     print(f"r_py peaks at l = {argmax_time(r_py)}, i.e. at -d rather than +d.")
86     print(f"Swapping the arguments reverses the lag axis: r_py[l] = r_yp[-l].")
87

```

```

88     # a few lags around the peak, for the report
89     print("\nlags near the peak:")
90     for lag in range(estimated_delay - 3, estimated_delay + 4):
91         print(f"  l = {lag:3d}    r_yp = {r_yp.get_value_at_time(lag):+8.3f}")
92
93     # Figures described below; the implementation follows.
94     #   - Figure 1: 2 stacked stem subplots - p[n] on top, the received y[n] below.
95     #     Mark the true delay on the y[n] subplot with a vertical dashed line (axvline).
96     #   - Figure 2: stem-plot r_yp[l] against its lag axis (use r_yp.times(), NOT 0..len-1,
97     #     because the lag axis starts at a negative number here). Draw a vertical line at
98     #     the estimated peak and annotate it with the value of d_hat.
99     #   - Figure 3: stem r_pp[l] and check by eye that it is even and peaks at l = 0.
100    #   - xlabel "lag l" on the correlation figures and "n" on the signal figure,
101    #     grid on, save as p06_signals.png, p06_crosscorr.png, p06_autocorr.png.
102
103    # ---- Figures ----
104    # Figure 1: the pattern and the noisy signal it is hiding in.
105    figure, axes = plt.subplots(2, 1, figsize=(9, 5.5))
106    axes[0].stem(pattern.times(), pattern.values, basefmt=" ")
107    axes[0].set_title("known pattern $p[n]$", fontsize=10)
108    axes[1].stem(y.times(), y.values, basefmt=" ")
109    axes[1].axvline(true_delay, color="C3", linestyle="--",
110                   label=f"true delay $d = {true_delay}$")
111    axes[1].set_title("received signal $y[n]$", fontsize=10)
112    axes[1].legend(fontsize=8)
113    for axis in axes:
114        axis.axhline(0, color="0.7", linewidth=0.8)
115        axis.set_xlabel("n")
116        axis.grid(True, alpha=0.3)
117    figure.tight_layout()
118    figure.savefig("p06_signals.png", dpi=140)
119    plt.close.figure)
120
121    # Figure 2: the cross-correlation, plotted against its LAG axis. Using
122    # r_yp.times() rather than 0..len-1 is the whole point: the axis starts negative.
123    figure, axis = plt.subplots(figsize=(9, 4))
124    axis.stem(r_yp.times(), r_yp.values, basefmt=" ")
125    axis.axvline(estimated_delay, color="C3", linestyle="--")
126    axis.annotate(f"$\hat{d} = {estimated_delay}$",
127                xy=(estimated_delay, r_yp.get_value_at_time(estimated_delay)),
128                xytext=(estimated_delay + 4, r_yp.get_value_at_time(estimated_delay) * 0.85),
129                fontsize=10, arrowprops=dict(arrowstyle="->", color="C3"))
130    axis.axhline(0, color="0.7", linewidth=0.8)
131    axis.set_title("cross-correlation $r_{yp}[l]$", fontsize=10)
132    axis.set_xlabel("lag $l$")
133    axis.grid(True, alpha=0.3)
134    figure.tight_layout()
135    figure.savefig("p06_crosscorr.png", dpi=140)
136    plt.close.figure)
137
138    # Figure 3: autocorrelation - even, and peaking at lag 0.
139    figure, axis = plt.subplots(figsize=(7, 3.6))
140    axis.stem(r_pp.times(), r_pp.values, basefmt=" ")
141    axis.axvline(0, color="C3", linestyle="--", label="lag 0")
142    axis.axhline(0, color="0.7", linewidth=0.8)
143    axis.set_title(f"autocorrelation $r_{pp}[l]$, peak = energy = {pattern_energy:.0f}",
144                  fontsize=10)
145    axis.set_xlabel("lag $l$")
146    axis.grid(True, alpha=0.3)
147    axis.legend(fontsize=8)
148    figure.tight_layout()
149    figure.savefig("p06_autocorr.png", dpi=140)
150    plt.close.figure)
151    print("\nsaved p06_signals.png, p06_crosscorr.png, p06_autocorr.png")
152
153    if __name__ == "__main__":
154        main()

```

Expected console output

```
pattern p[n]: range [0, 4] -> +1.000 +2.000 +3.000 +2.000 +1.000
r_pp[l]      : range [-4, 4] -> +1.000 +4.000 +10.000 +16.000 +19.000 +16.000 +10.000 +4.000 +1.000
autocorrelation peaks at l = 0 (expected 0), peak value = 19.000 = energy of p = 19.000
symmetric about 0: True

correlation range      = [-9, 25]
estimated delay d_hat  = 7
true delay d           = 7
estimate correct       = True

peak value r_yp[d_hat] = 30.626
estimated amplitude    = 1.612 (true 1.5)

r_py peaks at l = -7, i.e. at -d rather than +d.
Swapping the arguments reverses the lag axis: r_py[l] = r_yp[-l].

lags near the peak:
l = 4   r_yp = +7.930
l = 5   r_yp = +17.408
l = 6   r_yp = +26.566
l = 7   r_yp = +30.626
l = 8   r_yp = +24.932
l = 9   r_yp = +13.886
l = 10  r_yp = +4.054

saved p06_signals.png, p06_crosscorr.png, p06_autocorr.png
```

Problem 7. Matched-filter detection of a pattern buried in noise

Total Marks: 10
Suggested Time: 30 minutes
New

A known waveform $p[n]$ has been transmitted several times, at unknown instants, through a noisy channel. The *matched filter* for p is its time reverse, $h[n] = p[-n]$; among all filters of a given energy it maximises the output signal-to-noise ratio at the moment the pattern is aligned. Thresholding the filter output therefore locates every occurrence.

Two files are provided: `received_signal.txt` (120 samples starting at $n = -20$) and `pattern.txt` (6 samples). Both use the usual two-line format.

Files provided: `received_signal.txt`, `pattern.txt` (copy them next to your script before running).

Tasks

1. Read both files and report their ranges and the energy $\sum_n p[n]^2$.
2. Compute the matched-filter output $z[l] = (y * \tilde{p})[l]$ and report its peak.
3. Threshold at $0.55 \times$ the peak and detect the pattern start indices. Require a local maximum as well as a threshold crossing, otherwise one broad hump raises several detections.
4. Repeat with a 6-point moving average in place of the matched filter. Scale *both* filters to unit energy first, so both pass the same noise power and the comparison is fair.
5. Report peak divided by standard deviation for each, and compare against the predicted peaks $\|p\|$ and $\sum_n p[n]/\sqrt{6}$.
6. Print a detection report table.

Marks Distribution

Component	Marks
File reading and matched filter construction	2
Matched-filter output over the correct lag range	2
Threshold plus local-maximum detection logic	3
Fair (unit-energy) comparison against the moving average	2
Detection report	1
Total	10

Template

```

1  """
2  Problem 07 - Matched-filter detection of a pattern buried in noise (TEMPLATE)
3
4  The matched filter for a known pattern  $p[n]$  is its time reverse,  $h[n] = p[-n]$ .
5
6  Files provided: received_signal.txt, pattern.txt
7  Format: line 1 = "n_start n_end", line 2 = the samples.
8  """
9
10 import numpy as np
11 from signal_lti import DiscreteSignal, LTISystem
12
13

```

```

14 def read_signal_from_file(path):
15     # TODO
16     raise NotImplementedError
17
18
19 def make_signal(start_time, values):
20     # TODO
21     raise NotImplementedError
22
23
24 def flip(signal):
25     # TODO
26     raise NotImplementedError
27
28
29 def matched_filter_output(received, pattern):
30     """z[l] = (received * flip(pattern))[l]; large when the pattern starts at l."""
31     # TODO
32     raise NotImplementedError
33
34
35 def detect_peaks(signal, threshold):
36     """
37     Indices where the signal is above 'threshold' AND is a local maximum.
38     Without the local-maximum test one broad hump raises several detections.
39     """
40     # TODO
41     raise NotImplementedError
42
43
44 def main():
45     received_path = "received_signal.txt"
46     pattern_path = "pattern.txt"
47
48     # TODO 1: read the received signal and the pattern; report ranges and the
49     #           pattern energy.
50     # TODO 2: compute the matched-filter output and its peak.
51     # TODO 3: threshold at 0.55 * peak, detect the local maxima, and print the
52     #           list of detected pattern start indices.
53     # TODO 4: repeat with a 6-point moving average instead of the matched filter.
54     #           Scale BOTH filters to unit energy first, so both pass the same amount
55     #           of noise power and the comparison is fair. Report peak / std for each,
56     #           and compare against the predicted peaks ||p|| and sum(p)/sqrt(6).
57     # TODO 5: print a detection report table.
58
59     # PLOTTING (write this part yourself):
60     # - Figure 1: plot the received signal over its own times() axis. With 120
61     #   samples the stems get crowded, so consider plot() with markers instead of
62     #   stem(), and thin the x-tick labels.
63     # - Figure 2: plot the matched-filter output z[l] against z.times(). Draw the
64     #   threshold as a horizontal dashed line (axhline) and mark every detected peak
65     #   with a distinct marker (e.g. red triangles) plus an annotation of its index.
66     # - Figure 3: put the received signal and the moving-average output on two rows
67     #   so the difference in contrast is visible.
68     # - Add a legend to the detection figure, xlabel "n", grid on, and save as
69     #   p07_received.png, p07_matched.png, p07_comparison.png.
70
71
72 if __name__ == "__main__":
73     main()

```

Problem 7 — Solution

Solution — approach and discussion

The detector finds the pattern starting at $n = -12, 15, 58, 91$, which is exactly where the four copies were placed. Notice the detections are *start* indices: because $z[l] = \sum_n y[n]p[n-l]$, the correlation peaks at the lag where the pattern begins, not where it is centred.

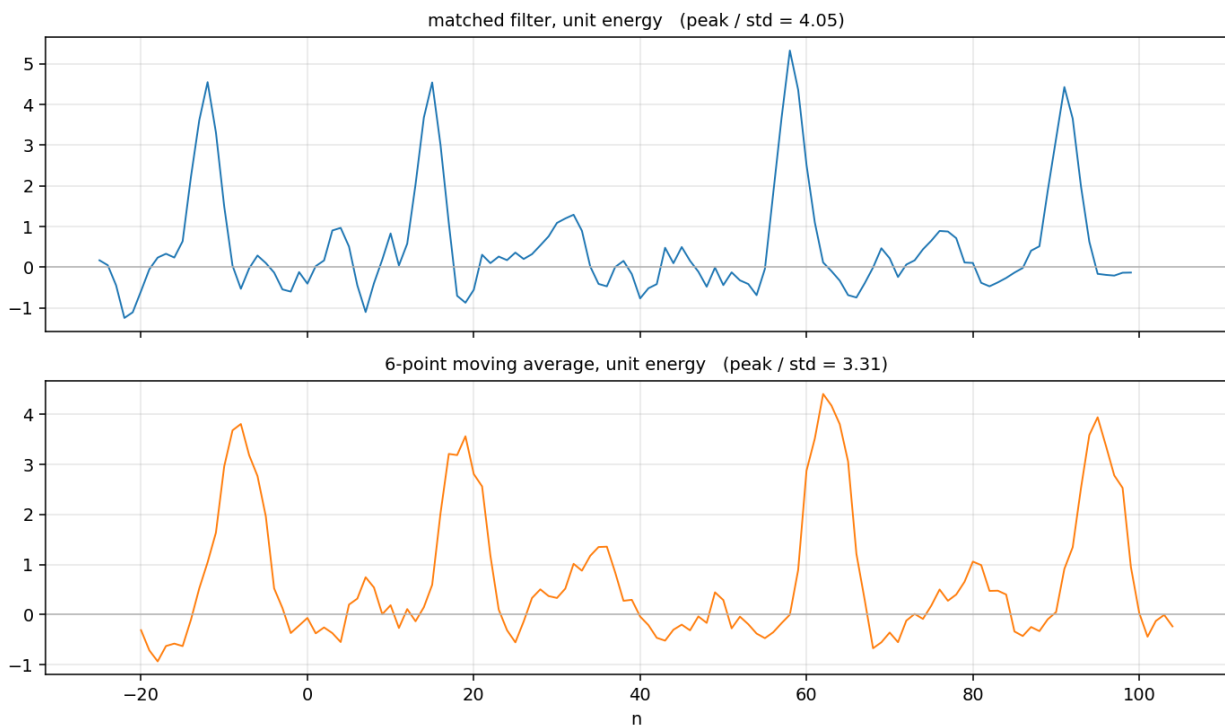
Why the local-maximum test matters. The matched-filter output around a detection is a broad hump roughly $2\times$ the pattern length wide. Thresholding alone would report every sample of that hump, giving four or five detections per true occurrence. Requiring $z[n] \geq z[n-1]$ and $z[n] > z[n+1]$ collapses each hump to a single index.

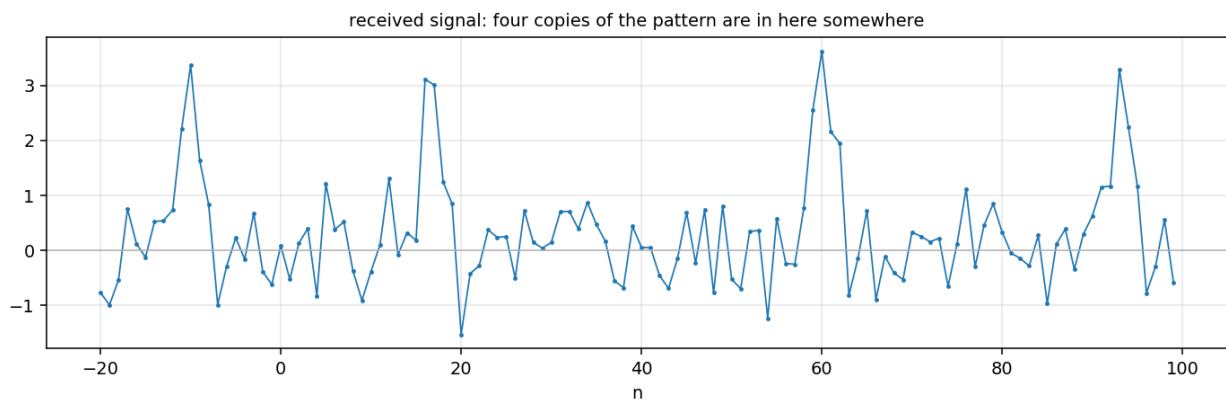
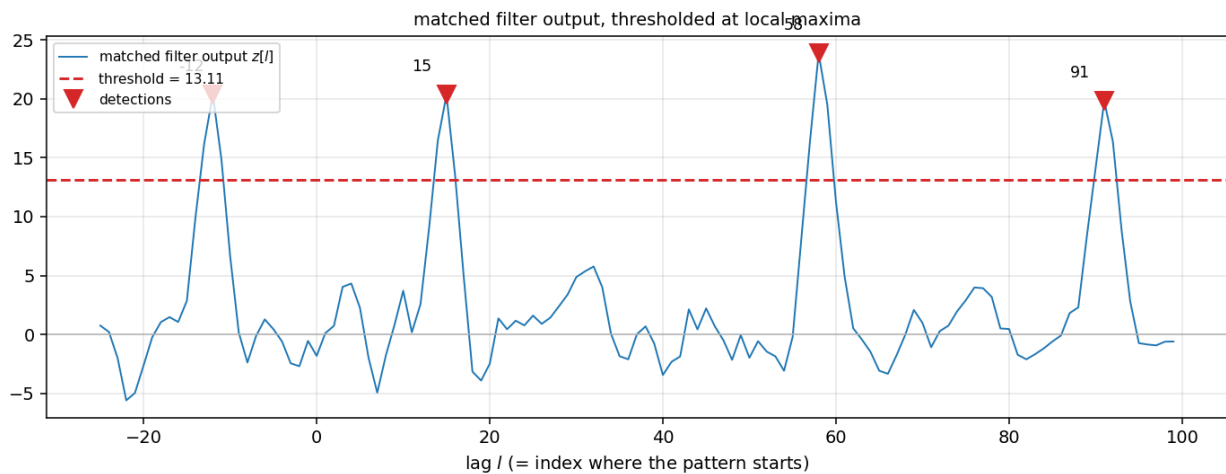
Why the comparison must be energy-normalised. A raw 6-point moving average has taps of $1/6$ and total energy $1/6$, so it attenuates *everything*, noise included, and would look artificially good on a raw peak comparison. Scaling both filters to unit energy fixes the output noise power, so the only remaining difference is how well each filter matches the pattern shape. Then the predicted peaks are

$$\text{matched: } \|p\| = \sqrt{20} \approx 4.47, \quad \text{average: } \frac{\sum_n p[n]}{\sqrt{6}} = \frac{8}{\sqrt{6}} \approx 3.27,$$

against measured contrasts of 4.05 and 3.31. The gap comes from the last tap of the pattern being -1 : the moving average adds it in with the same positive weight as the rest, so the pattern partially cancels its own peak, while the matched filter weights it by -1 and turns it into extra signal. This is the Cauchy–Schwarz statement that among unit-energy filters, the one proportional to p maximises the output at alignment.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 07 - Matched-filter detection of a pattern buried in noise (SOLUTION)
3
4  The matched filter for a known pattern  $p[n]$  is its time reverse,  $h[n] = p[-n]$ .
5  Convolution of the received signal with  $h$  maximises the output signal-to-noise ratio
6  at the instant the pattern is fully aligned, so thresholding the filter output
7  locates every occurrence of the pattern.
8  """
9
10 import numpy as np
11 import matplotlib.pyplot as plt
12 from signal_lti import DiscreteSignal, LTISystem
13
14
15 def read_signal_from_file(path):
16     with open(path, "r", encoding="utf-8") as handle:
17         start_time, end_time = map(int, handle.readline().split())
18         values = list(map(float, handle.readlines().split()))
19         if len(values) != end_time - start_time + 1:
20             raise ValueError(f"{path}: sample count does not match the declared range")
21         signal = DiscreteSignal(start_time, end_time)
22         for offset, value in enumerate(values):
23             signal.set_value_at_time(start_time + offset, value)
24         return signal
25
26
27 def make_signal(start_time, values):
28     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
29     for offset, value in enumerate(values):

```

```

30     signal.set_value_at_time(start_time + offset, value)
31     return signal
32
33
34 def flip(signal):
35     reversed_signal = DiscreteSignal(-signal.end_time, -signal.start_time)
36     for n in range(signal.start_time, signal.end_time + 1):
37         reversed_signal.set_value_at_time(-n, signal.get_value_at_time(n))
38     return reversed_signal
39
40
41 def matched_filter_output(received, pattern):
42     """
43     z[l] = sum_n received[n] * pattern[n-l] = (received * flip(pattern))[l].
44     z[l] is large when the pattern starts at index l.
45     """
46     return LTISystem(flip(pattern)).output(received)
47
48
49 def detect_peaks(signal, threshold):
50     """
51     Indices where the signal exceeds 'threshold' and is a local maximum.
52     The local-maximum test stops one broad hump from raising several detections.
53     """
54     detections = []
55     for n in range(signal.start_time, signal.end_time + 1):
56         value = signal.get_value_at_time(n)
57         if value < threshold:
58             continue
59         if value >= signal.get_value_at_time(n - 1) and value > signal.get_value_at_time(n + 1):
60             detections.append(n)
61     return detections
62
63
64 def main():
65     received = read_signal_from_file("received_signal.txt")
66     pattern = read_signal_from_file("pattern.txt")
67
68     pattern_energy = float(np.sum(pattern.values ** 2))
69     print(f"received: range [{received.start_time}, {received.end_time}], "
70           f"{len(received)} samples")
71     print(f"pattern : range [{pattern.start_time}, {pattern.end_time}], "
72           f"values {' '.join(f'{v:.1f}' for v in pattern.values)}, "
73           f"energy = {pattern_energy:.2f}")
74     print()
75
76     # ---- Task 1: matched-filter output ----
77     z = matched_filter_output(received, pattern)
78     print(f"matched filter output range = [{z.start_time}, {z.end_time}]")
79     print(f"peak value = {np.max(z.values):.3f}")
80
81     # ---- Task 2: threshold and detect ----
82     threshold = 0.55 * float(np.max(z.values))
83     detections = detect_peaks(z, threshold)
84     print(f"threshold = 0.55 * peak = {threshold:.3f}")
85     print(f"detected pattern start indices: {detections}")
86     print()
87
88     # ---- Task 3: compare against a plain 6-point moving average ----
89     # For a fair comparison both filters are scaled to unit energy, so both pass
90     # exactly the same amount of noise power; only the shape of the taps differs.
91     def unit_energy(signal):
92         norm = float(np.sqrt(np.sum(signal.values ** 2)))
93         return signal.multiply(1.0 / norm)
94
95     z_matched = LTISystem(unit_energy(flip(pattern))).output(received)
96     z_average = LTISystem(unit_energy(make_signal(0, [1.0] * 6))).output(received)
97
98     def contrast(signal):
99         """peak height measured in units of the signal's own standard deviation."""
100         return float(np.max(signal.values) / np.std(signal.values))
101
102     print(f"peak-to-noise contrast, matched filter = {contrast(z_matched):.2f}")
103     print(f"peak-to-noise contrast, moving average = {contrast(z_average):.2f}")

```

```

104 print(f"predicted peak, matched = ||p|| = {np.sqrt(pattern_energy):.3f}")
105 print(f"predicted peak, average = sum(p)/sqrt(6) = "
106       f"{float(np.sum(pattern.values)) / np.sqrt(6):.3f}")
107 print("The matched filter wins because it weights each sample in proportion to")
108 print("the pattern itself, while the moving average weights them all equally and")
109 print("lets the pattern's negative tap cancel part of its own peak.")
110 print()
111
112 # ---- Task 4: report the detections ----
113 print("detection report")
114 print(f"{'start n':>8} {'z[n]':>10} {'z/energy':>10}")
115 for n in detections:
116     value = z.get_value_at_time(n)
117     print(f"{n:8d} {value:10.3f} {value / pattern_energy:10.3f}")
118
119 # Figures described below; the implementation follows.
120 # - Figure 1: stem-plot the received signal over its own times() axis. With 120
121 #   samples the stems get crowded, so consider plot() with markers instead of
122 #   stem(), and thin the x-tick labels.
123 # - Figure 2: plot the matched-filter output z[l] against z.times(). Draw the
124 #   threshold as a horizontal dashed line (axhline) and mark every detected peak
125 #   with a distinct marker (e.g. red triangles) plus an annotation of its index.
126 # - Figure 3: put the received signal and the smoothed (moving-average) output on
127 #   two rows so the difference in contrast is visible.
128 # - Add a legend to the detection figure, xlabel "n", grid on, and save as
129 #   p07_received.png, p07_matched.png, p07_comparison.png.
130
131 # ---- Figures ----
132 # 120 samples is too many for a readable stem plot, so use lines with markers.
133 figure, axis = plt.subplots(figsize=(10, 3.4))
134 axis.plot(received.times(), received.values, linewidth=0.9, marker=".", markersize=3)
135 axis.axhline(0, color="0.7", linewidth=0.8)
136 axis.set_title("received signal: four copies of the pattern are in here somewhere",
137               fontsize=10)
138 axis.set_xlabel("n")
139 axis.grid(True, alpha=0.3)
140 figure.tight_layout()
141 figure.savefig("p07_received.png", dpi=140)
142 plt.close(figure)
143
144 # Figure 2: the detector. Threshold as a horizontal line, detections marked.
145 figure, axis = plt.subplots(figsize=(10, 4))
146 axis.plot(z.times(), z.values, linewidth=1.0, label="matched filter output $z[l]$")
147 axis.axhline(threshold, color="C3", linestyle="--",
148             label=f"threshold = {threshold:.2f}")
149 axis.plot(detections, [z.get_value_at_time(n) for n in detections],
150         "v", color="C3", markersize=10, linestyle="none", label="detections")
151 for n in detections:
152     axis.annotate(str(n), xy=(n, z.get_value_at_time(n)),
153                  xytext=(n - 4, z.get_value_at_time(n) + 2.0), fontsize=9)
154 axis.axhline(0, color="0.7", linewidth=0.8)
155 axis.set_title("matched filter output, thresholded at local maxima", fontsize=10)
156 axis.set_xlabel("lag $l$ (= index where the pattern starts)")
157 axis.grid(True, alpha=0.3)
158 axis.legend(fontsize=8, loc="upper left")
159 figure.tight_layout()
160 figure.savefig("p07_matched.png", dpi=140)
161 plt.close(figure)
162
163 # Figure 3: matched filter vs moving average, both at unit energy.
164 figure, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
165 axes[0].plot(z_matched.times(), z_matched.values, linewidth=1.0, color="C0")
166 axes[0].set_title(f"matched filter, unit energy (peak / std = {contrast(z_matched):.2f})",
167                 fontsize=10)
168 axes[1].plot(z_average.times(), z_average.values, linewidth=1.0, color="C1")
169 axes[1].set_title(f"6-point moving average, unit energy (peak / std = {contrast(z_average):.2f})",
170                 fontsize=10)
171 for axis in axes:
172     axis.axhline(0, color="0.7", linewidth=0.8)
173     axis.grid(True, alpha=0.3)
174 axes[-1].set_xlabel("n")
175 figure.tight_layout()
176 figure.savefig("p07_comparison.png", dpi=140)
177 plt.close(figure)

```

```
178     print("\nsaved p07_received.png, p07_matched.png, p07_comparison.png")
179
180     if __name__ == "__main__":
181         main()
```

Expected console output

```
received: range [-20, 99], 120 samples
pattern : range [0, 5], values +1.0 +2.0 +3.0 +2.0 +1.0 -1.0, energy = 20.00

matched filter output range = [-25, 99]
peak value = 23.830
threshold = 0.55 * peak = 13.107
detected pattern start indices: [-12, 15, 58, 91]

peak-to-noise contrast, matched filter = 4.05
peak-to-noise contrast, moving average = 3.31
predicted peak, matched = ||p|| = 4.472
predicted peak, average = sum(p)/sqrt(6) = 3.266
The matched filter wins because it weights each sample in proportion to
the pattern itself, while the moving average weights them all equally and
lets the pattern's negative tap cancel part of its own peak.

detection report
  start n      z[n]    z/energy
    -12     20.354     1.018
     15     20.320     1.016
     58     23.830     1.192
     91     19.812     0.991

saved p07_received.png, p07_matched.png, p07_comparison.png
```

Problem 8. A first-order recursive system and its truncated impulse response

Total Marks: 10

Suggested Time: 30 minutes

New

Consider the recursive (IIR) system

$$y[n] = x[n] + a y[n-1], \quad y[-1] = 0, \quad a = 0.8.$$

Its impulse response is $h[n] = a^n u[n]$, which never ends — so a finite `DiscreteSignal` can only hold a truncated version $h_N[n] = a^n$ for $0 \leq n < N$. Convolution with h_N is therefore an approximation, and the aim here is to measure how good an approximation it is. Use $x[n] = \{1, 1, 1, 1, 1, 0, 0, 0, 2, -1, 0, 0, 0, 0, 0\}$ on $[0, 15]$ and compare on the window $0 \leq n \leq 40$.

Tasks

1. Implement `recursive_output(x, a, start, end)` evaluating the difference equation directly. This is the exact answer.
2. Feed a unit impulse through the recursion and confirm the measured impulse response really is a^n .
3. For $N \in \{2, 5, 10, 20, 40, 80\}$ build h_N , convolve, and tabulate the maximum error against the exact output.
4. Put the predicted bound $(\sum_n |x[n]|) |a|^N / (1 - |a|)$ next to each measured error and comment on the ratio column.
5. For $a \in \{0.8, 1.0, 1.2\}$ report $\sum_n |h[n]|$ over 60 taps and state which systems are BIBO stable.

Marks Distribution

Component	Marks
Direct recursion implemented correctly with $y[-1] = 0$	3
Truncated h_N and verification against a^n	2
Error table across several N	3
Comparison against the predicted tail bound	1
Stability discussion	1
Total	10

Template

```

1  """
2  Problem 08 - A first-order recursive system and its truncated impulse response
3  (TEMPLATE)
4
5      y[n] = x[n] + a * y[n-1],    y[-1] = 0
6      h[n] = a^n u[n]    (infinite), truncated to h_N[n] = a^n for 0 <= n < N.
7  """
8
9  import numpy as np
10 from signal_lti import DiscreteSignal, LTISystem
11
12
```

```

13 def make_signal(start_time, values):
14     # TODO
15     raise NotImplementedError
16
17
18 def recursive_output(x, a, start_time, end_time):
19     """Direct evaluation of  $y[n] = x[n] + a*y[n-1]$  on a chosen window."""
20     # TODO
21     raise NotImplementedError
22
23
24 def truncated_impulse_response(a, length):
25     """ $h_N[n] = a^n$  for  $n = 0 \dots \text{length}-1$ ."""
26     # TODO
27     raise NotImplementedError
28
29
30 def max_abs_diff_on(first, second, start_time, end_time):
31     """Largest gap between two signals over a chosen comparison window."""
32     # TODO
33     raise NotImplementedError
34
35
36 def main():
37     a = 0.8
38     x = make_signal(0, [1, 1, 1, 1, 1, 0, 0, 0, 2, -1, 0, 0, 0, 0, 0, 0])
39     window_start, window_end = 0, 40
40
41     # TODO 1: compute the exact output from the difference equation.
42     # TODO 2: feed a unit impulse through the recursion and confirm the measured
43     #           impulse response really is  $a^n$ .
44     # TODO 3: for  $N$  in (2, 5, 10, 20, 40, 80) convolve  $x$  with  $h_N$  and tabulate the
45     #           max error against the exact output, next to the predicted bound
46     #           ( $\sum |x| * |a|^N / (1 - |a|)$ ). Comment on the ratio column.
47     # TODO 4: for  $a$  in (0.8, 1.0, 1.2) report  $\sum |h[n]|$  over 60 taps and state
48     #           which systems are BIBO stable.
49
50     # PLOTTING (write this part yourself):
51     # - Figure 1: 2 stacked stem subplots -  $x[n]$  on top and the exact  $y[n]$  below,
52     #   over the window 0..40. The decaying tail after the input stops is the point
53     #   to look at.
54     # - Figure 2: overlay the exact output with the convolution results for  $N = 2, 5$ 
55     #   and 20 on one axes (use plot with markers rather than stem so four curves stay
56     #   readable), and add a legend naming each  $N$ .
57     # - Figure 3: plot max error against  $N$  on a semilogy axis (plt.semilogy), together
58     #   with the predicted bound as a dashed line. A straight line on a log axis is the
59     #   visual proof that the error decays geometrically.
60     # - xlabel "n" (or "N" on figure 3), grid on, save as p08_response.png,
61     #   p08_truncation.png, p08_error_vs_N.png.
62
63
64 if __name__ == "__main__":
65     main()

```

Problem 8 — Solution

Solution — approach and discussion

The truncation error is governed entirely by the tail that was thrown away:

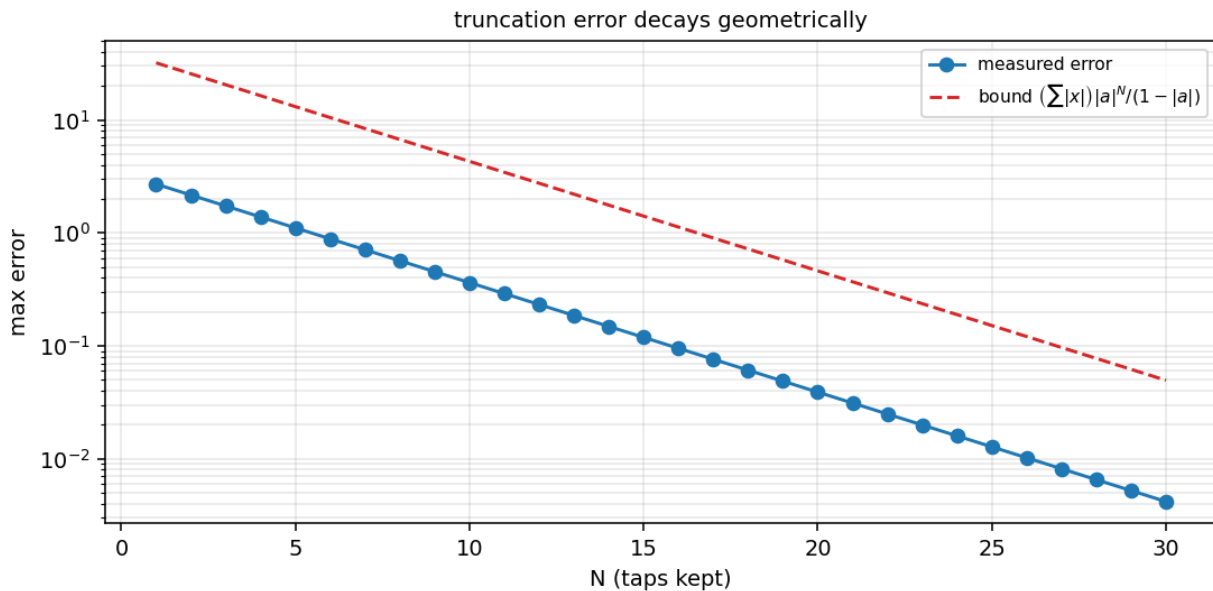
$$y[n] - y_N[n] = \sum_{m \geq N} a^m x[n-m] \implies |y[n] - y_N[n]| \leq \left(\max_n |x[n]| \right) \sum_{m \geq N} |a|^m = \left(\max_n |x[n]| \right) \frac{|a|^N}{1 - |a|}.$$

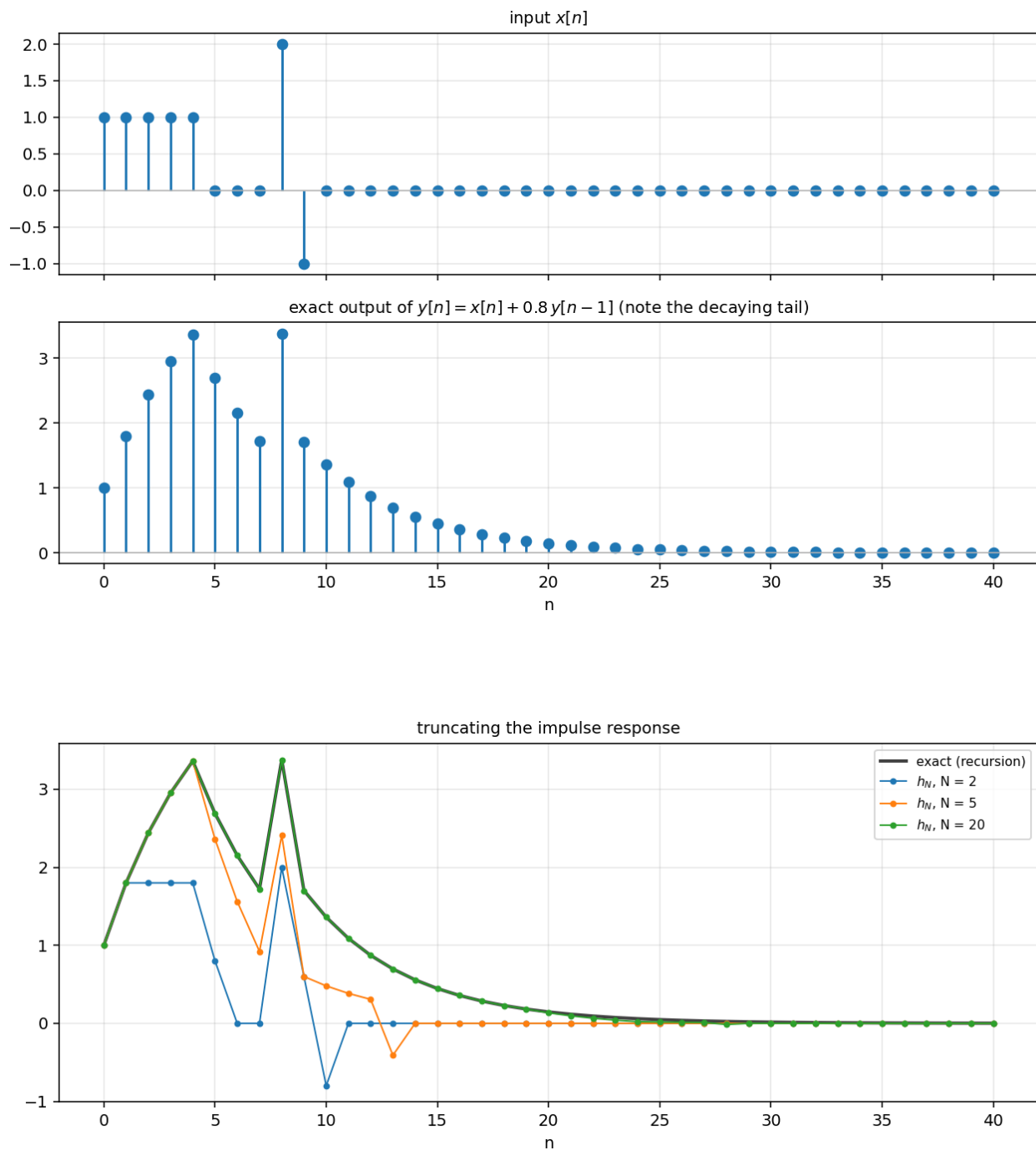
Using $\sum_n |x[n]|$ in place of $\max_n |x[n]|$, as the tasks ask, gives a looser but still valid bound. The measured errors sit at a constant 0.084 of that bound for $N = 2, 5, 10, 20$ — constant because both the error and the bound fall by exactly a per extra tap. On a semilogy plot both are straight lines of the same slope, which is the cleanest visual proof that the decay really is geometric.

At $N = 40$ and $N = 80$ the ratio collapses. That is not a change in behaviour: the comparison window only runs to $n = 40$, so once N exceeds the window the retained taps already cover every sample being compared and the error drops to round-off.

The stability point. $\sum_n |h[n]|$ over 60 taps is 5.0 for $a = 0.8$, 60 for $a = 1.0$ and about 2.8×10^5 for $a = 1.2$. In the limit the first converges to $1/(1 - 0.8) = 5$ and the other two diverge. This is the trap worth internalising: *any* finite-length impulse response has a finite absolute sum, so a truncated model of an unstable system looks perfectly stable. Truncation does not just lose accuracy, it can lose the qualitative behaviour entirely.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 08 - A first-order recursive system and its truncated impulse response
3  (SOLUTION)
4
5       $y[n] = x[n] + a * y[n-1]$ ,  $y[-1] = 0$ 
6  Its impulse response is  $h[n] = a^n u[n]$ , which never ends. A finite
7  DiscreteSignal can only hold a truncated version  $h_N[n] = a^n$  for  $0 \leq n < N$ ,
8  so convolving with  $h_N$  is an approximation whose error is governed by the tail
9  sum  $|a|^N / (1 - |a|)$ .
10 """
11
12 import numpy as np
13 import matplotlib.pyplot as plt

```

```

14 from signal_lti import DiscreteSignal, LTISystem
15
16
17 def make_signal(start_time, values):
18     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
19     for offset, value in enumerate(values):
20         signal.set_value_at_time(start_time + offset, value)
21     return signal
22
23
24 def recursive_output(x, a, start_time, end_time):
25     """Direct evaluation of  $y[n] = x[n] + a*y[n-1]$  on a chosen window."""
26     y = DiscreteSignal(start_time, end_time)
27     previous = 0.0
28     for n in range(start_time, end_time + 1):
29         current = x.get_value_at_time(n) + a * previous
30         y.set_value_at_time(n, current)
31         previous = current
32     return y
33
34
35 def truncated_impulse_response(a, length):
36     """ $h_N[n] = a^n$  for  $n = 0, 1, \dots, \text{length}-1$ ."""
37     h = DiscreteSignal(0, length - 1)
38     for n in range(length):
39         h.set_value_at_time(n, a ** n)
40     return h
41
42
43 def max_abs_diff_on(first, second, start_time, end_time):
44     """Largest gap between two signals over a chosen comparison window."""
45     worst = 0.0
46     for n in range(start_time, end_time + 1):
47         worst = max(worst, abs(first.get_value_at_time(n) - second.get_value_at_time(n)))
48     return worst
49
50
51 def main():
52     a = 0.8
53     x = make_signal(0, [1, 1, 1, 1, 1, 0, 0, 0, 2, -1, 0, 0, 0, 0, 0, 0])
54     window_start, window_end = 0, 40
55
56     # ---- Task 1: the exact answer, straight from the difference equation ----
57     y_recursive = recursive_output(x, a, window_start, window_end)
58
59     # ---- Task 2: check the truncated impulse response really is  $a^n$  ----
60     unit_impulse = make_signal(0, [1.0])
61     h_measured = recursive_output(unit_impulse, a, 0, 19)
62     h_formula = truncated_impulse_response(a, 20)
63     print(f"a = {a}")
64     print("first 6 samples of the measured impulse response: ")
65     + " ".join(f"{v:.4f}" for v in h_measured.values[:6]))
66     print("first 6 samples of  $a^n$  : ")
67     + " ".join(f"{v:.4f}" for v in h_formula.values[:6]))
68     print(f"max |measured -  $a^n$ | = ")
69     + f"{max_abs_diff_on(h_measured, h_formula, 0, 19):.3e}")
70     print()
71
72     # ---- Task 3: convolution with truncated h, for several truncation lengths ----
73     print(f"{'N':>4} {'max error':>14} {'predicted tail':>16} {'ratio':>8}")
74     input_sum = float(np.sum(np.abs(x.values)))
75     for length in (2, 5, 10, 20, 40, 80):
76         h_truncated = truncated_impulse_response(a, length)
77         y_convolution = LTISystem(h_truncated).output(x)
78         error = max_abs_diff_on(y_recursive, y_convolution, window_start, window_end)
79         predicted = input_sum * abs(a) ** length / (1 - abs(a))
80         ratio = error / predicted if predicted > 0 else float("nan")
81         print(f"{length:4d} {error:14.3e} {predicted:16.3e} {ratio:8.3f}")
82     print()
83     print("The error falls off geometrically, by a factor of a per extra tap kept,")
84     print("and stays under the bound  $(\sum |x|) * |a|^N / (1 - |a|)$ ." )
85     print()
86
87     # ---- Task 4: what changes when the system is unstable ----

```

```

88     for unstable_a in (0.8, 1.0, 1.2):
89         tail = [abs(unstable_a) ** n for n in range(60)]
90         total = sum(tail)
91         stable = abs(unstable_a) < 1
92         print(f"a = {unstable_a}: sum |h[n]| over 60 taps = {total:12.4f}, "
93               f"BIBO stable = {stable}")
94     print("For |a| >= 1 the sum of |h[n]| diverges: no finite truncation ever")
95     print("approximates the system, and a bounded input can drive it unbounded.")
96
97     # Figures described below; the implementation follows.
98     # - Figure 1: 2 stacked stem subplots - x[n] on top and y_recursive[n] below,
99     #   over the window 0..40. The decaying tail after the input stops is the point
100    #   to look at.
101    # - Figure 2: overlay y_recursive with the convolution results for N = 2, 5 and 20
102    #   on one axes (use plot with markers rather than stem so four curves stay readable),
103    #   and add a legend naming each N.
104    # - Figure 3: plot max error against N on a semilogy axis (plt.semilogy), together
105    #   with the predicted bound as a dashed line. A straight line on a log axis is the
106    #   visual proof that the error decays geometrically.
107    # - xlabel "n" (or "N" on figure 3), grid on, save as p08_response.png,
108    #   p08_truncation.png, p08_error_vs_N.png.
109
110    # ---- Figures ----
111    # Figure 1: input and the exact recursive output.
112    figure, axes = plt.subplots(2, 1, figsize=(9, 5.5), sharex=True)
113    axes[0].stem(range(window_start, window_end + 1),
114                [x.get_value_at_time(n) for n in range(window_start, window_end + 1)],
115                baselfmt=" ")
116    axes[0].set_title("input $x[n]$", fontsize=10)
117    axes[1].stem(y_recursive.times(), y_recursive.values, baselfmt=" ")
118    axes[1].set_title(f"exact output of $y[n] = x[n] + \{a\}y[n-1]$ (note the decaying tail)",
119                    fontsize=10)
120    for axis in axes:
121        axis.axhline(0, color="0.7", linewidth=0.8)
122        axis.grid(True, alpha=0.3)
123    axes[-1].set_xlabel("n")
124    figure.tight_layout()
125    figure.savefig("p08_response.png", dpi=140)
126    plt.close(figure)
127
128    # Figure 2: truncation at several N. Lines, not stems, so four series stay readable.
129    figure, axis = plt.subplots(figsize=(9, 4))
130    axis.plot(y_recursive.times(), y_recursive.values, linewidth=2.0,
131            color="0.25", label="exact (recursion)")
132    for length in (2, 5, 20):
133        y_convolution = LTISystem(truncated_impulse_response(a, length)).output(x)
134        samples = [y_convolution.get_value_at_time(n)
135                  for n in range(window_start, window_end + 1)]
136        axis.plot(range(window_start, window_end + 1), samples,
137                marker=".", linewidth=1.0, label=f"$h_N$, N = {length}")
138    axis.set_title("truncating the impulse response", fontsize=10)
139    axis.set_xlabel("n")
140    axis.grid(True, alpha=0.3)
141    axis.legend(fontsize=8)
142    figure.tight_layout()
143    figure.savefig("p08_truncation.png", dpi=140)
144    plt.close(figure)
145
146    # Figure 3: error vs N on a log axis. Straight line = geometric decay.
147    lengths = list(range(1, 31))
148    errors, bounds = [], []
149    for length in lengths:
150        y_convolution = LTISystem(truncated_impulse_response(a, length)).output(x)
151        errors.append(max_abs_diff_on(y_recursive, y_convolution, window_start, window_end))
152        bounds.append(input_sum * abs(a) ** length / (1 - abs(a)))
153
154    figure, axis = plt.subplots(figsize=(8, 4))
155    axis.semilogy(lengths, np.array(errors) + 1e-18, marker="o", label="measured error")
156    axis.semilogy(lengths, bounds, linestyle="--", color="C3",
157                label="bound $\left(\sum |x| \right) |a|^N / (1 - |a|)$")
158    axis.set_title("truncation error decays geometrically", fontsize=10)
159    axis.set_xlabel("N (taps kept)")
160    axis.set_ylabel("max error")
161    axis.grid(True, alpha=0.3, which="both")

```

```

162     axis.legend(fontsize=8)
163     figure.tight_layout()
164     figure.savefig("p08_error_vs_N.png", dpi=140)
165     plt.close(figure)
166     print("\nsaved p08_response.png, p08_truncation.png, p08_error_vs_N.png")
167
168 if __name__ == "__main__":
169     main()

```

Expected console output

```

a = 0.8
first 6 samples of the measured impulse response: 1.0000 0.8000 0.6400 0.5120 0.4096 0.3277
first 6 samples of a^n                          : 1.0000 0.8000 0.6400 0.5120 0.4096 0.3277
max |measured - a^n| = 5.551e-17

```

N	max error	predicted tail	ratio
2	2.161e+00	2.560e+01	0.084
5	1.107e+00	1.311e+01	0.084
10	3.626e-01	4.295e+00	0.084
20	3.893e-02	4.612e-01	0.084
40	1.329e-04	5.317e-03	0.025
80	4.441e-16	7.067e-07	0.000

The error falls off geometrically, by a factor of a per extra tap kept, and stays under the bound $(\sum |x|) * |a|^N / (1 - |a|)$.

```

a = 0.8: sum |h[n]| over 60 taps =      5.0000, BIBO stable = True
a = 1.0: sum |h[n]| over 60 taps =     60.0000, BIBO stable = False
a = 1.2: sum |h[n]| over 60 taps =  281732.5718, BIBO stable = False
For |a| >= 1 the sum of |h[n]| diverges: no finite truncation ever
approximates the system, and a bounded input can drive it unbounded.

```

```

saved p08_response.png, p08_truncation.png, p08_error_vs_N.png

```

Problem 9. Overlap-add block convolution

Total Marks: 10

Suggested Time: 25 minutes

New

Long recordings are filtered in blocks: cut the input into consecutive pieces,

$$x[n] = \sum_b x_b[n],$$

convolve each piece with h , and add the partial outputs. Linearity guarantees the result is exact. The subtlety is that each partial output is $\text{len}(h) - 1$ samples *longer* than its block, so neighbouring partial outputs overlap — hence the name.

`long_signal.txt` holds 200 samples on $[0, 199]$. Use $h[n] = \{0.4, 0.3, 0.2, 0.1\}$ on $[0, 3]$ and a block length of 16.

Files provided: `long_signal.txt` (copy them next to your script before running).

Tasks

1. Read the file and convolve it directly. This is the reference answer.
2. Implement `split_into_blocks(x, L)`. Each block must keep the *absolute* time indices it had inside x ; the last block may be shorter.
3. For the first three blocks, print the input range and the output range each one produces, and confirm the growth is $\text{len}(h) - 1$ samples.
4. Implement `overlap_add(x, h, L)` and verify it reproduces the direct convolution exactly.
5. Repeat for block lengths 1, 4, 16, 37, 200, 500 and confirm the answer never depends on the block length.

Marks Distribution

Component	Marks
Block splitting preserving absolute time indices	3
Per-block convolution and accumulation	3
Correct overall output range	1
Verification against direct convolution	2
Block-length independence demonstrated	1
Total	10

Template

```

1  """
2  Problem 09 - Overlap-add block convolution (TEMPLATE)
3
4   $x[n] = \sum_b x_b[n]$ ; convolve each block separately and add the partial outputs.
5  Each partial output is  $\text{len}(h)-1$  samples longer than its block, which is exactly
6  why neighbouring partial outputs overlap.
7
8  File provided: long_signal.txt
9  """
10
11 import numpy as np
12 from signal_lti import DiscreteSignal, LTISystem

```

```

13
14
15 def read_signal_from_file(path):
16     # TODO
17     raise NotImplementedError
18
19
20 def make_signal(start_time, values):
21     # TODO
22     raise NotImplementedError
23
24
25 def max_abs_diff(first, second):
26     # TODO
27     raise NotImplementedError
28
29
30 def split_into_blocks(x, block_length):
31     """
32     Cut x into consecutive blocks of block_length samples. Each block must KEEP
33     the absolute time indices it had inside x - that is what makes the final
34     addition trivial. The last block may be shorter.
35     """
36     # TODO
37     raise NotImplementedError
38
39
40 def overlap_add(x, h, block_length):
41     """Full convolution assembled from per-block convolutions."""
42     # TODO
43     raise NotImplementedError
44
45
46 def main():
47     h = make_signal(0, [0.4, 0.3, 0.2, 0.1])
48     block_length = 16
49
50     # TODO 1: read long_signal.txt and convolve it directly - this is the reference.
51     # TODO 2: split into blocks and print, for the first three blocks, the input
52     #         range and the output range each one produces.
53     # TODO 3: assemble the overlap-add result and verify it equals the reference.
54     # TODO 4: repeat for block lengths 1, 4, 16, 37, 200 and 500 and confirm the
55     #         answer never depends on the block length.
56
57     # PLOTTING (write this part yourself):
58     # - Figure 1: plot the long input x[n] as a line with small markers (200 samples
59     #   is too many for a readable stem plot). Draw vertical dashed lines at every
60     #   block boundary with axvline so the segmentation is visible.
61     # - Figure 2: plot the partial output of the first three blocks on the same axes,
62     #   each over its own times() array. The 3-sample overlap between neighbouring
63     #   partial outputs is the thing to look for; add a legend naming the blocks.
64     # - Figure 3: two rows - the direct output on top, the overlap-add output below -
65     #   then a third row with their difference. Use semilogy on the difference row if
66     #   you want to see the floating-point floor.
67     # - xlabel "n", grid on, save as p09_blocks.png, p09_partials.png, p09_verify.png.
68
69
70 if __name__ == "__main__":
71     main()

```

Problem 9 — Solution

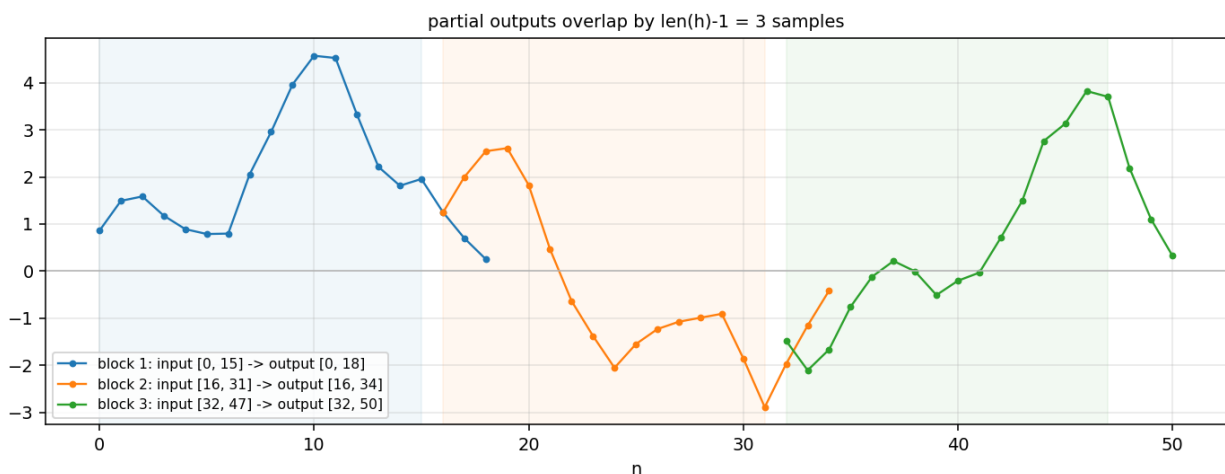
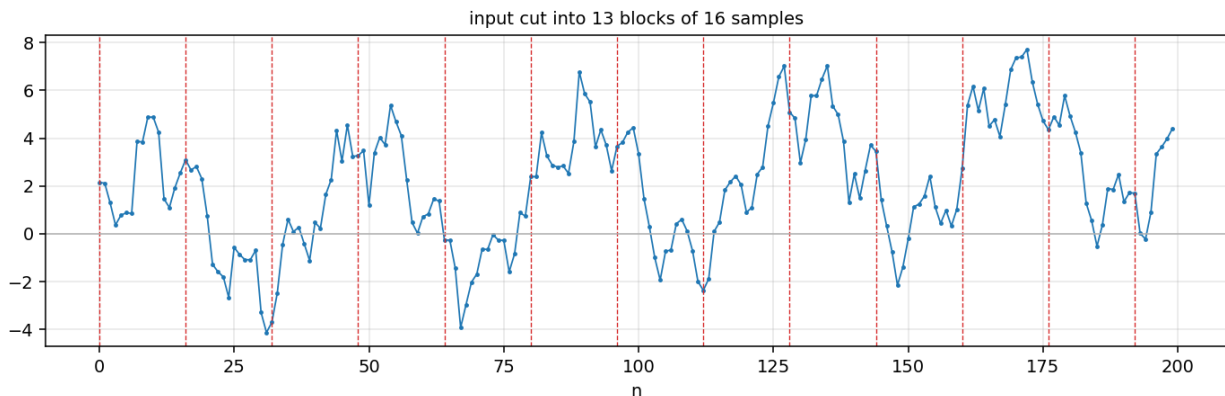
Solution — approach and discussion

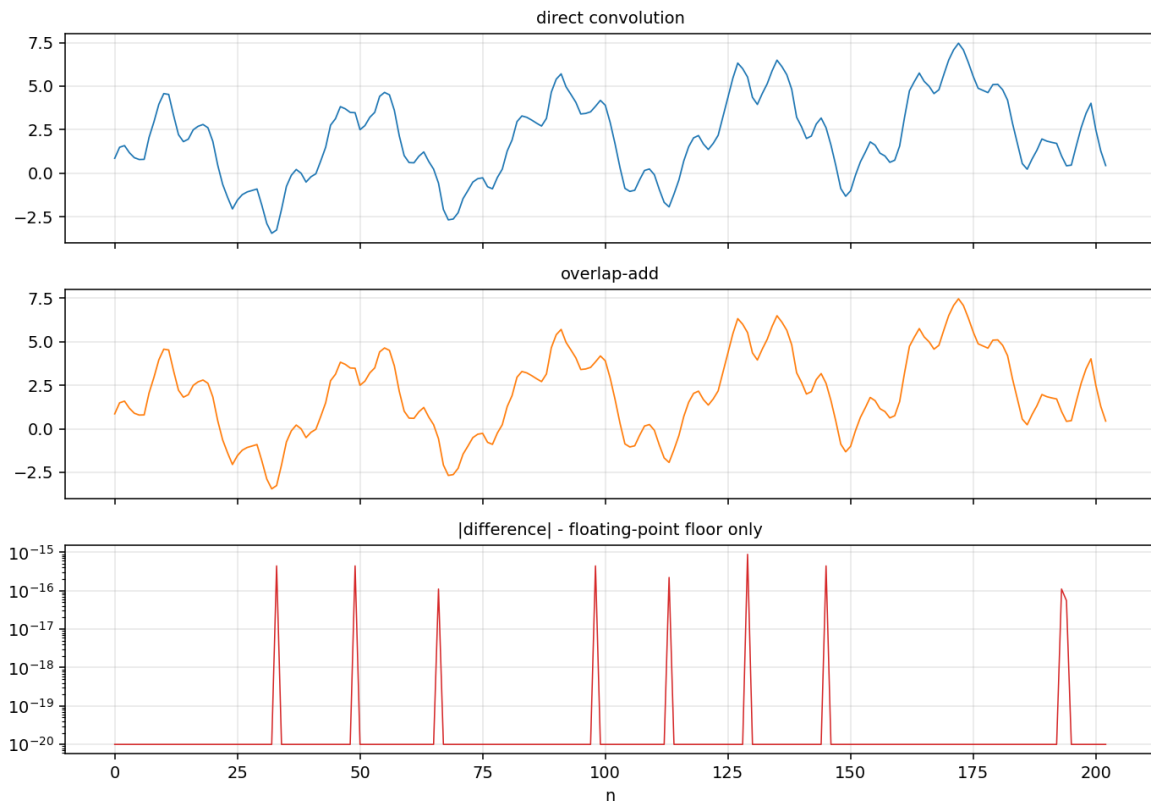
Keeping the absolute time indices is what makes this problem easy instead of fiddly. Block 2 covers $[16, 31]$ and its partial output covers $[16, 34]$ — your offline's `add` then places every partial output at the right absolute position with no manual offset arithmetic at all. The textbook version of overlap-add, where blocks are extracted to index 0 and shifted back afterwards, is exactly the same algorithm with the bookkeeping done by hand; here the `DiscreteSignal` range does it for you.

The overlap is $\text{len}(h) - 1 = 3$ samples between consecutive partial outputs. Those 3 samples are precisely the part of block b 's response that spills into block $b + 1$'s window, and adding them back is what makes the reconstruction exact rather than merely approximate. Dropping them — which is what happens if you truncate each partial output to its block length — produces a visible click at every block boundary.

The block-length sweep is the real test. $L = 1$ reduces the method to your offline's `output_by_superposition` (one impulse at a time), and $L \geq 200$ reduces it to a single direct convolution. Both extremes and everything between give the same answer to round-off, which is the practical statement of linearity.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 09 - Overlap-add block convolution (SOLUTION)
3
4  A long input is cut into non-overlapping blocks
5       $x[n] = \sum_b x_b[n]$ ,
6  each block is convolved with  $h$  separately, and the partial outputs are added
7  back. Because convolution is linear and each block keeps its own absolute time
8  indices, the sum of the partial outputs is exactly the full convolution. Each
9  partial output is  $\text{len}(h)-1$  samples longer than its block, which is why the
10 blocks overlap when they are added.
11 """
12
13 import numpy as np
14 import matplotlib.pyplot as plt
15 from signal_lti import DiscreteSignal, LTISystem
16
17
18 def read_signal_from_file(path):
19     with open(path, "r", encoding="utf-8") as handle:
20         start_time, end_time = map(int, handle.readline().split())
21         values = list(map(float, handle.readline().split()))
22         if len(values) != end_time - start_time + 1:
23             raise ValueError(f"{path}: sample count does not match the declared range")
24         signal = DiscreteSignal(start_time, end_time)
25         for offset, value in enumerate(values):
26             signal.set_value_at_time(start_time + offset, value)
27         return signal
28
29
30 def make_signal(start_time, values):
31     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
32     for offset, value in enumerate(values):
33         signal.set_value_at_time(start_time + offset, value)
34     return signal
35
36

```

```

37 def max_abs_diff(first, second):
38     difference = first.add(second.multiply(-1))
39     return float(np.max(np.abs(difference.values)))
40
41
42 def split_into_blocks(x, block_length):
43     """
44     Cut x into consecutive blocks of block_length samples. Every block keeps the
45     absolute time indices it had inside x, so no bookkeeping is needed later.
46     """
47     blocks = []
48     start_time = x.start_time
49     while start_time <= x.end_time:
50         end_time = min(start_time + block_length - 1, x.end_time)
51         block = DiscreteSignal(start_time, end_time)
52         for n in range(start_time, end_time + 1):
53             block.set_value_at_time(n, x.get_value_at_time(n))
54         blocks.append(block)
55         start_time = end_time + 1
56     return blocks
57
58
59 def overlap_add(x, h, block_length):
60     """Full convolution assembled from per-block convolutions."""
61     system = LTISystem(h)
62     result = DiscreteSignal(x.start_time + h.start_time, x.end_time + h.end_time)
63     for block in split_into_blocks(x, block_length):
64         result = result.add(system.output(block))
65     return result
66
67
68 def main():
69     x = read_signal_from_file("long_signal.txt")
70     h = make_signal(0, [0.4, 0.3, 0.2, 0.1])
71     block_length = 16
72
73     # ---- Task 1: reference answer, one convolution over the whole input ----
74     y_direct = LTISystem(h).output(x)
75
76     # ---- Task 2: block processing ----
77     blocks = split_into_blocks(x, block_length)
78     y_blocks = overlap_add(x, h, block_length)
79
80     print(f"input : range [{x.start_time}, {x.end_time}], {len(x)} samples")
81     print(f"h       : {len(h)} taps over [{h.start_time}, {h.end_time}]")
82     print(f"blocks: {len(blocks)} blocks of at most {block_length} samples")
83     print()
84     print("first three blocks and the range each one produces:")
85     for index, block in enumerate(blocks[:3], start=1):
86         partial = LTISystem(h).output(block)
87         print(f"  block {index}: input [{block.start_time:3d}, {block.end_time:3d}] "
88               f"  f-> output [{partial.start_time:3d}, {partial.end_time:3d}] "
89               f"  f({len(partial)} samples, {len(h) - 1} more than the block)")
90     print()
91
92     # ---- Task 3: verification ----
93     difference = max_abs_diff(y_direct, y_blocks)
94     print(f"direct output      : range [{y_direct.start_time}, {y_direct.end_time}]")
95     print(f"overlap-add output : range [{y_blocks.start_time}, {y_blocks.end_time}]")
96     print(f"max |y_direct - y_overlap_add| = {difference:.3e}")
97     print("Identical:", difference < 1e-12)
98     print()
99
100    # ---- Task 4: the answer must not depend on the block length ----
101    print(f"{'block length':>13} {'max difference':>16}")
102    for length in (1, 4, 16, 37, 200, 500):
103        error = max_abs_diff(y_direct, overlap_add(x, h, length))
104        print(f"{length:13d} {error:16.3e}")
105    print()
106    print("Consecutive blocks overlap in exactly len(h)-1 = 3 output samples;")
107    print("adding those overlaps back is what makes the reconstruction exact.")
108
109    # Figures described below; the implementation follows.
110    # - Figure 1: plot the long input x[n] as a line with small markers (200 samples

```

```

111 # is too many for a readable stem plot). Draw vertical dashed lines at every
112 # block boundary with axvline so the segmentation is visible.
113 # - Figure 2: plot the partial output of the first three blocks on the same axes,
114 # each over its own times() array. The 3-sample overlap between neighbouring
115 # partial outputs is the thing to look for; add a legend naming the blocks.
116 # - Figure 3: two rows - y_direct on top, y_blocks below - then a third row with
117 # their difference. Use semilogy on the difference row if you want to see the
118 # floating-point floor.
119 # - xlabel "n", grid on, save as p09_blocks.png, p09_partials.png, p09_verify.png.
120
121 # ---- Figures ----
122 # Figure 1: the input with the block boundaries drawn on.
123 figure, axis = plt.subplots(figsize=(10, 3.4))
124 axis.plot(x.times(), x.values, linewidth=0.9, marker=".", markersize=3)
125 for block in blocks:
126     axis.axvline(block.start_time, color="C3", linestyle="--", linewidth=0.7)
127 axis.axhline(0, color="0.7", linewidth=0.8)
128 axis.set_title(f"input cut into {len(blocks)} blocks of {block_length} samples", fontsize=10)
129 axis.set_xlabel("n")
130 axis.grid(True, alpha=0.3)
131 figure.tight_layout()
132 figure.savefig("p09_blocks.png", dpi=140)
133 plt.close.figure)
134
135 # Figure 2: the first three partial outputs. Each is plotted over its own
136 # times() array, which is what makes the 3-sample overlap visible.
137 figure, axis = plt.subplots(figsize=(10, 4))
138 for index, block in enumerate(blocks[:3], start=1):
139     partial = LTISystem(h).output(block)
140     axis.plot(partial.times(), partial.values, marker="o", markersize=3,
141              linewidth=1.2, label=f"block {index}: input [{block.start_time}, {block.end_time}]"
142              f" -> output [{partial.start_time}, {partial.end_time}]")
143     axis.axvspan(block.start_time, block.end_time, alpha=0.06,
144                 color=f"C{index - 1}")
145 axis.axhline(0, color="0.7", linewidth=0.8)
146 axis.set_title(f"partial outputs overlap by len(h)-1 = {len(h) - 1} samples", fontsize=10)
147 axis.set_xlabel("n")
148 axis.grid(True, alpha=0.3)
149 axis.legend(fontsize=8)
150 figure.tight_layout()
151 figure.savefig("p09_partials.png", dpi=140)
152 plt.close.figure)
153
154 # Figure 3: the two results and their difference.
155 error_signal = y_direct.add(y_blocks.multiply(-1))
156 figure, axes = plt.subplots(3, 1, figsize=(10, 7), sharex=True)
157 axes[0].plot(y_direct.times(), y_direct.values, linewidth=0.9)
158 axes[0].set_title("direct convolution", fontsize=10)
159 axes[1].plot(y_blocks.times(), y_blocks.values, linewidth=0.9, color="C1")
160 axes[1].set_title("overlap-add", fontsize=10)
161 axes[2].semilogy(error_signal.times(), np.abs(error_signal.values) + 1e-20,
162                 linewidth=0.8, color="C3")
163 axes[2].set_title("|difference| - floating-point floor only", fontsize=10)
164 for axis in axes:
165     axis.grid(True, alpha=0.3)
166 axes[-1].set_xlabel("n")
167 figure.tight_layout()
168 figure.savefig("p09_verify.png", dpi=140)
169 plt.close.figure)
170 print("\nsaved p09_blocks.png, p09_partials.png, p09_verify.png")
171
172 if __name__ == "__main__":
173     main()

```

Expected console output

```

input : range [0, 199], 200 samples
h      : 4 taps over [0, 3]
blocks: 13 blocks of at most 16 samples

```

```
first three blocks and the range each one produces:
  block 1: input [ 0, 15] -> output [ 0, 18] (19 samples, 3 more than the block)
  block 2: input [16, 31] -> output [16, 34] (19 samples, 3 more than the block)
  block 3: input [32, 47] -> output [32, 50] (19 samples, 3 more than the block)

direct output      : range [0, 202]
overlap-add output : range [0, 202]
max |y_direct - y_overlap_add| = 8.882e-16
Identical: True

  block length  max difference
           1      0.000e+00
           4      8.882e-16
          16      8.882e-16
          37      2.220e-16
         200      0.000e+00
         500      0.000e+00

Consecutive blocks overlap in exactly len(h)-1 = 3 output samples;
adding those overlaps back is what makes the reconstruction exact.

saved p09_blocks.png, p09_partials.png, p09_verify.png
```

Problem 10. Circular convolution versus linear convolution

Total Marks: 10

Suggested Time: 30 minutes

New

Circular convolution of period N treats both sequences as periodic:

$$y_c[n] = \sum_{m=0}^{N-1} x[m] h[(n-m) \bmod N], \quad n = 0, \dots, N-1.$$

It agrees with ordinary (linear) convolution only when N is large enough to hold the whole result, that is $N \geq N_x + N_h - 1$. For smaller N the tail wraps around and adds on top of the beginning — time-domain aliasing. This is the property that makes FFT-based filtering work, and the reason zero padding is not optional. Use $x[n] = \{1, 2, 3, 4, 0, -1\}$ and $h[n] = \{1, -1, 2, 0.5\}$, both starting at $n = 0$.

Tasks

1. Compute the linear convolution and state its length as $N_x + N_h - 1$.
2. Implement `zero_pad_to(signal, N)` and `circular_convolve(x, h, N)` from the definition, with your own modulo indexing.
3. Show the circular result equals the linear one at $N = N_x + N_h - 1$.
4. Implement `alias_linear_result(y, N)` which folds the linear result by adding $y[n]$ into index $n \bmod N$. For $N = 6, 7, 8$ print it beside the circular result and confirm they match.
5. Tabulate $\max_n |y_{\text{lin}} - y_c|$ for N from N_x to $N_x + N_h + 2$ and identify the smallest safe N .

Marks Distribution

Component	Marks
<code>circular_convolve</code> from the definition with correct modulo indexing	4
Equality demonstrated at sufficient N	2
Aliasing predicted by folding and confirmed	3
Smallest safe N identified	1
Total	10

Template

```

1  """
2  Problem 10 - Circular convolution versus linear convolution (TEMPLATE)
3
4      y_c[n] = sum_{m=0}^{N-1} x[m] * h[(n - m) mod N],    n = 0 .. N-1
5  It matches the linear convolution only when N >= Nx + Nh - 1.
6  """
7
8  import numpy as np
9  from signal_lti import DiscreteSignal, LTISystem
10
11
12  def make_signal(start_time, values):
13      # TODO
14      raise NotImplementedError
15

```

```

16
17 def zero_pad_to(signal, length):
18     """Same samples, stored on [0, length-1]. Reject signals not starting at 0."""
19     # TODO
20     raise NotImplementedError
21
22
23 def circular_convolve(x, h, period):
24     """N-point circular convolution, written out from the definition."""
25     # TODO
26     raise NotImplementedError
27
28
29 def alias_linear_result(y_linear, period):
30     """
31     Fold the linear result modulo 'period': add y_linear[n] into index n % period.
32     This is what wrap-around predicts, and it should match circular_convolve exactly.
33     """
34     # TODO
35     raise NotImplementedError
36
37
38 def max_abs_diff(first, second):
39     # TODO
40     raise NotImplementedError
41
42
43 def main():
44     x = make_signal(0, [1, 2, 3, 4, 0, -1])
45     h = make_signal(0, [1, -1, 2, 0.5])
46
47     # TODO 1: compute the linear convolution and state its length as  $N_x + N_h - 1$ .
48     # TODO 2: circular convolution at  $N = N_x + N_h - 1$ ; confirm it equals the linear one.
49     # TODO 3: for  $N = 6, 7, 8$  print the circular result next to the folded linear
50     #           result and confirm they match. Report how many samples wrapped.
51     # TODO 4: tabulate  $\max|\text{linear} - \text{circular}|$  for  $N$  from  $N_x$  up to  $N_x + N_h + 2$  and identify
52     #           the smallest safe  $N$ .
53
54     # PLOTTING (write this part yourself):
55     # - Figure 1: 3 stacked stem subplots -  $x[n]$ ,  $h[n]$ ,  $y_{\text{linear}}[n]$ .
56     # - Figure 2: for  $N = 6$  put the circular result and the folded linear result on the
57     #           same axes with two marker styles, so the aliased samples at  $n = 0, 1, 2$  line up.
58     #           Shade or mark the aliased region so you can see which samples were corrupted.
59     # - Figure 3: plot the max error against  $N$  as a stem or semilogy plot, and draw a
60     #           vertical line at  $N = N_x + N_h - 1$  to mark where the error drops to zero.
61     # - xlabel "n" (or "N" on figure 3), grid on, legends where two series share an axes,
62     #           save as p10_linear.png, p10_aliasing.png, p10_error_vs_N.png.
63
64
65 if __name__ == "__main__":
66     main()

```

Problem 10 — Solution

Solution — approach and discussion

With $N_x = 6$ and $N_h = 4$ the linear result has 9 samples, $\{1, 1, 3, 5.5, 3, 8.5, 3, -2, -0.5\}$, and at $N = 9$ the circular result is identical.

The aliasing table is the point of the problem. At $N = 6$ the last three linear samples $(3, -2, -0.5)$ have nowhere to go, so they wrap onto indices 0, 1, 2:

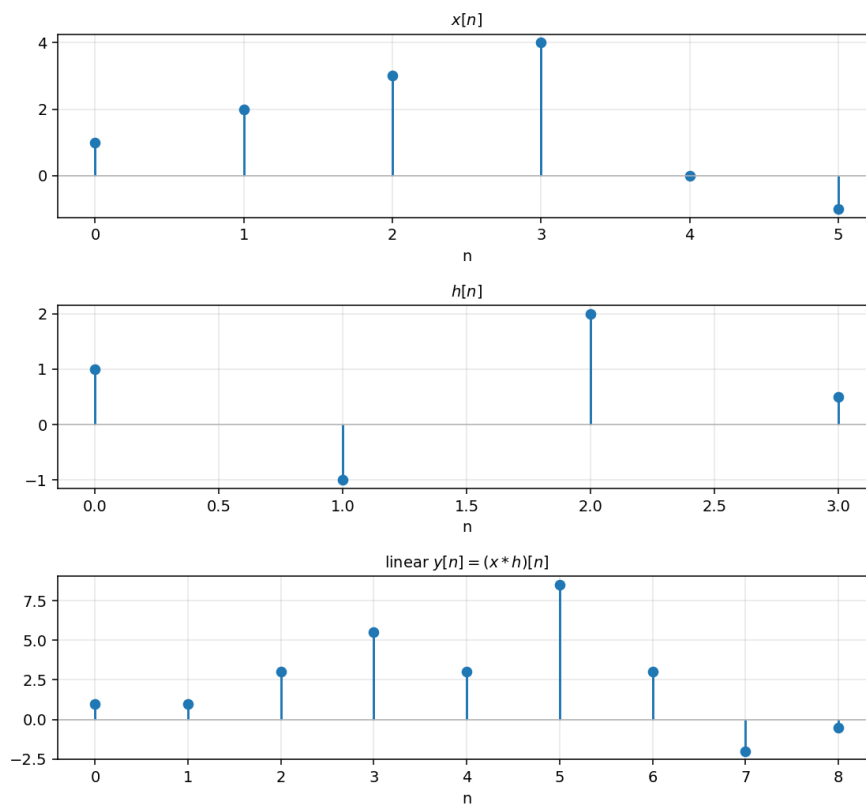
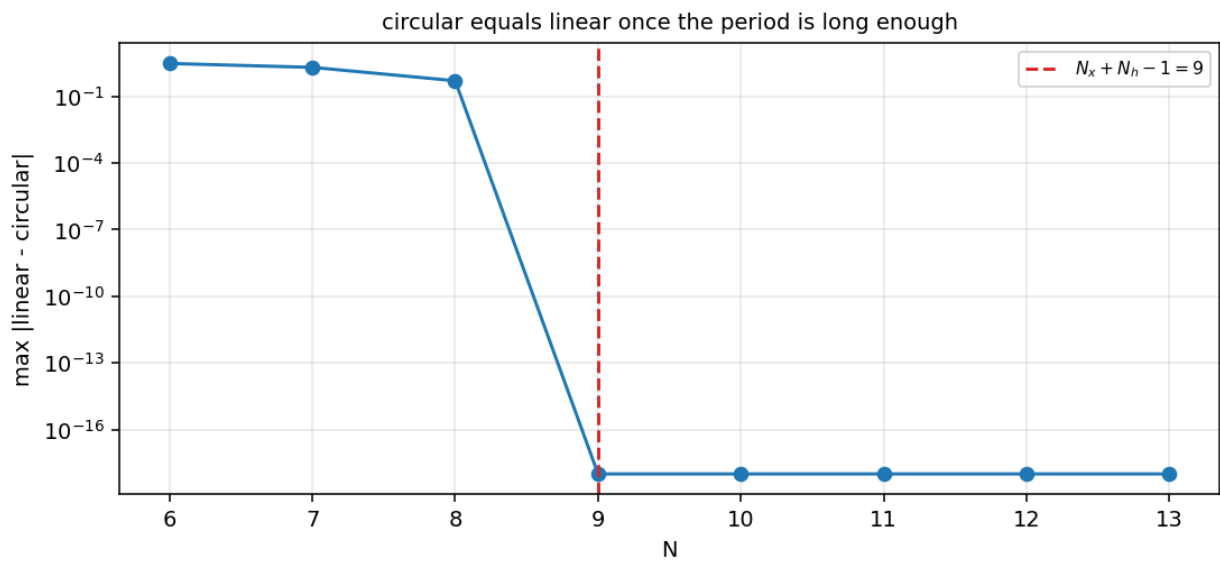
$$y_c[0] = 1 + 3 = 4, \quad y_c[1] = 1 + (-2) = -1, \quad y_c[2] = 3 + (-0.5) = 2.5,$$

which is exactly what `alias_linear_result` predicts, and exactly what the circular convolution produces. The number of corrupted samples is $\max(0, N_x + N_h - 1 - N)$, so $N = 6, 7, 8$ corrupt 3, 2, 1 samples and $N = 9$ corrupts none.

Two implementation notes. The modulo must be applied to $(n - m)$ *before* indexing, and Python's `%` already returns a non-negative result for a positive modulus, so $(n - m) \bmod N$ needs no special handling for negative differences — a genuine convenience here compared with C. And insisting that both signals start at $n = 0$ is not pedantry: circular convolution is only defined relative to a chosen origin, so a signal on $[-2, 3]$ has no unambiguous circular interpretation until you say where index 0 is.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 10 - Circular convolution versus linear convolution (SOLUTION)
3
4  Circular convolution of period N:
5       $y_c[n] = \sum_{m=0}^{N-1} x[m] * h[(n - m) \bmod N]$ ,  $n = 0 \dots N-1$ .
6  It equals the linear convolution only when N is at least  $N_x + N_h - 1$ . For a
7  smaller N the tail of the linear result wraps around and adds on top of the
8  beginning: this is time-domain aliasing.
9  """
10

```

```

11 import numpy as np
12 import matplotlib.pyplot as plt
13 from signal_lti import DiscreteSignal, LTISystem
14
15
16 def make_signal(start_time, values):
17     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
18     for offset, value in enumerate(values):
19         signal.set_value_at_time(start_time + offset, value)
20     return signal
21
22
23 def zero_pad_to(signal, length):
24     """Same samples, stored on [0, length-1]. Requires a signal starting at 0."""
25     if signal.start_time != 0:
26         raise ValueError("circular convolution assumes signals indexed from 0")
27     if length < len(signal):
28         raise ValueError("cannot pad to a length shorter than the signal")
29     padded = DiscreteSignal(0, length - 1)
30     for n in range(len(signal)):
31         padded.set_value_at_time(n, signal.get_value_at_time(n))
32     return padded
33
34
35 def circular_convolve(x, h, period):
36     """N-point circular convolution, written out from the definition."""
37     x_padded = zero_pad_to(x, period)
38     h_padded = zero_pad_to(h, period)
39
40     result = DiscreteSignal(0, period - 1)
41     for n in range(period):
42         total = 0.0
43         for m in range(period):
44             total += (x_padded.get_value_at_time(m)
45                      * h_padded.get_value_at_time((n - m) % period))
46         result.set_value_at_time(n, total)
47     return result
48
49
50 def alias_linear_result(y_linear, period):
51     """Fold the linear result modulo 'period': what wrap-around predicts."""
52     folded = DiscreteSignal(0, period - 1)
53     for n in range(y_linear.start_time, y_linear.end_time + 1):
54         index = n % period
55         folded.set_value_at_time(index,
56                                  folded.get_value_at_time(index)
57                                  + y_linear.get_value_at_time(n))
58     return folded
59
60
61 def max_abs_diff(first, second):
62     difference = first.add(second.multiply(-1))
63     return float(np.max(np.abs(difference.values)))
64
65
66 def describe(name, signal):
67     values = " ".join(f"{v:+7.3f}" for v in signal.values)
68     return f"{name}: [{signal.start_time}..{signal.end_time}] {values}"
69
70
71 def main():
72     x = make_signal(0, [1, 2, 3, 4, 0, -1])
73     h = make_signal(0, [1, -1, 2, 0.5])
74
75     minimum_period = len(x) + len(h) - 1
76     y_linear = LTISystem(h).output(x)
77
78     print(describe("x", x))
79     print(describe("h", h))
80     print(describe("y_linear", y_linear))
81     print(f"linear length = {len(x)} + {len(h)} - 1 = {minimum_period}")
82     print()
83
84     # ---- Task 1: N large enough - circular equals linear ----

```

```

85     y_circular_full = circular_convolve(x, h, minimum_period)
86     error_full = max_abs_diff(y_linear, y_circular_full)
87     print(describe(f"circular N={minimum_period}", y_circular_full))
88     print(f"max |linear - circular| at N = {minimum_period}: {error_full:.3e}")
89     print()
90
91     # ---- Task 2: N too small - aliasing appears ----
92     for period in (6, 7, 8):
93         y_circular = circular_convolve(x, h, period)
94         y_folded = alias_linear_result(y_linear, period)
95         print(describe(f"circular N={period}", y_circular))
96         print(describe(f"folded N={period}", y_folded))
97         print(f"    circular matches the folded linear result: "
98               f"{max_abs_diff(y_circular, y_folded) < 1e-12}, "
99               f"wrapped samples = {max(0, minimum_period - period)}")
100        print()
101
102    # ---- Task 3: the smallest safe period ----
103    print(f"{'N':>4} {'max |linear - circular|':>26}")
104    for period in range(len(x), minimum_period + 4):
105        y_circular = circular_convolve(x, h, period)
106        error = max_abs_diff(y_linear, y_circular)
107        print(f"{period:4d} {error:26.3e}")
108    print()
109    print(f"The error vanishes from N = {minimum_period} onwards, which is exactly")
110    print("the point where the linear result fits inside one period.")
111
112    # Figures described below; the implementation follows.
113    # - Figure 1: 3 stacked stem subplots - x[n], h[n], y_linear[n].
114    # - Figure 2: for N = 6 put the circular result and the folded linear result on the
115    #   same axes with two marker styles, so the aliased samples at n = 0, 1, 2 line up.
116    #   Shade or mark the aliased region so you can see which samples were corrupted.
117    # - Figure 3: plot the max error against N as a stem or semilogy plot, and draw a
118    #   vertical line at N = Nx + Nh - 1 to mark where the error drops to zero.
119    # - xlabel "n" (or "N" on figure 3), grid on, legends where two series share an axes,
120    #   save as p10_linear.png, p10_aliasing.png, p10_error_vs_N.png.
121
122    # ---- Figures ----
123    # Figure 1: the ingredients and the linear result.
124    figure, axes = plt.subplots(3, 1, figsize=(8, 7.5))
125    for axis, (label, signal) in zip(axes, [("$x[n]$", x), ("h[n]$", h),
126                                           ("linear $y[n] = (x*h)[n]$", y_linear)]):
127        axis.stem(signal.times(), signal.values, basefmt=" ")
128        axis.axhline(0, color="0.7", linewidth=0.8)
129        axis.set_title(label, fontsize=10)
130        axis.set_xlabel("n")
131        axis.grid(True, alpha=0.3)
132    figure.tight_layout()
133    figure.savefig("p10_linear.png", dpi=140)
134    plt.close(figure)
135
136    # Figure 2: aliasing at N = 6. The wrapped samples land on n = 0, 1, 2.
137    period = 6
138    y_circular = circular_convolve(x, h, period)
139    y_folded = alias_linear_result(y_linear, period)
140    wrapped = minimum_period - period
141
142    figure, axis = plt.subplots(figsize=(8, 4))
143    axis.stem(y_circular.times(), y_circular.values, linefmt="C0-",
144             markerfmt="C0o", basefmt=" ", label=f"circular, N = {period}")
145    markers, _, _ = axis.stem(y_folded.times(), y_folded.values, linefmt="C1--",
146                             markerfmt="C1s", basefmt=" ", label="linear result folded mod N")
147    markers.set_markerfacecolor("none")
148    axis.axvspan(-0.5, wrapped - 0.5, alpha=0.12, color="C3",
149               label=f"{wrapped} corrupted samples")
150    axis.axhline(0, color="0.7", linewidth=0.8)
151    axis.set_title("time-domain aliasing when the period is too short", fontsize=10)
152    axis.set_xlabel("n")
153    axis.grid(True, alpha=0.3)
154    axis.legend(fontsize=8)
155    figure.tight_layout()
156    figure.savefig("p10_aliasing.png", dpi=140)
157    plt.close(figure)
158

```

```

159     # Figure 3: error against N, with the safe threshold marked.
160     periods = list(range(len(x), minimum_period + 5))
161     errors = [max_abs_diff(y_linear, circular_convolve(x, h, p)) for p in periods]
162
163     figure, axis = plt.subplots(figsize=(8, 3.8))
164     axis.semilogy(periods, np.array(errors) + 1e-18, marker="o")
165     axis.axvline(minimum_period, color="C3", linestyle="--",
166                 label=f"$N_x+N_h-1 = \{minimum\_period\}$")
167     axis.set_title("circular equals linear once the period is long enough", fontsize=10)
168     axis.set_xlabel("N")
169     axis.set_ylabel("max |linear - circular|")
170     axis.grid(True, alpha=0.3, which="both")
171     axis.legend(fontsize=8)
172     figure.tight_layout()
173     figure.savefig("p10_error_vs_N.png", dpi=140)
174     plt.close(figure)
175     print("\nsaved p10_linear.png, p10_aliasing.png, p10_error_vs_N.png")
176
177 if __name__ == "__main__":
178     main()

```

Expected console output

```

x      : [0..5] +1.000 +2.000 +3.000 +4.000 +0.000 -1.000
h      : [0..3] +1.000 -1.000 +2.000 +0.500
y_linear: [0..8] +1.000 +1.000 +3.000 +5.500 +3.000 +8.500 +3.000 -2.000 -0.500
linear length = 6 + 4 - 1 = 9

circular N=9: [0..8] +1.000 +1.000 +3.000 +5.500 +3.000 +8.500 +3.000 -2.000 -0.500
max |linear - circular| at N = 9: 0.000e+00

circular N=6: [0..5] +4.000 -1.000 +2.500 +5.500 +3.000 +8.500
folded  N=6: [0..5] +4.000 -1.000 +2.500 +5.500 +3.000 +8.500
    circular matches the folded linear result: True, wrapped samples = 3

circular N=7: [0..6] -1.000 +0.500 +3.000 +5.500 +3.000 +8.500 +3.000
folded  N=7: [0..6] -1.000 +0.500 +3.000 +5.500 +3.000 +8.500 +3.000
    circular matches the folded linear result: True, wrapped samples = 2

circular N=8: [0..7] +0.500 +1.000 +3.000 +5.500 +3.000 +8.500 +3.000 -2.000
folded  N=8: [0..7] +0.500 +1.000 +3.000 +5.500 +3.000 +8.500 +3.000 -2.000
    circular matches the folded linear result: True, wrapped samples = 1

    N      max |linear - circular|
    6      3.000e+00
    7      2.000e+00
    8      5.000e-01
    9      0.000e+00
    10     0.000e+00
    11     0.000e+00
    12     0.000e+00

The error vanishes from N = 9 onwards, which is exactly
the point where the linear result fits inside one period.

saved p10_linear.png, p10_aliasing.png, p10_error_vs_N.png

```

Problem 11. Upsampling by zero stuffing and interpolation

Total Marks: 10

Suggested Time: 30 minutes

New

Raising the sample rate by an integer factor L takes two steps, and only the second one is a filter:

$$\text{expand: } x_{\uparrow}[n] = \begin{cases} x[n/L], & L \mid n \\ 0, & \text{otherwise} \end{cases} \quad \text{interpolate: } y = x_{\uparrow} * h.$$

With the triangular kernel $h[m] = 1 - |m|/L$ for $|m| < L$, the result is exactly piecewise-linear interpolation. The two conditions $h[0] = 1$ and $h[\pm L] = 0$ are what guarantee the original samples survive untouched: $y[Ln] = x[n]$. Use $L = 3$ and $x[n] = \{0, 1, 3, 2, -1, 0\}$ on $[-1, 4]$.

Tasks

1. Implement `upsample(x, L)` inserting $L-1$ zeros between samples. The range scales too: $[L n_{\min}, L n_{\max}]$.
2. Implement `downsample(x, M)` returning $x[Mn]$, working out the new range with a ceiling and a floor.
3. Build the triangular kernel and interpolate. Report the resulting range and values.
4. Verify $y[Ln] = x[n]$ for every original sample and report the worst mismatch.
5. Downsample the interpolated signal by L and confirm x comes back exactly.
6. Repeat with the sample-and-hold kernel $h[m] = 1$ for $0 \leq m < L$, and explain which one shifts the output and why.

Marks Distribution

Component	Marks
<code>upsample</code> with correctly scaled range	3
<code>downsample</code> with correct ceiling/floor range	2
Triangular kernel and interpolation	2
Sample-preservation check	2
Comparison with sample-and-hold	1
Total	10

Template

```

1  """
2  Problem 11 - Upsampling by zero stuffing and interpolation (TEMPLATE)
3
4  1. expand:      x_up[n] = x[n/L] when L divides n, else 0
5  2. interpolate: y = x_up * h
6  With h[m] = 1 - |m|/L for |m| < L the result is piecewise-linear interpolation,
7  and h[0] = 1, h[+-L] = 0 guarantee y[L*n] = x[n].
8  """
9
10 import numpy as np
11 from signal_lti import DiscreteSignal, LTISystem
12
13
14 def make_signal(start_time, values):
15     # TODO

```

```

16     raise NotImplementedError
17
18
19 def upsample(x, factor):
20     """Insert factor-1 zeros between neighbouring samples. Scale the range too."""
21     # TODO
22     raise NotImplementedError
23
24
25 def downsample(x, factor):
26     """ $y[n] = x[\text{factor} * n]$ . Work out the new range with ceil / floor."""
27     # TODO
28     raise NotImplementedError
29
30
31 def triangular_kernel(factor):
32     """ $h[m] = 1 - |m|/\text{factor}$  for  $|m| \leq \text{factor}-1$  (centred, non-causal)."""
33     # TODO
34     raise NotImplementedError
35
36
37 def hold_kernel(factor):
38     """ $h[m] = 1$  for  $0 \leq m < \text{factor}$  (causal staircase)."""
39     # TODO
40     raise NotImplementedError
41
42
43 def max_abs_diff(first, second):
44     # TODO
45     raise NotImplementedError
46
47
48 def main():
49     factor = 3
50     x = make_signal(-1, [0, 1, 3, 2, -1, 0])
51
52     # TODO 1: build  $x_{\text{up}}$  and report how the length and range changed.
53     # TODO 2: interpolate with the triangular kernel.
54     # TODO 3: check  $y[L * n] == x[n]$  for every original sample and report the worst mismatch.
55     # TODO 4: downsample the interpolated signal by L and confirm  $x$  comes back exactly.
56     # TODO 5: repeat with the hold kernel and print one or two lines contrasting the two,
57     #           including which one shifts the output and why.
58
59     # PLOTTING (write this part yourself):
60     # - Figure 1: 3 stacked stem subplots -  $x[n]$ ,  $x_{\text{up}}[n]$ ,  $h_{\text{triangle}}[m]$ . Keep the
61     #    $x_{\text{up}}$  subplot on its own (much wider) time axis; do not force a shared axis here.
62     # - Figure 2: stem  $y_{\text{linear}}[n]$  and then overlay the original samples as large hollow
63     #   circles placed at  $n = L * k$ . Every circle should sit exactly on a stem tip: that is
64     #   the visual proof that interpolation preserved the samples.
65     # - Figure 3: put the triangular and hold outputs on the same axes as step-like lines
66     #   (plt.step with where='post' suits the hold version) so the staircase versus
67     #   straight-line difference is obvious. Add a legend.
68     # - xlabel "n", grid on, save as p11_expand.png, p11_interpolated.png, p11_kernels.png.
69
70
71 if __name__ == "__main__":
72     main()

```

Problem 11 — Solution

Solution — approach and discussion

The sample-preservation property is worth deriving rather than just observing. Since $x_{\uparrow}$ is nonzero only at multiples of L ,

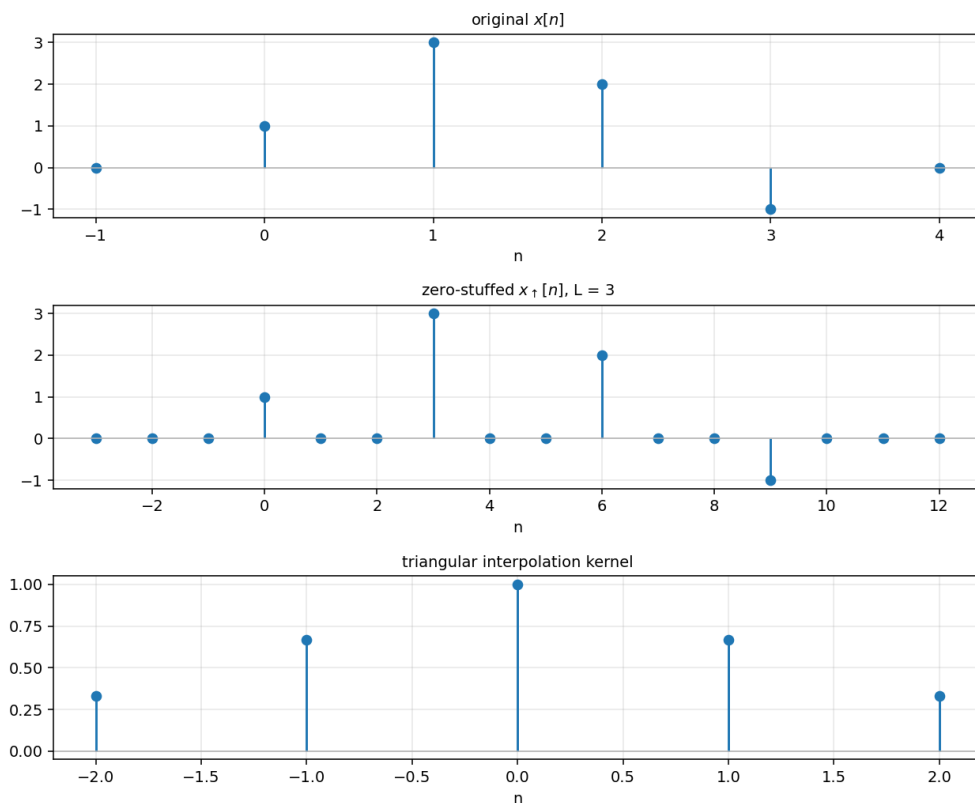
$$y[Ln] = \sum_k x_{\uparrow}[k] h[Ln - k] = \sum_j x[j] h(L(n - j)),$$

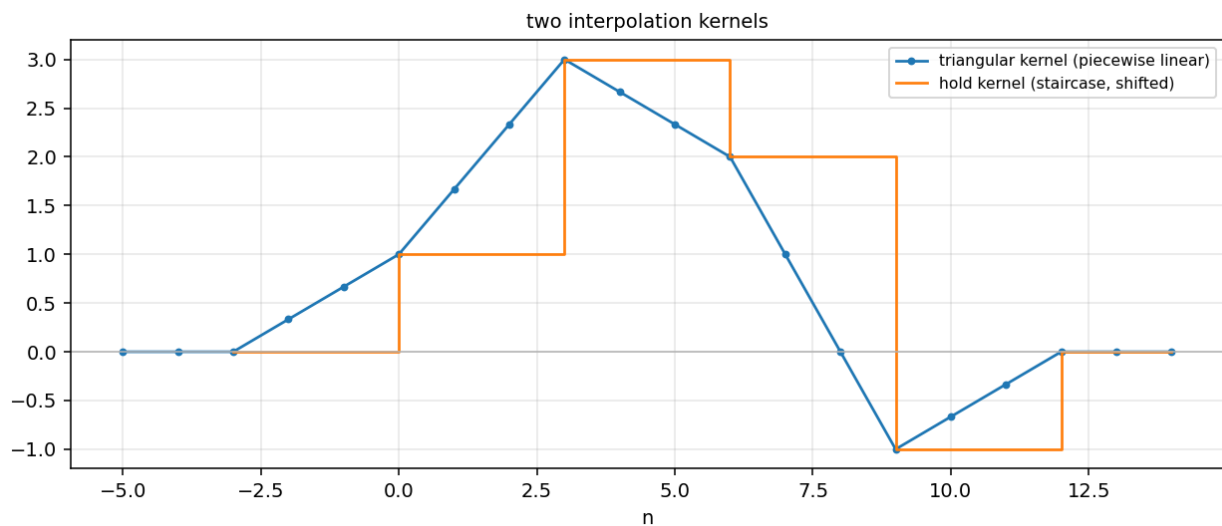
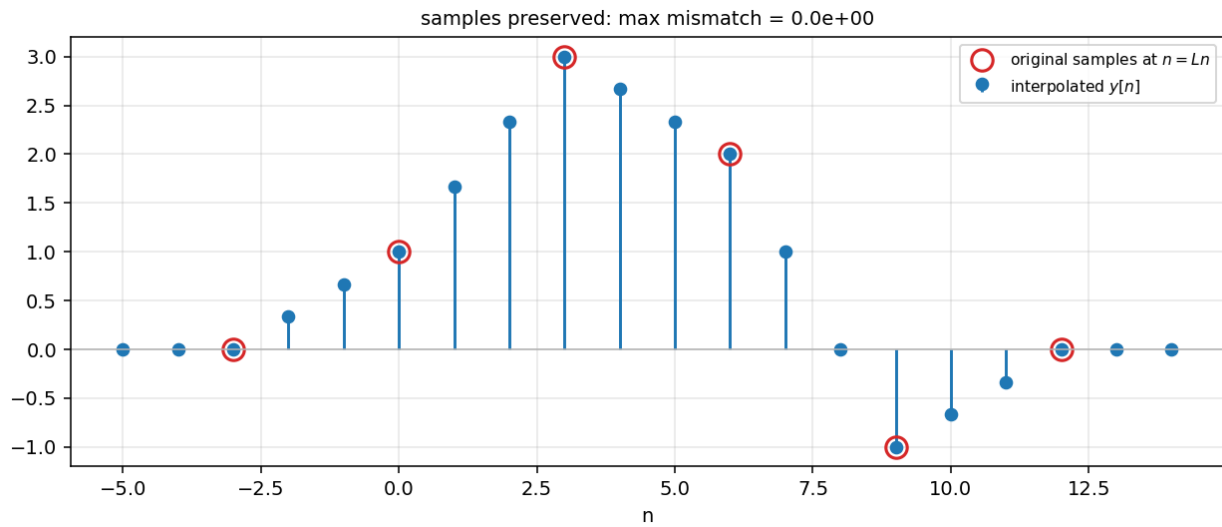
and the triangular kernel vanishes at every nonzero multiple of L while $h[0] = 1$. Only the $j = n$ term survives, giving $y[Ln] = x[n]$ exactly — which is why `downsample` then returns the original signal with zero error. Any kernel with $h[0] = 1$ and $h[\pm L] = h[\pm 2L] = \dots = 0$ would do the same; the triangle is just the simplest one, and the ideal sinc is the other end of the same family.

The alignment difference. The triangular kernel is stored on $[-(L - 1), L - 1]$, so it is centred and non-causal: its group delay is zero and the interpolated signal stays aligned with the original. The hold kernel lives on $[0, L - 1]$, group delay $(L - 1)/2 = 1$, so its output is shifted by one sample of the new (fast) rate. Both reproduce the original sample values, but only one of them puts them back in the right place — exactly the trade-off Problem 12 examines in general.

A common slip is scaling the amplitude. Zero stuffing does *not* need a factor of L here, because the kernel is normalised to $h[0] = 1$ rather than to unit DC gain. If you instead used $h = \{1/3, 1/3, 1/3\}$ -style normalisation you would need the factor, and the sample-preservation check is what would catch the omission.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 11 - Upsampling by zero stuffing and interpolation (SOLUTION)
3
4  Rate change in two steps:
5      1. expand:  $x_{up}[n] = x[n/L]$  when  $L$  divides  $n$ , and 0 otherwise;
6      2. interpolate:  $y = x_{up} * h$ , where  $h$  is the interpolation kernel.
7  With the triangular kernel  $h[m] = 1 - |m|/L$  for  $|m| < L$ , the result is exactly
8  piecewise-linear interpolation, and  $h[0] = 1$  with  $h[+L] = 0$  guarantees that the
9  original samples survive untouched at  $y[L*n] = x[n]$ .
10 """
11
12 import numpy as np
13 import matplotlib.pyplot as plt
14 from signal_lti import DiscreteSignal, LTISystem
15
16
17 def make_signal(start_time, values):
18     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
19     for offset, value in enumerate(values):
20         signal.set_value_at_time(start_time + offset, value)

```

```

21     return signal
22
23
24 def upsample(x, factor):
25     """Insert factor-1 zeros between neighbouring samples of x."""
26     if factor < 1:
27         raise ValueError("upsampling factor must be at least 1")
28     expanded = DiscreteSignal(factor * x.start_time, factor * x.end_time)
29     for n in range(x.start_time, x.end_time + 1):
30         expanded.set_value_at_time(factor * n, x.get_value_at_time(n))
31     return expanded
32
33
34 def downsample(x, factor):
35     """Keep every factor-th sample: y[n] = x[factor*n]."""
36     if factor < 1:
37         raise ValueError("downsampling factor must be at least 1")
38     start_time = int(np.ceil(x.start_time / factor))
39     end_time = int(np.floor(x.end_time / factor))
40     compressed = DiscreteSignal(start_time, end_time)
41     for n in range(start_time, end_time + 1):
42         compressed.set_value_at_time(n, x.get_value_at_time(factor * n))
43     return compressed
44
45
46 def triangular_kernel(factor):
47     """h[m] = 1 - |m|/factor for |m| <= factor-1: linear interpolation."""
48     kernel = DiscreteSignal(-(factor - 1), factor - 1)
49     for m in range(-(factor - 1), factor):
50         kernel.set_value_at_time(m, 1.0 - abs(m) / factor)
51     return kernel
52
53
54 def hold_kernel(factor):
55     """h[m] = 1 for 0 <= m < factor: sample-and-hold (staircase) interpolation."""
56     kernel = DiscreteSignal(0, factor - 1)
57     for m in range(factor):
58         kernel.set_value_at_time(m, 1.0)
59     return kernel
60
61
62 def max_abs_diff(first, second):
63     difference = first.add(second.multiply(-1))
64     return float(np.max(np.abs(difference.values)))
65
66
67 def describe(name, signal):
68     values = " ".join(f"{v:+.3f}" for v in signal.values)
69     return f"{name}: [{signal.start_time}..{signal.end_time}] {values}"
70
71
72 def main():
73     factor = 3
74     x = make_signal(-1, [0, 1, 3, 2, -1, 0])
75
76     # ---- Task 1: expander ----
77     x_up = upsample(x, factor)
78     print(describe("x", x))
79     print(describe("x_up", x_up))
80     print(f"length {len(x)} -> {len(x_up)}, range scaled by {factor}")
81     print()
82
83     # ---- Task 2: interpolate with the triangular kernel ----
84     h_triangle = triangular_kernel(factor)
85     y_linear = LTISystem(h_triangle).output(x_up)
86     print(describe("h_triangle", h_triangle))
87     print(describe("y_linear", y_linear))
88     print()
89
90     # ---- Task 3: the original samples must be preserved ----
91     print("checking y[L*n] == x[n]:")
92     worst = 0.0
93     for n in range(x.start_time, x.end_time + 1):
94         original = x.get_value_at_time(n)

```

```

95     interpolated = y_linear.get_value_at_time(factor * n)
96     worst = max(worst, abs(original - interpolated))
97     print(f" n = {n:2d}: x[n] = {original:+.3f}, "
98           f"y[{factor * n:3d}] = {interpolated:+.3f}")
99     print(f"max mismatch = {worst:.3e}")
100    print()
101
102    # ---- Task 4: downsampling back recovers the original exactly ----
103    x_recovered = downsample(y_linear, factor)
104    print(describe("downsample(y_linear, L)", x_recovered))
105    print(f"max |x - downsample(y, L)| over the original range = "
106          f"{max_abs_diff(x, x_recovered):.3e}")
107    print()
108
109    # ---- Task 5: compare with sample-and-hold ----
110    y_hold = LTISystem(hold_kernel(factor)).output(x_up)
111    print(describe("y_hold", y_hold))
112    print("Sample-and-hold reproduces the samples too, but between them it is a")
113    print("staircase, while the triangular kernel joins them with straight lines.")
114    print("The hold kernel is causal (taps at 0..L-1) so its output is shifted;")
115    print("the triangular kernel is centred, so its output stays aligned with x.")
116
117    # Figures described below; the implementation follows.
118    # - Figure 1: 3 stacked stem subplots - x[n], x_up[n], h_triangle[m]. Keep the
119    #   x_up subplot on its own (much wider) time axis; do not force a shared axis here.
120    # - Figure 2: stem y_linear[n] and then overlay the original samples as large hollow
121    #   circles placed at n = L*k. Every circle should sit exactly on a stem tip: that is
122    #   the visual proof that interpolation preserved the samples.
123    # - Figure 3: put y_linear and y_hold on the same axes as step-like lines
124    #   (plt.step with where='post' suits the hold version) so the staircase versus
125    #   straight-line difference is obvious. Add a legend.
126    # - xlabel "n", grid on, save as p11_expand.png, p11_interpolated.png, p11_kernels.png.
127
128    # ---- Figures ----
129    # Figure 1: expansion. The upsampled signal lives on a much wider axis, so it
130    # gets its own subplot rather than a shared one.
131    figure, axes = plt.subplots(3, 1, figsize=(9, 7.5))
132    axes[0].stem(x.times(), x.values, basefmt=" ")
133    axes[0].set_title("original $x[n]$", fontsize=10)
134    axes[1].stem(x_up.times(), x_up.values, basefmt=" ")
135    axes[1].set_title(f"zero-stuffed $x_{\\uparrow}[n]$, L = {factor}", fontsize=10)
136    axes[2].stem(h_triangle.times(), h_triangle.values, basefmt=" ")
137    axes[2].set_title("triangular interpolation kernel", fontsize=10)
138    for axis in axes:
139        axis.axhline(0, color="0.7", linewidth=0.8)
140        axis.set_xlabel("n")
141        axis.grid(True, alpha=0.3)
142    figure.tight_layout()
143    figure.savefig("p11_expand.png", dpi=140)
144    plt.close(figure)
145
146    # Figure 2: every original sample must land exactly on a stem tip.
147    figure, axis = plt.subplots(figsize=(9, 4))
148    axis.stem(y_linear.times(), y_linear.values, basefmt=" ", label="interpolated $y[n]$"
149    axis.plot([factor * n for n in x.times()], x.values, "o", markersize=11,
150             markerfacecolor="none", markeredgecolor="C3", markeredgewidth=1.6,
151             linestyle="none", label="original samples at $n = Ln$")
152    axis.axhline(0, color="0.7", linewidth=0.8)
153    axis.set_title(f"samples preserved: max mismatch = {worst:.1e}", fontsize=10)
154    axis.set_xlabel("n")
155    axis.grid(True, alpha=0.3)
156    axis.legend(fontsize=8)
157    figure.tight_layout()
158    figure.savefig("p11_interpolated.png", dpi=140)
159    plt.close(figure)
160
161    # Figure 3: straight lines versus a staircase.
162    figure, axis = plt.subplots(figsize=(9, 4))
163    axis.plot(y_linear.times(), y_linear.values, marker="o", markersize=3,
164             linewidth=1.4, label="triangular kernel (piecewise linear)")
165    axis.step(y_hold.times(), y_hold.values, where="post", linewidth=1.4,
166            color="C1", label="hold kernel (staircase, shifted)")
167    axis.axhline(0, color="0.7", linewidth=0.8)
168    axis.set_title("two interpolation kernels", fontsize=10)

```

```

169     axis.set_xlabel("n")
170     axis.grid(True, alpha=0.3)
171     axis.legend(fontsize=8)
172     figure.tight_layout()
173     figure.savefig("p11_kernels.png", dpi=140)
174     plt.close(ffigure)
175     print("\nsaved p11_expand.png, p11_interpolated.png, p11_kernels.png")
176
177 if __name__ == "__main__":
178     main()

```

Expected console output

```

x          : [-1..4] +0.000 +1.000 +3.000 +2.000 -1.000 +0.000
x_up       : [-3..12] +0.000 +0.000 +0.000 +1.000 +0.000 +0.000 +3.000 +0.000 +0.000 +2.000 +0.000 +0.000 -
              1.000 +0.000 +0.000 +0.000
length 6 -> 16, range scaled by 3

h_triangle: [-2..2] +0.333 +0.667 +1.000 +0.667 +0.333
y_linear   : [-5..14] +0.000 +0.000 +0.000 +0.333 +0.667 +1.000 +1.667 +2.333 +3.000 +2.667 +2.333 +2.000
              +1.000 +0.000 -1.000 -0.667 -0.333 +0.000 +0.000 +0.000

checking y[L*n] == x[n]:
  n = -1: x[n] = +0.000, y[ -3] = +0.000
  n =  0: x[n] = +1.000, y[  0] = +1.000
  n =  1: x[n] = +3.000, y[  3] = +3.000
  n =  2: x[n] = +2.000, y[  6] = +2.000
  n =  3: x[n] = -1.000, y[  9] = -1.000
  n =  4: x[n] = +0.000, y[ 12] = +0.000
max mismatch = 0.000e+00

downsample(y_linear, L): [-1..4] +0.000 +1.000 +3.000 +2.000 -1.000 +0.000
max |x - downsample(y, L)| over the original range = 0.000e+00

y_hold     : [-3..14] +0.000 +0.000 +0.000 +1.000 +1.000 +1.000 +3.000 +3.000 +3.000 +2.000 +2.000 +2.000 -
              1.000 -1.000 -1.000 +0.000 +0.000 +0.000
Sample-and-hold reproduces the samples too, but between them it is a
staircase, while the triangular kernel joins them with straight lines.
The hold kernel is causal (taps at 0..L-1) so its output is shifted;
the triangular kernel is centred, so its output stays aligned with x.

saved p11_expand.png, p11_interpolated.png, p11_kernels.png

```

Problem 12. Filter delay and zero-phase forward-backward filtering

Total Marks: 10

Suggested Time: 30 minutes

New

A causal M -point moving average has its taps' centre of mass at $(M - 1)/2$, so it delays whatever it filters by that many samples — a real problem when you need to know *when* something happened. There are two cures:

- shift the output back by $(M - 1)/2$, equivalently centre the kernel;
- filter forwards, reverse, filter again, reverse back. That is convolution with $h * \tilde{h}$, which is an even sequence and therefore has zero phase.

Use a Gaussian-shaped bump centred at $n = 20$ on $[0, 40]$ and the 5-point moving average.

Tasks

1. Implement `group_delay(h) =  $\sum_n n h[n] / \sum_n h[n]$` and report it for the 5-point average.
2. Filter the bump and show the peak moved by exactly the group delay.
3. Shift the output back and confirm the peak realigns.
4. Build a centred kernel instead and show it produces exactly the same signal as the shifted output.
5. Implement `forward_backward(x, h)`. Compute $h * \tilde{h}$, verify it is even, verify its group delay is 0, and verify the result equals $x * (h * \tilde{h})$.
6. Compare the peak heights of x , the single-pass output and the two-pass output, and say what the second pass costs.

Marks Distribution

Component	Marks
<code>group_delay</code> and the measured shift	2
Delay compensation by shifting	2
Centred kernel giving an identical result	2
<code>forward_backward</code> implemented with <code>flip</code>	3
Evenness of $h * \tilde{h}$ and the cost of the second pass	1
Total	10

Template

```

1  """
2  Problem 12 - Filter delay and zero-phase forward-backward filtering (TEMPLATE)
3
4  A causal M-point moving average delays by (M-1)/2. Two cures: shift the output
5  back, or filter forwards, reverse, filter again, reverse back - which is the
6  same as convolving with h * flip(h), an even sequence with zero phase.
7  """
8
9  import numpy as np
10 from signal_lti import DiscreteSignal, LTISystem
11
12
13 def make_signal(start_time, values):
14     # TODO

```

```

15     raise NotImplementedError
16
17
18 def flip(signal):
19     # TODO
20     raise NotImplementedError
21
22
23 def moving_average(length):
24     # TODO
25     raise NotImplementedError
26
27
28 def group_delay(h):
29     """Centre of mass of the taps:  $\text{sum}(n \cdot h[n]) / \text{sum}(h[n])$ ."""
30     # TODO
31     raise NotImplementedError
32
33
34 def peak_time(signal):
35     """TIME INDEX of the maximum sample."""
36     # TODO
37     raise NotImplementedError
38
39
40 def forward_backward(x, h):
41     """Filter forwards, reverse, filter again, reverse back."""
42     # TODO
43     raise NotImplementedError
44
45
46 def is_even(signal, tolerance=1e-12):
47     """True when  $\text{signal}[n] == \text{signal}[-n]$  for every stored index."""
48     # TODO
49     raise NotImplementedError
50
51
52 def max_abs_diff(first, second):
53     # TODO
54     raise NotImplementedError
55
56
57 def main():
58     times = np.arange(0, 41)
59     x = make_signal(0, np.exp(-((times - 20.0) ** 2) / 12.0))
60     h = moving_average(5)
61
62     # TODO 1: report the group delay of h, then filter x and show the peak moved by it.
63     # TODO 2: compensate by shifting the output back; confirm the peak realigns.
64     # TODO 3: build a centred kernel instead and show it gives exactly the same signal
65     #           as the shifted output.
66     # TODO 4: apply forward-backward filtering. Compute  $h * \text{flip}(h)$ , verify it is even,
67     #           verify its group delay is 0, and verify  $y_{\text{fb}}$  equals  $x * (h * \text{flip}(h))$ .
68     # TODO 5: compare the peak heights of x, the single-pass output and  $y_{\text{fb}}$ , and say in
69     #           a printed line what the second pass costs you.
70
71     # PLOTTING (write this part yourself):
72     # - Figure 1: put  $x[n]$ , the causal output and the shifted output on one axes as
73     #   lines with markers, add a legend, and drop vertical dashed lines (axvline) at
74     #   each peak location so the 2-sample shift and its correction are easy to see.
75     # - Figure 2: 2 stem subplots -  $h[n]$  on top and  $h * \text{flip}(h)$  below. Draw a vertical
76     #   line at  $n = 0$  on the lower one to show the symmetry axis.
77     # - Figure 3: overlay  $x[n]$ , the causal output and  $y_{\text{fb}}[n]$  to compare peak heights,
78     #   and annotate each peak height with plt.annotate.
79     # - xlabel "n", grid on, legends on the overlays, save as p12_delay.png,
80     #   p12_kernels.png, p12_zero_phase.png.
81
82
83 if __name__ == "__main__":
84     main()

```

Problem 12 — Solution

Solution — approach and discussion

Why forward-backward filtering has zero phase. Write $\tilde{h}[m] = h[-m]$. Filtering forwards, reversing, filtering again and reversing back gives

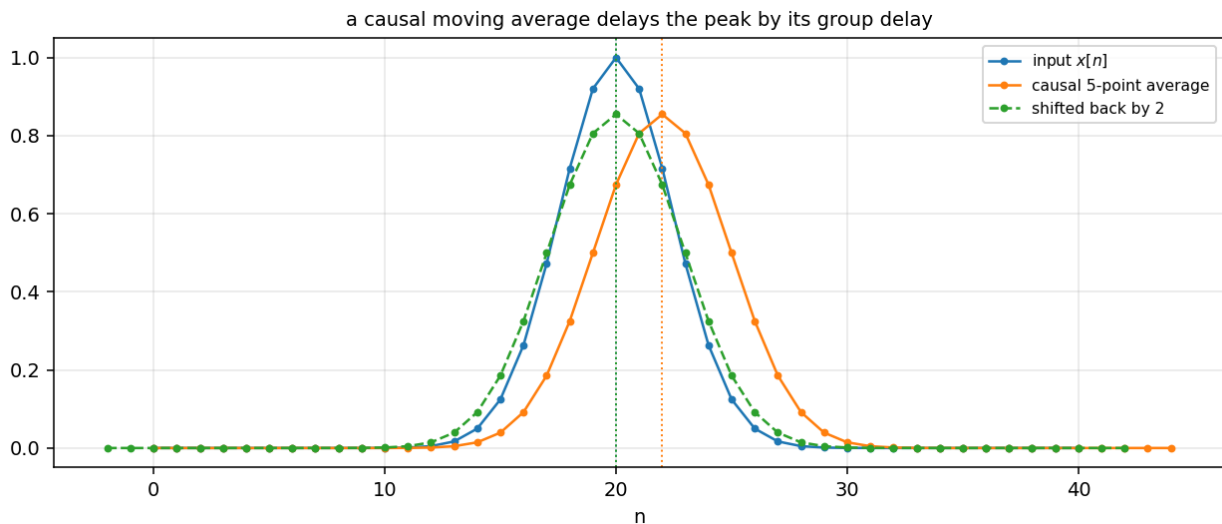
$$\mathcal{F}\{x\} = \widetilde{(x * h) * h} = x * h * \tilde{h},$$

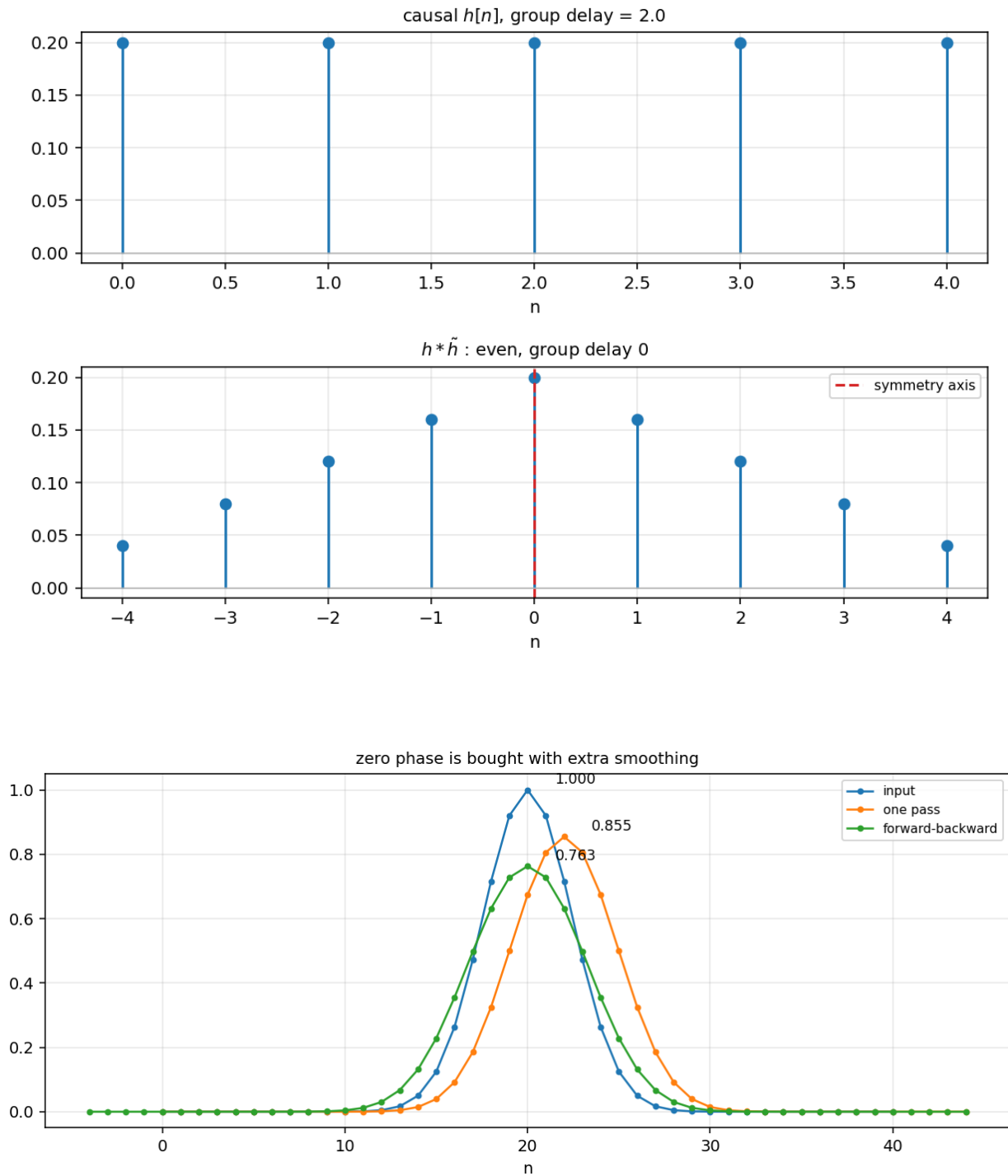
and $g = h * \tilde{h}$ satisfies $g[-n] = g[n]$, so its frequency response $G(\omega) = |H(\omega)|^2$ is real and non-negative. Real response means zero phase, which is why the peak does not move at all — not by a rounded number of samples, but exactly. The program confirms this three ways: g is symmetric on $[-4, 4]$, its group delay is 0, and the two-pass output matches $x * g$ to 10^{-16} .

Shifting versus centring. These give bitwise identical results here, because `shift` only relabels indices. It is worth knowing that this exactness depends on M being odd: for even M the group delay is a half-integer and no integer shift can fix it, which is one reason odd-length symmetric filters are preferred.

The cost. The bump's peak falls from 1.000 to 0.855 after one pass and to 0.763 after two. Zero phase is bought with a squared magnitude response — the filter smooths twice as hard. If you need a specific amount of smoothing *and* zero phase, use a shorter filter in the two-pass scheme, not the same one.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 12 - Filter delay and zero-phase forward-backward filtering (SOLUTION)
3
4  A causal M-point moving average has its "centre of mass" at (M-1)/2, so it
5  delays whatever it filters by that many samples. Two cures:
6  1. shift the output back by (M-1)/2 (only exact when M is odd);
7  2. filter forwards, reverse, filter again, reverse back. That is equivalent to
8  convolving with  $h * \text{flip}(h)$ , which is an even sequence and therefore has
9  zero phase - at the price of squaring the magnitude response.
10 """
11

```

```

12 import numpy as np
13 import matplotlib.pyplot as plt
14 from signal_lti import DiscreteSignal, LTISystem
15
16
17 def make_signal(start_time, values):
18     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
19     for offset, value in enumerate(values):
20         signal.set_value_at_time(start_time + offset, value)
21     return signal
22
23
24 def flip(signal):
25     reversed_signal = DiscreteSignal(-signal.end_time, -signal.start_time)
26     for n in range(signal.start_time, signal.end_time + 1):
27         reversed_signal.set_value_at_time(-n, signal.get_value_at_time(n))
28     return reversed_signal
29
30
31 def moving_average(length):
32     return make_signal(0, [1.0 / length] * length)
33
34
35 def group_delay(h):
36     """Centre of mass of the taps:  $\sum(n \cdot h[n]) / \sum(h[n])$ ."""
37     numerator = sum(n * h.get_value_at_time(n)
38                     for n in range(h.start_time, h.end_time + 1))
39     return numerator / float(np.sum(h.values))
40
41
42 def peak_time(signal):
43     return signal.start_time + int(np.argmax(signal.values))
44
45
46 def forward_backward(x, h):
47     """Filter forwards, reverse, filter again, reverse back."""
48     forward = LTISystem(h).output(x)
49     backward = LTISystem(h).output(flip(forward))
50     return flip(backward)
51
52
53 def max_abs_diff(first, second):
54     difference = first.add(second.multiply(-1))
55     return float(np.max(np.abs(difference.values)))
56
57
58 def is_even(signal, tolerance=1e-12):
59     """True when  $signal[n] == signal[-n]$  for every stored index."""
60     if signal.start_time != -signal.end_time:
61         return False
62     return bool(np.max(np.abs(signal.values - signal.values[::-1])) < tolerance)
63
64
65 def main():
66     # A symmetric bump centred at  $n = 20$ , so the peak location is unambiguous.
67     times = np.arange(0, 41)
68     x = make_signal(0, np.exp(-((times - 20.0) ** 2) / 12.0))
69
70     h = moving_average(5)
71     delay = group_delay(h)
72     print(f"h = 5-point moving average over [{h.start_time}, {h.end_time}]")
73     print(f"group delay =  $\sum(n \cdot h[n]) / \sum(h[n]) = {delay:.2f}$  samples")
74     print()
75
76     # ---- Task 1: the causal filter delays the signal ----
77     y_causal = LTISystem(h).output(x)
78     print(f"peak of x at n = {peak_time(x)}")
79     print(f"peak of y_causal at n = {peak_time(y_causal)} "
80           f"f(shifted by {peak_time(y_causal) - peak_time(x)})")
81
82     # ---- Task 2: compensate by shifting back ----
83     y_compensated = y_causal.shift(-int(delay))
84     print(f"peak of y_shifted at n = {peak_time(y_compensated)} (realigned)")
85     print()

```

```

86
87 # ---- Task 3: the same thing done with a centred kernel ----
88 h_centred = h.shift(-int(delay))
89 y_centred = LTISystem(h_centred).output(x)
90 print(f"h_centred spans [{h_centred.start_time}, {h_centred.end_time}], "
91       f"group delay = {group_delay(h_centred):.2f}")
92 print(f"peak of y_centred at n = {peak_time(y_centred)}")
93 print(f"max |y_shifted - y_centred| = {max_abs_diff(y_compensated, y_centred):.3e}")
94 print("Shifting the output and centring the kernel are the same operation.")
95 print()
96
97 # ---- Task 4: forward-backward filtering ----
98 y_fb = forward_backward(x, h)
99 h_effective = LTISystem(flip(h)).output(h)          # h * flip(h)
100 y_effective = LTISystem(h_effective).output(x)
101
102 print(f"h * flip(h) spans [{h_effective.start_time}, {h_effective.end_time}]")
103 print("taps: " + " ".join(f"{v:.4f}" for v in h_effective.values))
104 print(f"h * flip(h) is an even sequence: {is_even(h_effective)}")
105 print(f"its group delay = {group_delay(h_effective):.2f} (zero phase)")
106 print(f"peak of y_fb at n = {peak_time(y_fb)} (no shift at all)")
107 print(f"max |y_fb - x*(h*flip(h))| = {max_abs_diff(y_fb, y_effective):.3e}")
108 print()
109
110 # ---- Task 5: what the second pass costs ----
111 print(f"peak height of x      = {np.max(x.values):.4f}")
112 print(f"peak height of y_causal = {np.max(y_causal.values):.4f}")
113 print(f"peak height of y_fb    = {np.max(y_fb.values):.4f}")
114 print("Two passes smooth twice as hard: the phase error disappears but the")
115 print("magnitude response is squared, so the bump is flattened further.")
116
117 # Figures described below; the implementation follows.
118 # - Figure 1: put x[n], y_causal[n] and y_compensated[n] on one axes as lines with
119 #   markers, add a legend, and drop vertical dashed lines (axvline) at each peak
120 #   location so the 2-sample shift and its correction are easy to see.
121 # - Figure 2: 2 stem subplots - h[n] on top and h*flip(h) below. Draw a vertical
122 #   line at n = 0 on the lower one to show the symmetry axis.
123 # - Figure 3: overlay x[n], y_causal[n] and y_fb[n] to compare peak heights, and
124 #   annotate each peak height with plt.annotate.
125 # - xlabel "n", grid on, legends on the overlays, save as p12_delay.png,
126 #   p12_kernels.png, p12_zero_phase.png.
127
128 # ---- Figures ----
129 # Figure 1: the delay and its correction, with each peak marked.
130 figure, axis = plt.subplots(figsize=(9, 4))
131 for signal, label, style in [(x, "input  $x[n]$ ", "C0-"),
132                             (y_causal, "causal 5-point average", "C1-"),
133                             (y_compensated, "shifted back by 2", "C2--")]:
134     axis.plot(signal.times(), signal.values, style, marker=".", linewidth=1.3,
135              label=label)
136     axis.axvline(peak_time(signal), color=style[:2], linestyle=":", linewidth=1.0)
137 axis.set_title("a causal moving average delays the peak by its group delay", fontsize=10)
138 axis.set_xlabel("n")
139 axis.grid(True, alpha=0.3)
140 axis.legend(fontsize=8)
141 figure.tight_layout()
142 figure.savefig("p12_delay.png", dpi=140)
143 plt.close.figure)
144
145 # Figure 2: the kernels. The lower one is symmetric about n = 0.
146 figure, axes = plt.subplots(2, 1, figsize=(8, 5.5))
147 axes[0].stem(h.times(), h.values, basefmt=" ")
148 axes[0].set_title(f"causal  $h[n]$ , group delay = {delay:.1f}", fontsize=10)
149 axes[1].stem(h_effective.times(), h_effective.values, basefmt=" ")
150 axes[1].axvline(0, color="C3", linestyle="--", label="symmetry axis")
151 axes[1].set_title(r" $h * \tilde{h}$  : even, group delay 0", fontsize=10)
152 axes[1].legend(fontsize=8)
153 for axis in axes:
154     axis.axhline(0, color="0.7", linewidth=0.8)
155     axis.set_xlabel("n")
156     axis.grid(True, alpha=0.3)
157 figure.tight_layout()
158 figure.savefig("p12_kernels.png", dpi=140)
159 plt.close.figure)

```

```

160
161     # Figure 3: what the second pass costs in peak height.
162     figure, axis = plt.subplots(figsize=(9, 4))
163     for signal, label in [(x, "input"), (y_causal, "one pass"), (y_fb, "forward-backward")]:
164         axis.plot(signal.times(), signal.values, marker=".", linewidth=1.3, label=label)
165         top = float(np.max(signal.values))
166         axis.annotate(f"{{top:.3f}}", xy=(peak_time(signal), top),
167                      xytext=(peak_time(signal) + 1.5, top + 0.02), fontsize=9)
168     axis.set_title("zero phase is bought with extra smoothing", fontsize=10)
169     axis.set_xlabel("n")
170     axis.grid(True, alpha=0.3)
171     axis.legend(fontsize=8)
172     figure.tight_layout()
173     figure.savefig("p12_zero_phase.png", dpi=140)
174     plt.close(figure)
175     print("\nsaved p12_delay.png, p12_kernels.png, p12_zero_phase.png")
176
177 if __name__ == "__main__":
178     main()

```

Expected console output

```

h = 5-point moving average over [0, 4]
group delay = sum(n*h[n]) / sum(h[n]) = 2.00 samples

peak of x          at n = 20
peak of y_causal at n = 22  (shifted by 2)
peak of y_shifted at n = 20  (realigned)

h_centred spans [-2, 2], group delay = -0.00
peak of y_centred at n = 20
max |y_shifted - y_centred| = 0.000e+00
Shifting the output and centring the kernel are the same operation.

h * flip(h) spans [-4, 4]
taps: 0.0400 0.0800 0.1200 0.1600 0.2000 0.1600 0.1200 0.0800 0.0400
h * flip(h) is an even sequence: True
its group delay = 0.00  (zero phase)
peak of y_fb at n = 20  (no shift at all)
max |y_fb - x*(h*flip(h))| = 1.110e-16

peak height of x          = 1.0000
peak height of y_causal = 0.8546
peak height of y_fb      = 0.7630
Two passes smooth twice as hard: the phase error disappears but the
magnitude response is squared, so the bump is flattened further.

saved p12_delay.png, p12_kernels.png, p12_zero_phase.png

```

Problem 13. Choosing a smoothing filter by measuring the error

Total Marks: 10
Suggested Time: 30 minutes
New

Two files are provided: the clean signal that was actually there, and the noisy measurement you would have recorded. Seven candidate smoothing kernels are to be applied, their delays removed, and the winner chosen by mean squared error against the clean signal.

The lesson is that longer is not automatically better. A long filter removes more noise but also smears genuine features — and the clean signal here contains a step at $n = 90$ specifically to expose that.

Files provided: `clean_signal.txt`, `noisy_signal.txt` (copy them next to your script before running).

Tasks

1. Read both files and choose a comparison window trimming about 16 samples from each end, so even the longest filter's transient stays outside it.
2. Report the MSE of the raw noisy signal as a baseline.
3. For each candidate: filter, shift back by `round(group delay)`, compute the MSE, and print a table of taps / delay / MSE / improvement factor. Build the table by looping over the dictionary.
4. Report the best and worst filter.
5. Print the clean signal beside the 5-point and 15-point outputs for $n = 86 \dots 96$, and explain why the longest filter is not the best overall.

Marks Distribution

Component	Marks
Correct interior comparison window and baseline MSE	2
Delay compensation applied to every candidate	3
MSE computed and tabulated over all seven filters	3
Best/worst identified	1
Correct explanation of the length trade-off	1
Total	10

Template

```

1  """
2  Problem 13 - Choosing a smoothing filter by measuring the error (TEMPLATE)
3
4  Files provided: clean_signal.txt, noisy_signal.txt
5  Apply several candidate kernels, remove the delay each one introduces, and score
6  them by mean squared error against the clean signal.
7  """
8
9  import numpy as np
10 from signal_lti import DiscreteSignal, LTISystem
11
12
13 def read_signal_from_file(path):
14     # TODO
15     raise NotImplementedError

```

```

16
17
18 def make_signal(start_time, values):
19     # TODO
20     raise NotImplementedError
21
22
23 def group_delay(h):
24     # TODO
25     raise NotImplementedError
26
27
28 def mean_squared_error(estimate, reference, start_time, end_time):
29     """MSE over an INTERIOR window, so filter start-up transients are excluded."""
30     # TODO
31     raise NotImplementedError
32
33
34 def main():
35     candidates = {
36         "3-point moving average": None,      # TODO fill these in
37         "5-point moving average": None,
38         "9-point moving average": None,
39         "15-point moving average": None,
40         "weighted [0.5, 0.3, 0.2]": None,
41         "binomial [1,2,1]/4": None,
42         "binomial [1,4,6,4,1]/16": None,
43     }
44
45     # TODO 1: read both files. Choose a comparison window that trims ~16 samples from
46     #           each end, so even the longest filter's transient stays outside it.
47     # TODO 2: report the MSE of the raw noisy signal - this is the baseline.
48     # TODO 3: for each candidate, filter, shift back by round(group_delay), compute the
49     #           MSE, and print a table with taps / delay / MSE / improvement factor.
50     #           Build the table with a loop over the dictionary, not by hand.
51     # TODO 4: report the best and worst filter.
52     # TODO 5: the clean signal has a step at n = 90. Print the clean signal next to the
53     #           5-point and 15-point outputs for n = 86..96 and explain in a printed line
54     #           why the longest filter is NOT the best one overall.
55
56     # PLOTTING (write this part yourself):
57     # - Figure 1: plot clean[n] and noisy[n] on the same axes as lines (150 samples is
58     #   too many for stems). Use a thin line for the noisy one and a thick line for the
59     #   clean one, plus a legend.
60     # - Figure 2: one subplot per candidate filter (a 4x2 grid works for 7 filters).
61     #   In each, plot the clean signal faintly and the delay-compensated output on top,
62     #   and put the filter name and its MSE in the subplot title. Build this with a loop
63     #   over the candidates dictionary, not by hand.
64     # - Figure 3: a bar chart of MSE per filter (plt.bar with the names as tick labels,
65     #   rotated so they stay readable) with a horizontal line at the raw noisy MSE.
66     # - Figure 4: zoom the x-axis to n = 80..100 with plt.xlim and overlay the clean
67     #   signal, the 5-point output and the 15-point output to show the step smearing.
68     # - xlabel "n", grid on, legends everywhere two series share an axes, save as
69     #   p13_signals.png, p13_grid.png, p13_mse.png, p13_step_zoom.png.
70
71
72 if __name__ == "__main__":
73     main()

```

Problem 13 — Solution

Solution — approach and discussion

The baseline MSE is 1.164. The ranking comes out as

9-point (0.141) < 15-point (0.165) < 5-point (0.229) < binomial-5 (0.299) < 3-point (0.369) < ...

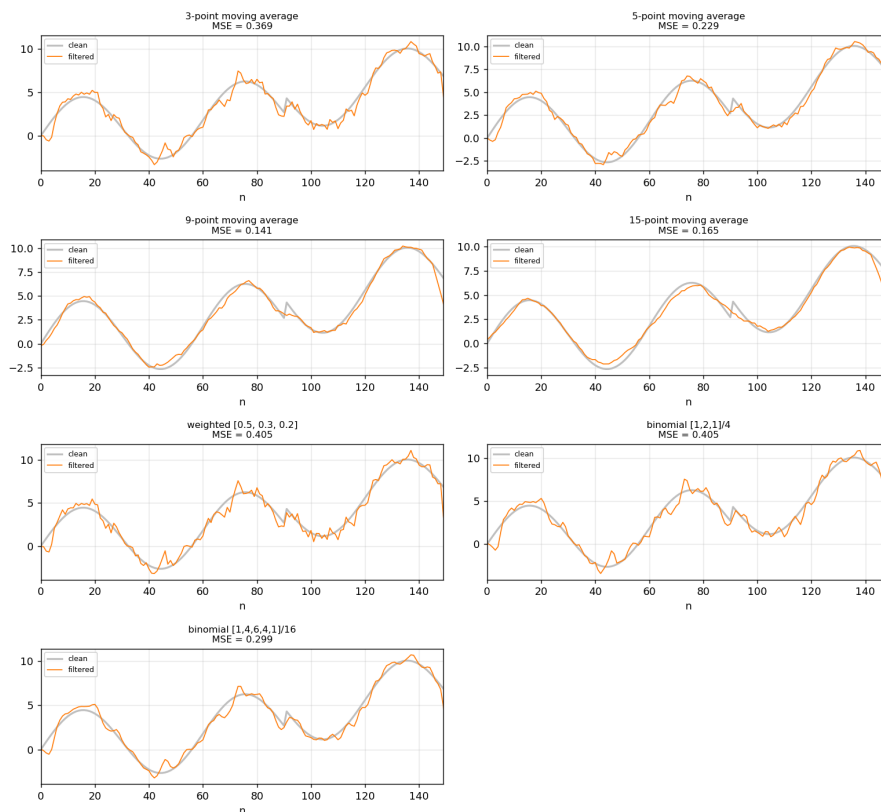
so the 9-point moving average wins with an $8.3\times$ improvement, and the 15-point average — the longest filter available — is *worse* than it.

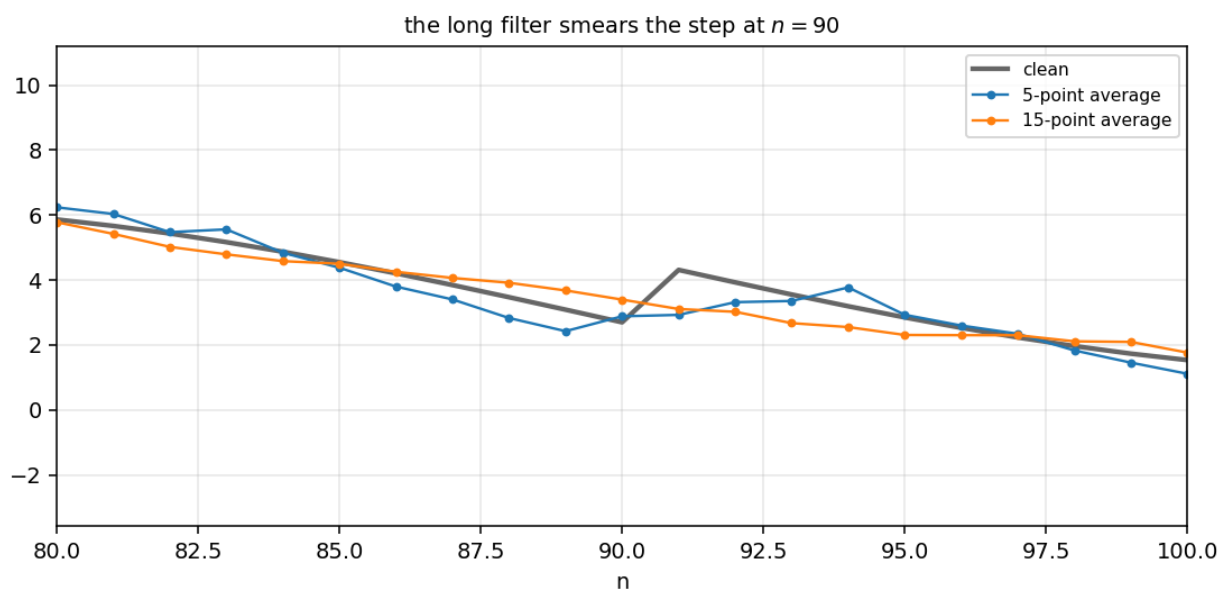
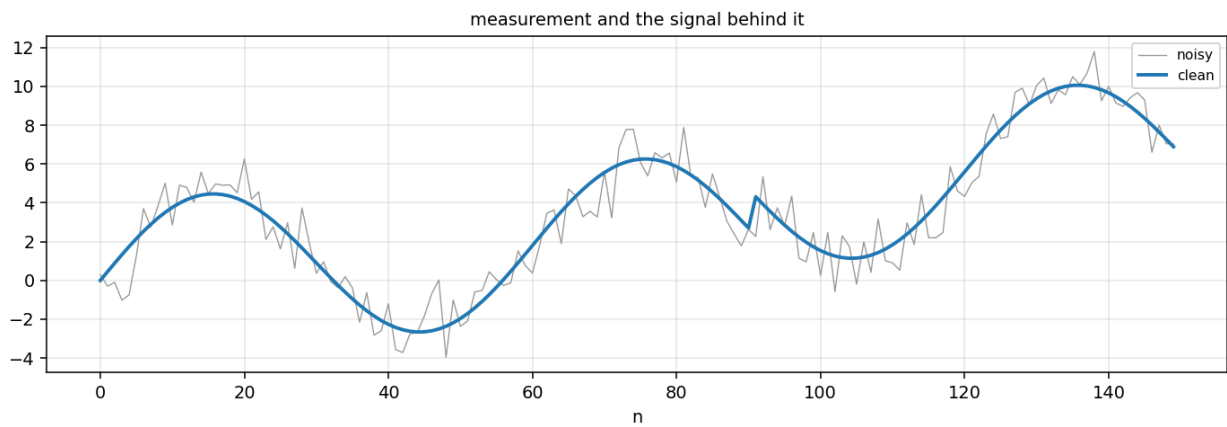
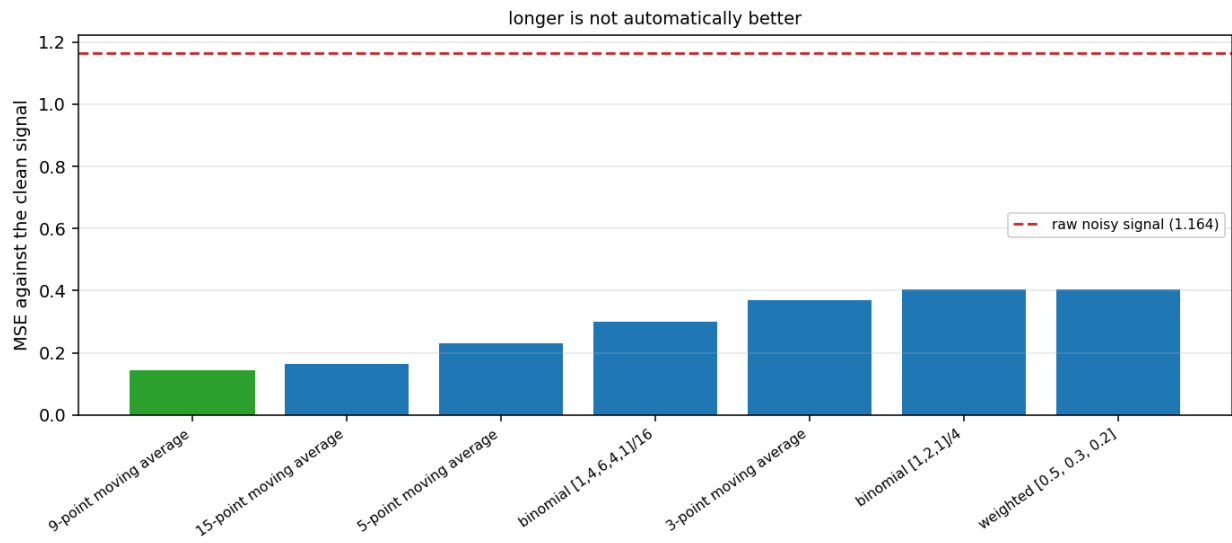
Why. For white noise of variance σ^2 , an M -point average reduces the noise variance to σ^2/M : monotonically better with length. But the same averaging biases the estimate wherever the underlying signal is not locally straight, and that bias grows with M . Total error is variance plus squared bias, so it falls, bottoms out, and rises again. The optimum here sits at $M \approx 9$; where exactly it sits depends on the noise level and on how sharp the features are, which is why this has to be measured rather than guessed.

The printout around $n = 90$ shows the bias directly: across the step the 15-point output lags badly and takes about fifteen samples to catch up, while the 5-point output tracks it within a few. Away from the step the ordering reverses. That is the whole trade-off in one table.

Two things worth noting on method. The interior window matters — scoring over the full range would penalise the long filters twice, once for bias and once for start-up transient, and the comparison would no longer be about filter length. And the weighted kernel $\{0.5, 0.3, 0.2\}$ finishes last despite having three taps, because it is asymmetric: its group delay of 0.7 cannot be removed by an integer shift, so it is scored while misaligned.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 13 - Choosing a smoothing filter by measuring the error (SOLUTION)
3
4  A noisy measurement and the clean signal behind it are both provided. Several
5  candidate smoothing kernels are applied, the delay each one introduces is
6  removed, and the mean squared error against the clean signal decides the winner.
7  The point of the exercise is that a longer filter is not automatically better:
8  it kills more noise but also rounds off genuine features.
9  """
10
11 import numpy as np
12 import matplotlib.pyplot as plt
13 from signal_lti import DiscreteSignal, LTISystem
14
15
16 def read_signal_from_file(path):
17     with open(path, "r", encoding="utf-8") as handle:
18         start_time, end_time = map(int, handle.readline().split())
19         values = list(map(float, handle.readline().split()))
20     if len(values) != end_time - start_time + 1:
21         raise ValueError(f"{path}: sample count does not match the declared range")
22     signal = DiscreteSignal(start_time, end_time)
23     for offset, value in enumerate(values):
24         signal.set_value_at_time(start_time + offset, value)
25     return signal
26
27
28 def make_signal(start_time, values):
29     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
30     for offset, value in enumerate(values):
31         signal.set_value_at_time(start_time + offset, value)
32     return signal
33
34
35 def group_delay(h):
36     numerator = sum(n * h.get_value_at_time(n)
37                     for n in range(h.start_time, h.end_time + 1))
38     return numerator / float(np.sum(h.values))
39
40
41 def mean_squared_error(estimate, reference, start_time, end_time):
42     """MSE over an interior window, so filter start-up transients are excluded."""
43     total = 0.0
44     count = 0
45     for n in range(start_time, end_time + 1):
46         gap = estimate.get_value_at_time(n) - reference.get_value_at_time(n)
47         total += gap * gap
48         count += 1
49     return total / count
50
51
52 def main():
53     clean = read_signal_from_file("clean_signal.txt")
54     noisy = read_signal_from_file("noisy_signal.txt")
55
56     candidates = {
57         "3-point moving average": make_signal(0, [1 / 3] * 3),
58         "5-point moving average": make_signal(0, [1 / 5] * 5),
59         "9-point moving average": make_signal(0, [1 / 9] * 9),
60         "15-point moving average": make_signal(0, [1 / 15] * 15),
61         "weighted [0.5, 0.3, 0.2]": make_signal(0, [0.5, 0.3, 0.2]),
62         "binomial [1,2,1]/4": make_signal(0, [0.25, 0.5, 0.25]),
63         "binomial [1,4,6,4,1]/16": make_signal(0, [1 / 16, 4 / 16, 6 / 16, 4 / 16, 1 / 16]),
64     }
65
66     # Compare on an interior window so the ramp-up and ramp-down of the longest
67     # filter never enters the score.
68     margin = 16
69     window_start = clean.start_time + margin
70     window_end = clean.end_time - margin
71

```

```

72 baseline = mean_squared_error(noisy, clean, window_start, window_end)
73 print(f"comparison window: n = {window_start} .. {window_end}")
74 print(f"MSE of the raw noisy signal = {baseline:.4f}")
75 print()
76 print(f"{'filter':>26} {'taps':>5} {'delay':>6} {'MSE':>9} {'improvement':>12}")
77
78 results = []
79 for name, h in candidates.items():
80     delay = group_delay(h)
81     y = LTISystem(h).output(noisy)
82     y_aligned = y.shift(-int(round(delay))) # undo the filter delay
83     error = mean_squared_error(y_aligned, clean, window_start, window_end)
84     results.append((error, name, len(h), delay))
85     print(f"{name:>26} {len(h):5d} {delay:6.2f} {error:9.4f} "
86           f"{baseline / error:11.2f}x")
87
88 results.sort()
89 best_error, best_name, _, _ = results[0]
90 worst_error, worst_name, _, _ = results[-1]
91 print()
92 print(f"best : {best_name} (MSE {best_error:.4f})")
93 print(f"worst: {worst_name} (MSE {worst_error:.4f})")
94 print()
95
96 # ---- Why the longest filter is not the best ----
97 # The clean signal contains a step at n = 90. A long filter smears it over
98 # its whole length, and that bias costs more than the extra noise it removes.
99 long_filter = candidates["15-point moving average"]
100 y_long = LTISystem(long_filter).output(noisy)
101 y_long = y_long.shift(-int(round(group_delay(long_filter))))
102 short_filter = candidates["5-point moving average"]
103 y_short = LTISystem(short_filter).output(noisy)
104 y_short = y_short.shift(-int(round(group_delay(short_filter))))
105
106 print("error right at the step (n = 86..96):")
107 print(f"{'n':>4} {'clean':>8} {'5-point':>9} {'15-point':>10}")
108 for n in range(86, 97):
109     print(f"{n:4d} {clean.get_value_at_time(n):8.3f} "
110           f"{y_short.get_value_at_time(n):9.3f} "
111           f"{y_long.get_value_at_time(n):10.3f}")
112
113 print()
114 print("Away from the step the 15-point filter has the lower noise, but across")
115 print("the step it lags badly. Filter length is a trade-off between noise")
116 print("rejection and how faithfully sharp features survive.")
117
118 # Figures described below; the implementation follows.
119 # - Figure 1: plot clean[n] and noisy[n] on the same axes as lines (150 samples is
120 #   too many for stems). Use a thin line for the noisy one and a thick line for the
121 #   clean one, plus a legend.
122 # - Figure 2: one subplot per candidate filter (a 4x2 grid works for 7 filters).
123 #   In each, plot the clean signal faintly and the delay-compensated output on top,
124 #   and put the filter name and its MSE in the subplot title. Build this with a loop
125 #   over the candidates dictionary, not by hand.
126 # - Figure 3: a bar chart of MSE per filter (plt.bar with the names as tick labels,
127 #   rotated so they stay readable) with a horizontal line at the raw noisy MSE.
128 # - Figure 4: zoom the x-axis to n = 80..100 with plt.xlim and overlay the clean
129 #   signal, the 5-point output and the 15-point output to show the step smearing.
130 # - xlabel "n", grid on, legends everywhere two series share an axes, save as
131 #   p13_signals.png, p13_grid.png, p13_mse.png, p13_step_zoom.png.
132
133 # ---- Figures ----
134 # Figure 1: what we are given.
135 figure, axis = plt.subplots(figsize=(10, 3.6))
136 axis.plot(noisy.times(), noisy.values, linewidth=0.7, color="0.6", label="noisy")
137 axis.plot(clean.times(), clean.values, linewidth=2.0, color="C0", label="clean")
138 axis.set_title("measurement and the signal behind it", fontsize=10)
139 axis.set_xlabel("n")
140 axis.grid(True, alpha=0.3)
141 axis.legend(fontsize=8)
142 figure.tight_layout()
143 figure.savefig("p13_signals.png", dpi=140)
144 plt.close(figure)
145
146 # Figure 2: one panel per candidate, built by looping over the dictionary.

```

```

146 figure, axes = plt.subplots(4, 2, figsize=(12, 11))
147 flat_axes = axes.ravel()
148 for axis, (name, kernel) in zip(flat_axes, candidates.items()):
149     filtered = LTISystem(kernel).output(noisy)
150     filtered = filtered.shift(-int(round(group_delay(kernel))))
151     score = mean_squared_error(filtered, clean, window_start, window_end)
152     axis.plot(clean.times(), clean.values, linewidth=1.8, color="0.75", label="clean")
153     axis.plot(filtered.times(), filtered.values, linewidth=1.0, color="C1",
154               label="filtered")
155     axis.set_xlim(clean.start_time, clean.end_time)
156     axis.set_title(f"{name}\nMSE = {score:.3f}", fontsize=9)
157     axis.set_xlabel("n")
158     axis.grid(True, alpha=0.3)
159     axis.legend(fontsize=7)
160 flat_axes[-1].axis("off") # 7 filters in an 8-panel grid
161 figure.tight_layout()
162 figure.savefig("p13_grid.png", dpi=130)
163 plt.close(figure)
164
165 # Figure 3: the scores side by side, against the do-nothing baseline.
166 names = [name for _, name, _ in results]
167 scores = [score for score, _, _ in results]
168
169 figure, axis = plt.subplots(figsize=(10, 4.5))
170 bars = axis.bar(range(len(names)), scores, color="C0")
171 bars[0].set_color("C2") # the winner
172 axis.axhline(baseline, color="C3", linestyle="--",
173             label=f"raw noisy signal ({baseline:.3f})")
174 axis.set_xticks(range(len(names)))
175 axis.set_xticklabels(names, rotation=35, ha="right", fontsize=8)
176 axis.set_ylabel("MSE against the clean signal")
177 axis.set_title("longer is not automatically better", fontsize=10)
178 axis.grid(True, alpha=0.3, axis="y")
179 axis.legend(fontsize=8)
180 figure.tight_layout()
181 figure.savefig("p13_mse.png", dpi=140)
182 plt.close(figure)
183
184 # Figure 4: zoom on the step, where the long filter loses.
185 figure, axis = plt.subplots(figsize=(8, 4))
186 axis.plot(clean.times(), clean.values, linewidth=2.2, color="0.4", label="clean")
187 axis.plot(y_short.times(), y_short.values, marker=".", linewidth=1.2,
188         label="5-point average")
189 axis.plot(y_long.times(), y_long.values, marker=".", linewidth=1.2,
190         label="15-point average")
191 axis.set_xlim(80, 100)
192 axis.set_title("the long filter smears the step at $n = 90$", fontsize=10)
193 axis.set_xlabel("n")
194 axis.grid(True, alpha=0.3)
195 axis.legend(fontsize=8)
196 figure.tight_layout()
197 figure.savefig("p13_step_zoom.png", dpi=140)
198 plt.close(figure)
199 print("\nsaved p13_signals.png, p13_grid.png, p13_mse.png, p13_step_zoom.png")
200
201 if __name__ == "__main__":
202     main()

```

Expected console output

```

comparison window: n = 16 .. 133
MSE of the raw noisy signal = 1.1639

```

	filter	taps	delay	MSE	improvement
	3-point moving average	3	1.00	0.3687	3.16x
	5-point moving average	5	2.00	0.2294	5.07x
	9-point moving average	9	4.00	0.1408	8.27x
	15-point moving average	15	7.00	0.1650	7.05x
	weighted [0.5, 0.3, 0.2]	3	0.70	0.4053	2.87x
	binomial [1,2,1]/4	3	1.00	0.4047	2.88x

```
binomial [1,4,6,4,1]/16    5    2.00    0.2988    3.90x

best : 9-point moving average (MSE 0.1408)
worst: weighted [0.5, 0.3, 0.2] (MSE 0.4053)

error right at the step (n = 86..96):
  n    clean    5-point    15-point
86    4.207    3.799    4.254
87    3.846    3.403    4.065
88    3.472    2.831    3.916
89    3.088    2.426    3.679
90    2.700    2.884    3.398
91    4.312    2.928    3.109
92    3.928    3.319    3.022
93    3.554    3.356    2.673
94    3.193    3.772    2.552
95    2.850    2.930    2.310
96    2.529    2.599    2.303

Away from the step the 15-point filter has the lower noise, but across
the step it lags badly. Filter length is a trade-off between noise
rejection and how faithfully sharp features survive.

saved p13_signals.png, p13_grid.png, p13_mse.png, p13_step_zoom.png
```

Problem 14. Causality, memory, BIBO stability, and attaining the bound

Total Marks: 10

Suggested Time: 30 minutes

New

Three system properties can be read straight off the impulse response:

$$\text{causal} \iff h[n] = 0 \quad \forall n < 0, \quad \text{memoryless} \iff h[n] = 0 \quad \forall n \neq 0,$$

$$\text{BIBO stable} \iff \sum_n |h[n]| < \infty, \quad \text{and then} \quad |y[n]| \leq \left(\max_n |x[n]| \right) \sum_n |h[n]|.$$

That last bound is not merely an upper limit — it is attained. Choosing $x[k] = \text{sign}(h[n_0 - k])$ makes every term of the convolution sum positive, so $|y[n_0]|$ reaches $\sum_n |h[n]|$ exactly. You will construct that input and watch the bound become an equality.

Tasks

1. Build the seven impulse responses listed in the template, including one stored on $[-2, 2]$ and two geometric ones r^n with $r = 0.8$ and $r = 1.2$.
2. Tabulate causal / memoryless / $\sum |h|$ / $\sum h$ for each.
3. Implement `worst_case_input(h, n0)` and show $y[0]$ equals $\sum_n |h[n]|$ exactly for every system.
4. Feed a random input with $|x[n]| \leq 1$ through each system and confirm $\max |y|$ stays under the bound.
5. Tabulate $\sum |h|$ for r^n truncated to $N = 10, 30, 100, 300$ taps at $r = 0.8, 1.0, 1.2$, and explain why every finite-length h looks stable.

Marks Distribution

Component	Marks
Property tests implemented correctly	3
<code>worst_case_input</code> with the right support and signs	3
Bound shown to be attained	2
Random-input check	1
Truncation discussion	1
Total	10

Template

```

1  """
2  Problem 14 - Causality, memory, BIBO stability, and attaining the bound
3  (TEMPLATE)
4
5  causal      <=> h[n] = 0 for all n < 0
6  memoryless  <=> h[n] = 0 for all n != 0
7  BIBO stable <=> sum |h[n]| < infinity, and |y[n]| <= (max|x|) * sum |h[n]|
8  """
9
10 import numpy as np
11 from signal_lti import DiscreteSignal, LTISystem
12

```

```

13
14 def make_signal(start_time, values):
15     # TODO
16     raise NotImplementedError
17
18
19 def is_causal(h, tolerance=1e-12):
20     # TODO
21     raise NotImplementedError
22
23
24 def is_memoryless(h, tolerance=1e-12):
25     # TODO
26     raise NotImplementedError
27
28
29 def absolute_sum(h):
30     """sum |h[n]|: the BIBO gain."""
31     # TODO
32     raise NotImplementedError
33
34
35 def dc_gain(h):
36     """sum h[n]: the gain applied to a constant input."""
37     # TODO
38     raise NotImplementedError
39
40
41 def worst_case_input(h, n0):
42     """
43     Bounded input with |x[k]| <= 1 that drives |y[n0]| as high as it can go.
44     Only the samples k with h[n0-k] != 0 matter.
45     """
46     # TODO
47     raise NotImplementedError
48
49
50 def geometric(ratio, length):
51     """h[n] = ratio^n for n = 0 .. length-1."""
52     # TODO
53     raise NotImplementedError
54
55
56 def main():
57     systems = {
58         "identity h[0]=1": None,          # TODO fill these in
59         "5-point moving average": None,
60         "centred 5-point average": None,  # note: starts at n = -2
61         "first difference": None,
62         "weighted [0.5,0.3,0.2]": None,
63         "0.8^n, 30 taps": None,
64         "1.2^n, 30 taps": None,
65     }
66
67     # TODO 1: table of causal / memoryless / sum|h| / sum h for each system.
68     # TODO 2: for each system build the worst-case input at n0 = 0, filter it, and
69     #          show that y[0] equals sum|h| exactly. This proves the bound is tight.
70     # TODO 3: feed a random input with |x[n]| <= 1 through each system and confirm
71     #          max|y| stays under sum|h|.
72     # TODO 4: tabulate sum|h| for h[n] = r^n truncated to N = 10, 30, 100, 300 taps
73     #          with r = 0.8, 1.0 and 1.2, and explain in a printed line why every
74     #          finite-length h looks stable no matter what r is.
75
76     # PLOTTING (write this part yourself):
77     # - Figure 1: a grid of stem subplots, one per impulse response (4x2 for 7 systems).
78     #   Draw a vertical line at n = 0 on each so causality can be judged by eye, and put
79     #   "causal / non-causal" plus sum|h| in each subplot title. Build it with a loop
80     #   over the systems dictionary.
81     # - Figure 2: for the 5-point moving average, stem the worst-case input on one row
82     #   and its output on the row below; mark y[0] with a distinct colour and annotate
83     #   that it equals sum|h|.
84     # - Figure 3: plot sum|h| against N for r = 0.8, 1.0 and 1.2 on a semilogy axis,
85     #   with a horizontal dashed line at 1/(1-0.8) = 5 to show what the stable case
86     #   converges to. Add a legend naming each r.

```

```
87     # - xlabel "n" (or "N" on figure 3), grid on, save as p14_responses.png,  
88     #     p14_worst_case.png, p14_convergence.png.  
89  
90  
91 if __name__ == "__main__":  
92     main()
```

Problem 14 — Solution

Solution — approach and discussion

Constructing the worst case. At the chosen output index,

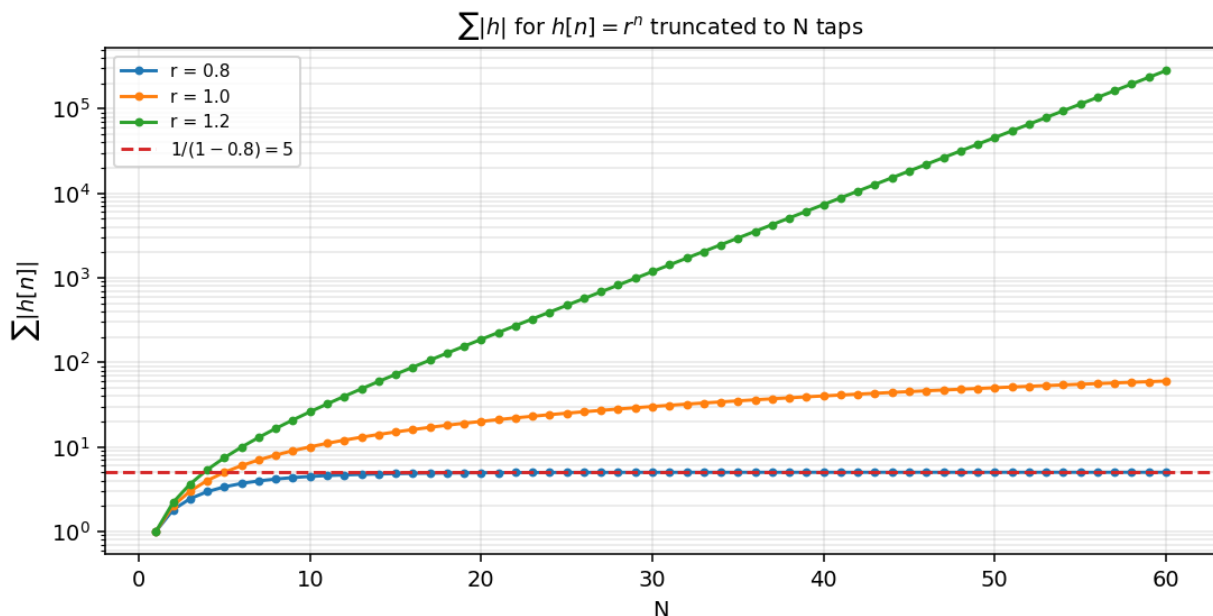
$$y[n_0] = \sum_k x[k] h[n_0 - k],$$

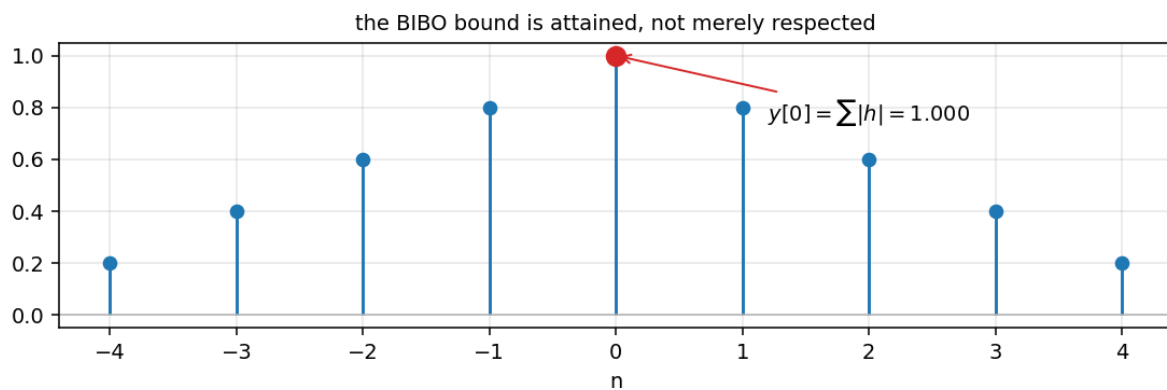
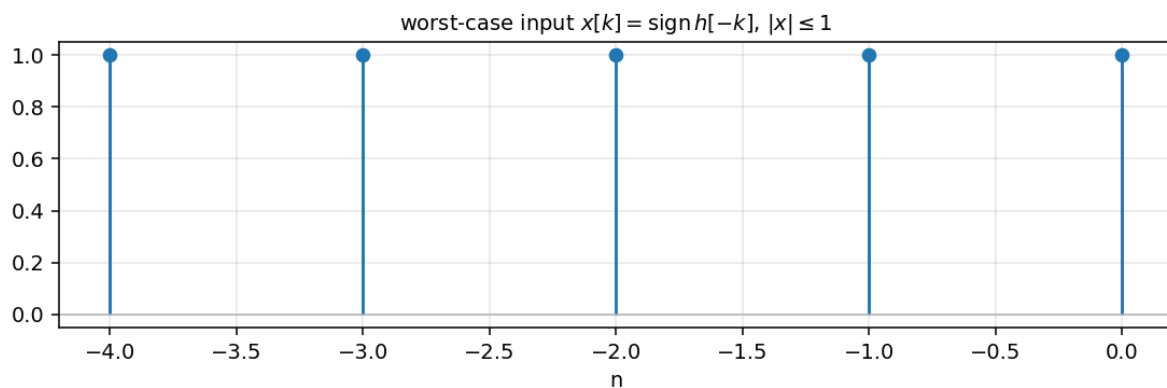
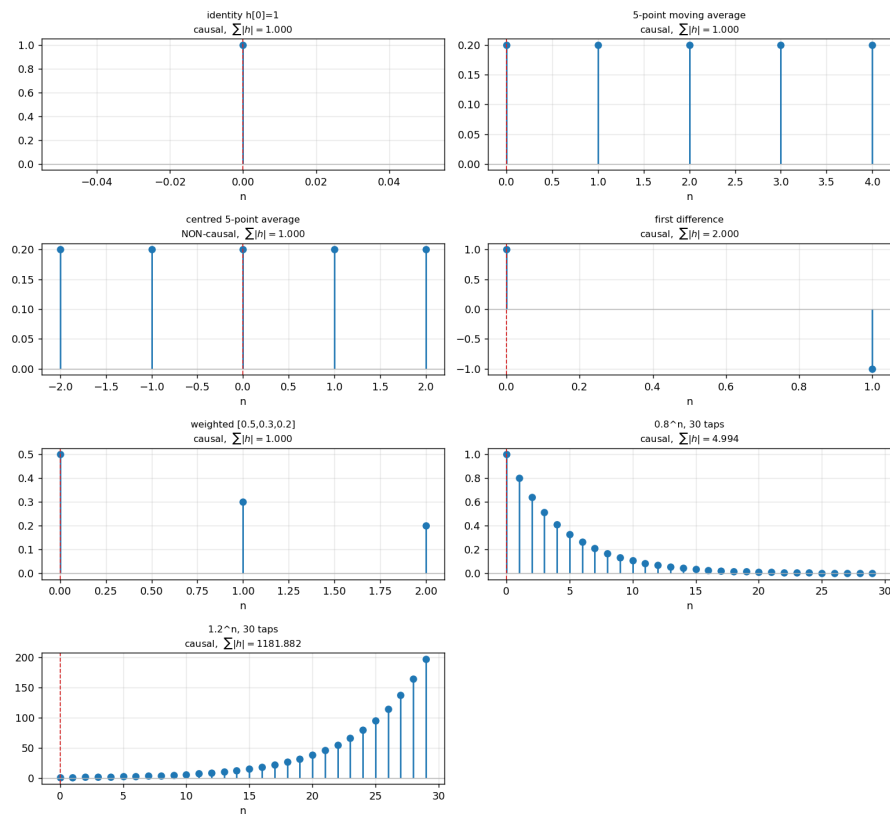
so setting $x[k] = \text{sign}(h[n_0 - k])$ turns every product into $|h[n_0 - k]|$ and gives $y[n_0] = \sum_n |h[n]|$. Only the indices with $h[n_0 - k] \neq 0$ matter, so the input need only be defined on $k \in [n_0 - n_{h,\max}, n_0 - n_{h,\min}]$ — note the reversal, which is easy to get backwards. The output confirms this: $y[0]$ equals $\sum |h|$ to the last digit for all seven systems.

Reading the table. The first difference $\{1, -1\}$ has $\sum |h| = 2$ but $\sum h = 0$: it can double a bounded input, yet it kills a constant entirely. The centred 5-point average is the only non-causal entry — and it is the same filter as the causal one, merely stored differently, which is the cleanest possible illustration that causality is a statement about *where* the taps sit, not what they are. The geometric response with $r = 1.2$ has $\sum |h| \approx 1182$ over 30 taps, and a random bounded input drives it to 440.

The truncation trap. Every finite-length impulse response is BIBO stable, since a finite sum of finite numbers is finite. So the $r = 1.2$ system *looks* stable at every N — it just has a big gain. Only the trend reveals the truth: as N grows, $\sum |h|$ converges to $1/(1 - 0.8) = 5$ for $r = 0.8$, grows linearly for $r = 1.0$, and explodes to 2.8×10^{24} by $N = 300$ for $r = 1.2$. Stability is a statement about the limit, and no single truncated model can settle it.

Figures produced by the plotting code





Reference implementation

```

2  Problem 14 - Reading causality, memory and BIBO stability off the impulse
3  response, and attaining the BIBO bound (SOLUTION)
4
5  causal      <=> h[n] = 0 for all n < 0
6  memoryless <=> h[n] = 0 for all n != 0
7  BIBO stable <=> sum_n |h[n]| < infinity, and then
8                  |y[n]| <= (max |x|) * sum_n |h[n]|
9  The bound is tight: choosing x[k] = sign(h[n0 - k]) makes every term of the
10 convolution sum positive, so |y[n0]| reaches sum |h| exactly.
11 """
12
13 import numpy as np
14 import matplotlib.pyplot as plt
15 from signal_lti import DiscreteSignal, LTISystem
16
17
18 def make_signal(start_time, values):
19     signal = DiscreteSignal(start_time, start_time + len(values) - 1)
20     for offset, value in enumerate(values):
21         signal.set_value_at_time(start_time + offset, value)
22     return signal
23
24
25 def is_causal(h, tolerance=1e-12):
26     return all(abs(h.get_value_at_time(n)) <= tolerance
27                for n in range(h.start_time, min(0, h.end_time + 1)))
28
29
30 def is_memoryless(h, tolerance=1e-12):
31     return all(abs(h.get_value_at_time(n)) <= tolerance
32                for n in range(h.start_time, h.end_time + 1) if n != 0)
33
34
35 def absolute_sum(h):
36     """sum |h[n]|: the BIBO gain of the system."""
37     return float(np.sum(np.abs(h.values)))
38
39
40 def dc_gain(h):
41     """sum h[n]: the gain applied to a constant input."""
42     return float(np.sum(h.values))
43
44
45 def worst_case_input(h, n0):
46     """
47     Bounded input with |x[k]| <= 1 that drives |y[n0]| to its largest value.
48     Only the samples k with h[n0-k] != 0 matter, i.e. k in [n0-h_end, n0-h_start].
49     """
50     x = DiscreteSignal(n0 - h.end_time, n0 - h.start_time)
51     for k in range(x.start_time, x.end_time + 1):
52         tap = h.get_value_at_time(n0 - k)
53         x.set_value_at_time(k, 1.0 if tap >= 0 else -1.0)
54     return x
55
56
57 def geometric(ratio, length):
58     """h[n] = ratio^n for n = 0..length-1."""
59     return make_signal(0, [ratio ** n for n in range(length)])
60
61
62 def main():
63     systems = {
64         "identity h[0]=1": make_signal(0, [1.0]),
65         "5-point moving average": make_signal(0, [0.2] * 5),
66         "centred 5-point average": make_signal(-2, [0.2] * 5),
67         "first difference": make_signal(0, [1.0, -1.0]),
68         "weighted [0.5,0.3,0.2]": make_signal(0, [0.5, 0.3, 0.2]),
69         "0.8^n, 30 taps": geometric(0.8, 30),
70         "1.2^n, 30 taps": geometric(1.2, 30),
71     }
72
73     # ---- Task 1: classify each system ----
74     print(f"{'system':>25} {'causal':>7} {'memoryless':>11} {'sum|h|':>10} {'sum h':>8}")
75     for name, h in systems.items():

```

```

76     print(f"{name:>25} {str(is_causal(h)):>7} {str(is_memoryless(h)):>11} "
77           f"{absolute_sum(h):10.4f} {dc_gain(h):8.4f}")
78     print()
79
80     # ---- Task 2: verify the bound is tight ----
81     print("attaining the BIBO bound with x[k] = sign(h[n0-k]), n0 = 0:")
82     print(f"{'system':>25} {'sum|h|':>10} {'max|y|':>10} {'y[0]':>10}")
83     for name, h in systems.items():
84         x = worst_case_input(h, 0)
85         y = LTISystem(h).output(x)
86         print(f"{name:>25} {absolute_sum(h):10.4f} "
87               f"{float(np.max(np.abs(y.values))):10.4f} "
88               f"{y.get_value_at_time(0):10.4f}")
89     print()
90
91     # ---- Task 3: an arbitrary bounded input must respect the bound ----
92     rng = np.random.default_rng(7)
93     x_random = make_signal(-5, rng.uniform(-1.0, 1.0, 40))
94     print("random input with |x[n]| <= 1:")
95     print(f"{'system':>25} {'max|y|':>10} {'bound sum|h|':>14} {'within bound':>14}")
96     for name, h in systems.items():
97         y = LTISystem(h).output(x_random)
98         peak = float(np.max(np.abs(y.values)))
99         bound = absolute_sum(h)
100        print(f"{name:>25} {peak:10.4f} {bound:14.4f} "
101              f"{str(peak <= bound + 1e-12):>14}")
102    print()
103
104    # ---- Task 4: what happens as the truncation is lifted ----
105    print("sum |h| for h[n] = r^n truncated to N taps:")
106    print(f"{'N':>5} {'r = 0.8':>12} {'r = 1.0':>12} {'r = 1.2':>12}")
107    for length in (10, 30, 100, 300):
108        row = [absolute_sum(geometric(ratio, length)) for ratio in (0.8, 1.0, 1.2)]
109        print(f"{length:5d} {row[0]:12.4f} {row[1]:12.4f} {row[2]:12.4e}")
110    print()
111    print("Every finite-length h is BIBO stable, so truncation always hides")
112    print("instability. Only the limit tells the truth: for r = 0.8 the sum")
113    print("converges to 1/(1-0.8) = 5, while for r >= 1 it grows without bound.")
114
115    # Figures described below; the implementation follows.
116    # - Figure 1: a grid of stem subplots, one per impulse response (4x2 for 7 systems).
117    #   Draw a vertical line at n = 0 on each so causality can be judged by eye, and put
118    #   "causal / non-causal" plus sum|h| in each subplot title. Build it with a loop
119    #   over the systems dictionary.
120    # - Figure 2: for the 5-point moving average, stem the worst-case input on one row
121    #   and its output on the row below; mark y[0] with a distinct colour and annotate
122    #   that it equals sum|h|.
123    # - Figure 3: plot sum|h| against N for r = 0.8, 1.0 and 1.2 on a semilogy axis,
124    #   with a horizontal dashed line at 1/(1-0.8) = 5 to show what the stable case
125    #   converges to. Add a legend naming each r.
126    # - xlabel "n" (or "N" on figure 3), grid on, save as p14_responses.png,
127    #   p14_worst-case.png, p14_convergence.png.
128
129    # ---- Figures ----
130    # Figure 1: every impulse response, judged by eye. Built by looping over the
131    # dictionary so adding an eighth system needs no new code.
132    figure, axes = plt.subplots(4, 2, figsize=(12, 11))
133    flat_axes = axes.ravel()
134    for axis, (name, response) in zip(flat_axes, systems.items()):
135        axis.stem(response.times(), response.values, basefmt=" ")
136        axis.axvline(0, color="C3", linestyle="--", linewidth=1.0)
137        axis.axhline(0, color="0.7", linewidth=0.8)
138        label = "causal" if is_causal(response) else "NON-causal"
139        axis.set_title(f"{name}\n{n|label}, $\sum|h| = {absolute_sum(response):.3f}$",
140                      fontsize=9)
141        axis.set_xlabel("n")
142        axis.grid(True, alpha=0.3)
143    flat_axes[-1].axis("off")
144    figure.tight_layout()
145    figure.savefig("p14_responses.png", dpi=130)
146    plt.close(figure)
147
148    # Figure 2: the input that attains the bound, and what it produces.
149    chosen = systems["5-point moving average"]

```

```

150     x_worst = worst_case_input(chosen, 0)
151     y_worst = LTISystem(chosen).output(x_worst)
152
153     figure, axes = plt.subplots(2, 1, figsize=(8, 5.5))
154     axes[0].stem(x_worst.times(), x_worst.values, basefmt=" ")
155     axes[0].set_title(r"worst-case input  $x[k] = \mathrm{sign}\{h[-k]\}$ ,  $|x| \leq 1$ ",
156                      fontsize=10)
157     markers, stems, _ = axes[1].stem(y_worst.times(), y_worst.values, basefmt=" ")
158     axes[1].plot([0], [y_worst.get_value_at_time(0)], "o", color="C3", markersize=9)
159     axes[1].annotate(f" $y[0] = \sum |h| = \{absolute\_sum(chosen):.3f\}$ ",
160                     xy=(0, y_worst.get_value_at_time(0)),
161                     xytext=(1.2, y_worst.get_value_at_time(0) * 0.75), fontsize=10,
162                     arrowprops=dict(arrowstyle="->", color="C3"))
163     axes[1].set_title("the BIBO bound is attained, not merely respected", fontsize=10)
164     for axis in axes:
165         axis.axhline(0, color="0.7", linewidth=0.8)
166         axis.set_xlabel("n")
167         axis.grid(True, alpha=0.3)
168     figure.tight_layout()
169     figure.savefig("p14_worst_case.png", dpi=140)
170     plt.close(figure)
171
172     # Figure 3: only the trend in N reveals stability.
173     lengths = list(range(1, 61))
174     figure, axis = plt.subplots(figsize=(8, 4.2))
175     for ratio in (0.8, 1.0, 1.2):
176         sums = [absolute_sum(geometric(ratio, length)) for length in lengths]
177         axis.semilogy(lengths, sums, marker=".", label=f"r = {ratio}")
178     axis.axhline(1 / (1 - 0.8), color="C3", linestyle="--",
179                 label=r" $1/(1-0.8) = 5$ ")
180     axis.set_title(r" $\sum |h|$  for  $h[n] = r^n$  truncated to N taps", fontsize=10)
181     axis.set_xlabel("N")
182     axis.set_ylabel(r" $\sum |h[n]|$ ")
183     axis.grid(True, alpha=0.3, which="both")
184     axis.legend(fontsize=8)
185     figure.tight_layout()
186     figure.savefig("p14_convergence.png", dpi=140)
187     plt.close(figure)
188     print("\nsaved p14_responses.png, p14_worst_case.png, p14_convergence.png")
189
190 if __name__ == "__main__":
191     main()

```

Expected console output

```

          system causal memoryless    sum|h|    sum h
identity h[0]=1   True      True    1.0000  1.0000
5-point moving average   True      False    1.0000  1.0000
centred 5-point average False      False    1.0000  1.0000
first difference        True      False    2.0000  0.0000
weighted [0.5,0.3,0.2]   True      False    1.0000  1.0000
0.8^n, 30 taps          True      False    4.9938  4.9938
1.2^n, 30 taps          True      False  1181.8816 1181.8816

attaining the BIBO bound with x[k] = sign(h[n0-k]), n0 = 0:
          system    sum|h|    max|y|    y[0]
identity h[0]=1    1.0000    1.0000    1.0000
5-point moving average    1.0000    1.0000    1.0000
centred 5-point average    1.0000    1.0000    1.0000
first difference        2.0000    2.0000    2.0000
weighted [0.5,0.3,0.2]    1.0000    1.0000    1.0000
0.8^n, 30 taps        4.9938    4.9938    4.9938
1.2^n, 30 taps      1181.8816  1181.8816  1181.8816

random input with |x[n]| <= 1:
          system    max|y|    bound sum|h|    within bound
identity h[0]=1    0.9925        1.0000        True
5-point moving average    0.5811        1.0000        True
centred 5-point average    0.5811        1.0000        True
first difference        1.7366        2.0000        True

```

weighted [0.5,0.3,0.2]	0.7015	1.0000	True
0.8 ⁿ , 30 taps	2.0494	4.9938	True
1.2 ⁿ , 30 taps	440.3594	1181.8816	True

sum |h| for h[n] = rⁿ truncated to N taps:

N	r = 0.8	r = 1.0	r = 1.2
10	4.4631	10.0000	2.5959e+01
30	4.9938	30.0000	1.1819e+03
100	5.0000	100.0000	4.1409e+08
300	5.0000	300.0000	2.8402e+24

Every finite-length h is BIBO stable, so truncation always hides instability. Only the limit tells the truth: for r = 0.8 the sum converges to 1/(1-0.8) = 5, while for r ≥ 1 it grows without bound.

saved p14_responses.png, p14_worst_case.png, p14_convergence.png

Problem 15. Sinusoids are eigenfunctions: measuring the frequency response

Total Marks: 10
Suggested Time: 30 minutes
New

Feed a sinusoid into an LTI system and the same sinusoid comes out, rescaled and shifted but never distorted:

$$x[n] = \cos(\omega n) \implies y[n] = |H(\omega)| \cos(\omega n + \angle H(\omega)), \quad H(\omega) = \sum_m h[m] e^{-j\omega m}.$$

This is the bridge from convolution to the frequency domain, and it can be verified with nothing but the convolution code you already have. You will measure the gain and phase from the output samples and check them against $H(\omega)$ computed from the taps.

Tasks

1. Implement `frequency_response(h, omega)` directly from the sum. Do not use an FFT.
2. Implement `steady_state_window(x, h)`, the indices where the output uses only stored samples of x . Derive it from the two ranges.
3. Implement `fit_sinusoid`: least-squares fit of $y[n] \approx A \cos \omega n + B \sin \omega n$ on that window, then $|H| = \sqrt{A^2 + B^2}$ and $\angle H = \text{atan2}(-B, A)$.
4. For the 3-point moving average and the first difference, tabulate measured and calculated gain and phase at $\omega/\pi \in \{0, \frac{1}{8}, \frac{1}{4}, \frac{1}{3}, \frac{1}{2}, \frac{2}{3}, \frac{3}{4}, 1\}$. Skip the phase where the gain is essentially zero.
5. Report the special values at $\omega = 0$, $\omega = \pi$, and $\omega = 2\pi/3$ for the moving average, and connect $\sum_n h[n]$ to $H(0)$.

Marks Distribution

Component	Marks
<code>frequency_response</code> from the definition	2
Correct steady-state window (transients excluded)	3
Amplitude and phase extraction	3
Tables for both filters	1
Interpretation: lowpass vs highpass, and the null	1
Total	10

Template

```

1  """
2  Problem 15 - Sinusoids are eigenfunctions: measuring the frequency response
3  (TEMPLATE)
4
5      x[n] = cos(w n) ==> y[n] = |H(w)| cos(w n + angle H(w))
6      H(w) = sum_m h[m] exp(-j w m)
7  """
8
9  import numpy as np
10 from signal_lti import DiscreteSignal, LTISystem
11

```

```

12
13 def make_signal(start_time, values):
14     # TODO
15     raise NotImplementedError
16
17
18 def frequency_response(h, omega):
19     """ $H(e^{j\omega}) = \sum_m h[m] \exp(-j\omega m)$ , straight from the definition."""
20     # TODO
21     raise NotImplementedError
22
23
24 def cosine_input(omega, start_time, end_time):
25     # TODO
26     raise NotImplementedError
27
28
29 def steady_state_window(x, h):
30     """
31     Indices where the output uses ONLY stored samples of x, so the start-up and
32     run-out transients are excluded. Derive it from the two ranges.
33     """
34     # TODO
35     raise NotImplementedError
36
37
38 def fit_sinusoid(y, omega, start_time, end_time):
39     """
40     Least-squares fit of  $y[n] \sim A \cos(\omega n) + B \sin(\omega n)$  on the window
41     (np.linalg.lstsq is fine - it is not a convolution).
42     Then  $|H| = \sqrt{A^2 + B^2}$  and angle  $H = \text{atan2}(-B, A)$ .
43     """
44     # TODO
45     raise NotImplementedError
46
47
48 def main():
49     frequencies = [0.0, np.pi / 8, np.pi / 4, np.pi / 3, np.pi / 2,
50                   2 * np.pi / 3, 3 * np.pi / 4, np.pi]
51
52     # TODO 1: for the 3-point moving average, and then for the first difference,
53     #         print a table of measured gain / calculated gain / measured phase /
54     #         calculated phase at every frequency in the list. Skip the phase
55     #         comparison where the gain is essentially zero.
56     # TODO 2: print the special values: both filters at  $\omega = 0$  and  $\omega = \pi$ , and the
57     #         moving average at  $\omega = 2\pi/3$ .
58     # TODO 3: state in a printed line which filter is lowpass and which is highpass,
59     #         and connect that to  $\sum h[n] = H(0)$ .
60
61     # PLOTTING (write this part yourself):
62     # - Figure 1: sweep  $\omega$  over  $0.. \pi$  with np.linspace(0, np.pi, 400), evaluate
63     #   frequency_response at each point, and plot  $|H|$  against  $\omega/\pi$  for both filters
64     #   on one axes. Overlay the measured gains from the table as scatter markers so the
65     #   agreement is visible. Add a legend.
66     # - Figure 2: the same sweep for the phase, using np.angle. Plot  $\text{phase}/\pi$  against
67     #    $\omega/\pi$ , again with the measured points scattered on top.
68     # - Figure 3: for one interesting frequency ( $\omega = 2\pi/3$  on the moving average) plot
69     #    $x[n]$  and  $y[n]$  over the first ~40 samples on the same axes as lines with markers,
70     #   so the complete cancellation after the start-up transient is obvious.
71     # - xlabel " $\omega / \pi$ " on figures 1-2 and "n" on figure 3, grid on, legends,
72     #   save as p15_magnitude.png, p15_phase.png, p15_null.png.
73
74
75 if __name__ == "__main__":
76     main()

```

Problem 15 — Solution

Solution — approach and discussion

The steady-state window is the whole difficulty. The output $y[n]$ only equals $|H| \cos(\omega n + \angle H)$ once the filter is fully overlapped with the stored input, that is for

$$n \in [n_{x,\min} + n_{h,\max}, n_{x,\max} + n_{h,\min}].$$

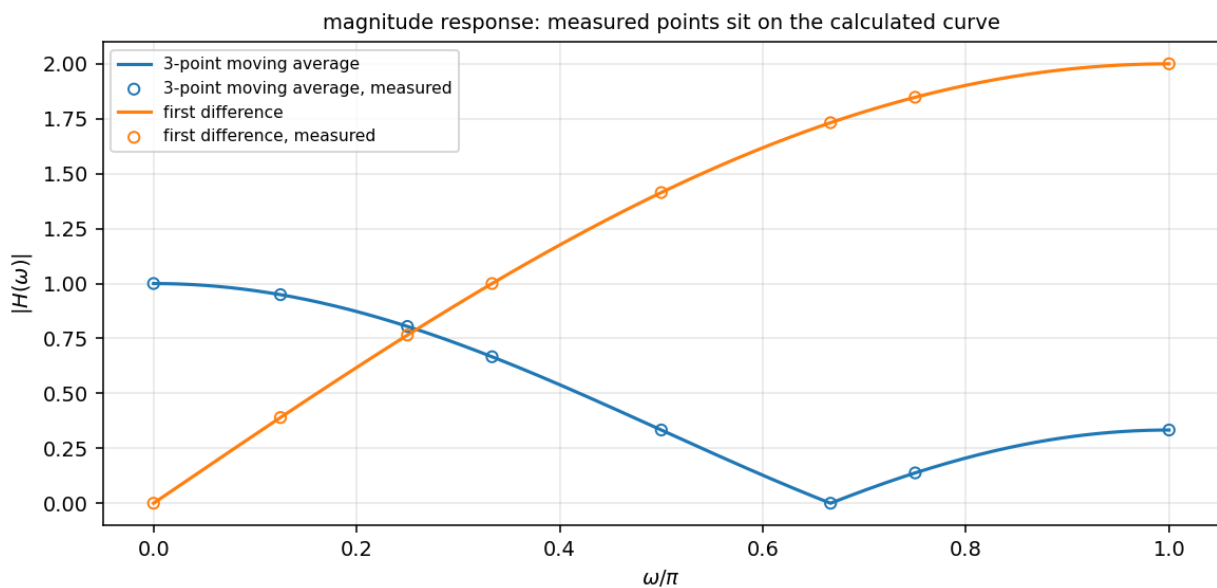
Fitting outside it mixes in the start-up and run-out transients and corrupts both estimates. Restricted correctly, the measured values match the calculated ones to four decimal places at every frequency in the table — which is the numerical statement that sinusoids really are eigenfunctions.

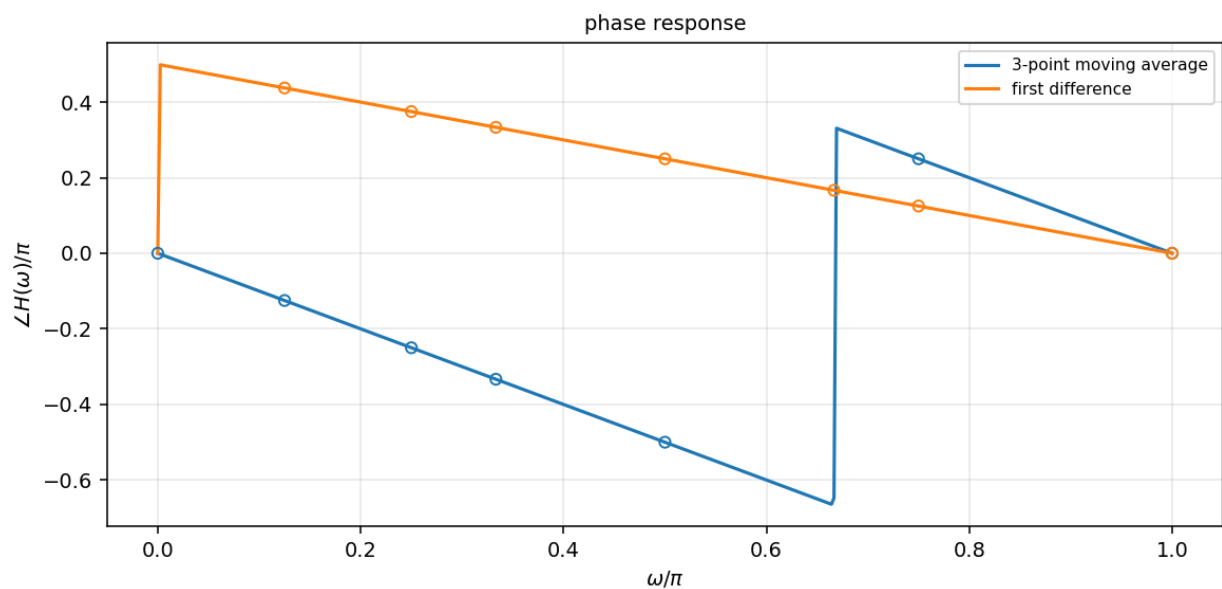
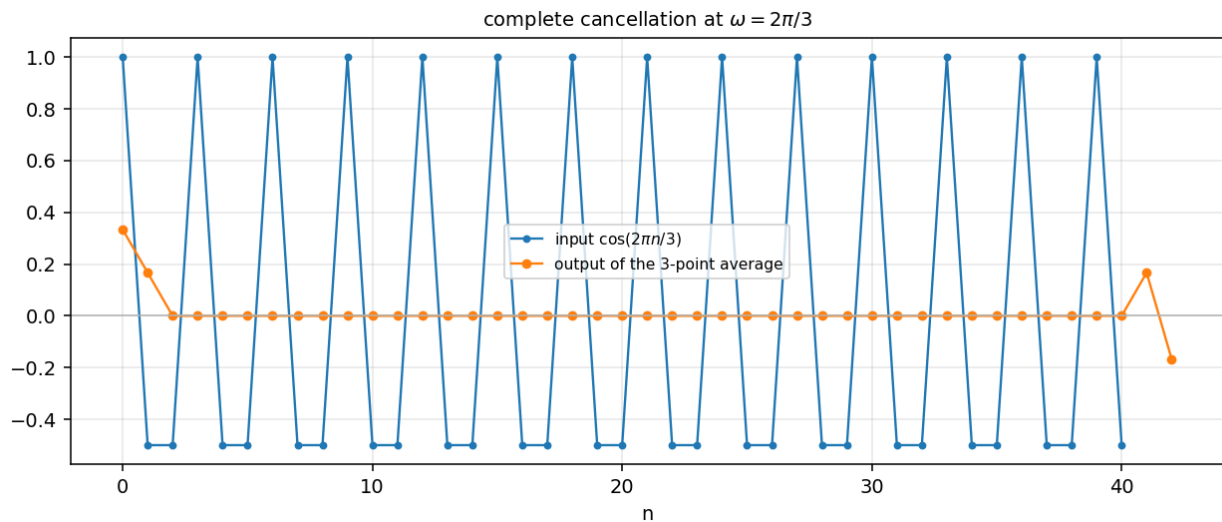
Extracting the phase. Expanding $|H| \cos(\omega n + \phi) = |H| \cos \phi \cos \omega n - |H| \sin \phi \sin \omega n$ identifies $A = |H| \cos \phi$ and $B = -|H| \sin \phi$, hence $\phi = \text{atan2}(-B, A)$. The minus sign is easy to lose. Where the gain is zero the phase is undefined, so the comparison is skipped rather than reported as a mismatch.

Reading the results. The moving average has $|H(0)| = 1$ and $|H(\pi)| = 1/3$: lowpass. The first difference has $|H(0)| = 0$ and $|H(\pi)| = 2$: highpass. Both facts are already visible in the taps, since $H(0) = \sum_n h[n]$ — a filter whose taps sum to zero blocks a constant, and one whose taps sum to one passes it unchanged.

The most striking entry is $\omega = 2\pi/3$, where the 3-point average gives $|H| = 10^{-16}$: a complete null. Three samples spaced one third of a period apart are 120° apart on the unit circle and sum to exactly zero, so the filter annihilates that frequency outright. The output is not small, it is zero — and plotting x and y together at that frequency makes it obvious in a way the table does not.

Figures produced by the plotting code





Reference implementation

```

1  """
2  Problem 15 - Sinusoids are eigenfunctions: measuring the frequency response
3  (SOLUTION)
4
5  For an LTI system, a sinusoid in gives the same sinusoid out, only rescaled and
6  delayed:
7       $x[n] = \cos(\omega n) \implies y[n] = |H(\omega)| \cos(\omega n + \angle H(\omega))$ ,
8       $H(\omega) = \sum_m h[m] \exp(-j \omega m)$ .
9  This solution feeds several frequencies through the convolution, fits the
10 amplitude and phase of the steady-state output, and checks them against  $H(\omega)$ 
11 evaluated directly from the taps.
12 """
13
14 import numpy as np
15 import matplotlib.pyplot as plt
16 from signal_lti import DiscreteSignal, LTISystem
17
18
19 def make_signal(start_time, values):

```



```

94
95 moving_average_3 = make_signal(0, [1 / 3] * 3)
96 first_difference = make_signal(0, [1.0, -1.0])
97
98 analyse("3-point moving average", moving_average_3, frequencies)
99 analyse("first difference", first_difference, frequencies)
100
101 # ---- The two extremes, in words ----
102 print(f"moving average at w = 0 : |H| = "
103       f"{abs(frequency_response(moving_average_3, 0.0)):.4f} (passes DC)")
104 print(f"moving average at w = pi : |H| = "
105       f"{abs(frequency_response(moving_average_3, np.pi)):.4f} (heavily attenuated)")
106 print(f"moving average at w = 2pi/3: |H| = "
107       f"{abs(frequency_response(moving_average_3, 2 * np.pi / 3)):.2e} (exact null)")
108 print(f"first difference at w = 0 : |H| = "
109       f"{abs(frequency_response(first_difference, 0.0)):.4f} (blocks DC)")
110 print(f"first difference at w = pi: |H| = "
111       f"{abs(frequency_response(first_difference, np.pi)):.4f} (maximum gain)")
112 print()
113 print("A moving average is a lowpass filter and a first difference is a")
114 print("highpass filter, and both facts are already visible in sum h[n]:")
115 print("sum h = H(0), so a zero DC sum is exactly a blocked constant.")
116 print()
117 print("Note the null: at w = 2pi/3 a 3-point average annihilates the input")
118 print("completely, because the three samples it averages are spaced one third")
119 print("of a period apart and add to zero.")
120
121 # Figures described below; the implementation follows.
122 # - Figure 1: sweep omega over 0..pi with np.linspace(0, np.pi, 400), evaluate
123 #   frequency_response at each point, and plot |H| against omega/pi for both filters
124 #   on one axes. Overlay the measured gains from the table as scatter markers so the
125 #   agreement is visible. Add a legend.
126 # - Figure 2: the same sweep for the phase, using np.angle. Plot phase/pi against
127 #   omega/pi, again with the measured points scattered on top.
128 # - Figure 3: for one interesting frequency (w = 2pi/3 on the moving average) plot
129 #   x[n] and y[n] over the first ~40 samples on the same axes as lines with markers,
130 #   so the complete cancellation after the start-up transient is obvious.
131 # - xlabel "omega / pi" on figures 1-2 and "n" on figure 3, grid on, legends,
132 #   save as p15_magnitude.png, p15_phase.png, p15_null.png.
133
134 # ---- Figures ----
135 # Sweep omega continuously for the curves, and overlay the measured points.
136 sweep = np.linspace(0, np.pi, 400)
137 filters = [("3-point moving average", moving_average_3),
138           ("first difference", first_difference)]
139
140 def measure(h, omega):
141     """Re-run the measurement so the scatter points come from the data, not the formula."""
142     x = cosine_input(omega, 0, 120)
143     y = LTISystem(h).output(x)
144     return fit_sinusoid(y, omega, *steady_state_window(x, h))
145
146 # Figure 1: magnitude response.
147 figure, axis = plt.subplots(figsize=(8.5, 4.2))
148 for index, (name, h) in enumerate(filters):
149     curve = [abs(frequency_response(h, w)) for w in sweep]
150     axis.plot(sweep / np.pi, curve, color=f"C{index}", linewidth=1.6, label=name)
151     measured = [measure(h, w)[0] for w in frequencies]
152     axis.scatter(np.array(frequencies) / np.pi, measured, color=f"C{index}",
153                marker="o", zorder=3, s=28,
154                facecolors="none", label=f"{name}, measured")
155 axis.set_title("magnitude response: measured points sit on the calculated curve",
156               fontsize=10)
157 axis.set_xlabel(r"$\omega / \pi$")
158 axis.set_ylabel(r"$|H(\omega)|$")
159 axis.grid(True, alpha=0.3)
160 axis.legend(fontsize=8)
161 figure.tight_layout()
162 figure.savefig("p15_magnitude.png", dpi=140)
163 plt.close(figure)
164
165 # Figure 2: phase response. Skip measured points where the gain is ~0,
166 # because the phase is undefined there.
167 figure, axis = plt.subplots(figsize=(8.5, 4.2))

```

```

168     for index, (name, h) in enumerate(filters):
169         curve = [np.angle(frequency_response(h, w)) for w in sweep]
170         axis.plot(sweep / np.pi, np.array(curve) / np.pi, color=f"C{index}",
171                  linewidth=1.6, label=name)
172         usable = [w for w in frequencies if abs(frequency_response(h, w)) > 1e-6]
173         measured = [wrap_to_pi(measure(h, w)[1]) / np.pi for w in usable]
174         axis.scatter(np.array(usable) / np.pi, measured, color=f"C{index}",
175                     marker="o", zorder=3, s=28, facecolors="none")
176     axis.set_title("phase response", fontsize=10)
177     axis.set_xlabel(r"$\omega / \pi$")
178     axis.set_ylabel(r"$\angle H(\omega) / \pi$")
179     axis.grid(True, alpha=0.3)
180     axis.legend(fontsize=8)
181     figure.tight_layout()
182     figure.savefig("p15_phase.png", dpi=140)
183     plt.close(figure)
184
185     # Figure 3: the null. After the start-up transient the output is exactly zero.
186     omega_null = 2 * np.pi / 3
187     x_null = cosine_input(omega_null, 0, 40)
188     y_null = LTISystem(moving_average_3).output(x_null)
189
190     figure, axis = plt.subplots(figsize=(9, 4))
191     axis.plot(x_null.times(), x_null.values, marker=".", linewidth=1.2,
192              label=r"input $\cos(2\pi n/3)$")
193     axis.plot(y_null.times(), y_null.values, marker="o", markersize=4, linewidth=1.2,
194              label="output of the 3-point average")
195     axis.axhline(0, color="0.7", linewidth=0.8)
196     axis.set_title(r"complete cancellation at $\omega = 2\pi/3$", fontsize=10)
197     axis.set_xlabel("n")
198     axis.grid(True, alpha=0.3)
199     axis.legend(fontsize=8)
200     figure.tight_layout()
201     figure.savefig("p15_null.png", dpi=140)
202     plt.close(figure)
203     print("\nsaved p15_magnitude.png, p15_phase.png, p15_null.png")
204
205 if __name__ == "__main__":
206     main()

```

Expected console output

```

--- 3-point moving average: taps over [0, 2] = +0.3333 +0.3333 +0.3333
  w/pi  |H| meas  |H| calc  phase meas  phase calc
0.000   1.0000   1.0000   0.0000   0.0000
0.125   0.9493   0.9493  -0.3927  -0.3927
0.250   0.8047   0.8047  -0.7854  -0.7854
0.333   0.6667   0.6667  -1.0472  -1.0472
0.500   0.3333   0.3333  -1.5708  -1.5708
0.667   0.0000   0.0000    n/a      n/a
0.750   0.1381   0.1381   0.7854   0.7854
1.000   0.3333   0.3333   0.0000   0.0000

--- first difference: taps over [0, 1] = +1.0000 -1.0000
  w/pi  |H| meas  |H| calc  phase meas  phase calc
0.000   0.0000   0.0000    n/a      n/a
0.125   0.3902   0.3902   1.3744   1.3744
0.250   0.7654   0.7654   1.1781   1.1781
0.333   1.0000   1.0000   1.0472   1.0472
0.500   1.4142   1.4142   0.7854   0.7854
0.667   1.7321   1.7321   0.5236   0.5236
0.750   1.8478   1.8478   0.3927   0.3927
1.000   2.0000   2.0000   0.0000   0.0000

moving average at w = 0 : |H| = 1.0000 (passes DC)
moving average at w = pi : |H| = 0.3333 (heavily attenuated)
moving average at w = 2pi/3: |H| = 1.24e-16 (exact null)
first difference at w = 0 : |H| = 0.0000 (blocks DC)
first difference at w = pi : |H| = 2.0000 (maximum gain)

```

A moving average is a lowpass filter and a first difference is a highpass filter, and both facts are already visible in $\sum h[n]$: $\sum h = H(0)$, so a zero DC sum is exactly a blocked constant.

Note the null: at $w = 2\pi/3$ a 3-point average annihilates the input completely, because the three samples it averages are spaced one third of a period apart and add to zero.

saved p15_magnitude.png, p15_phase.png, p15_null.png